

Structural Polyglot Orchestration, Native O-core, and World Execution

A technical account of the language, compiler, runtime, and World architecture

Implementation snapshot
github.com/lostadi/OSTADIX
Source snapshot `f472c983deb83d260d4d52e77aa4dd8740710dc5`
Package coordinate `0.4.0 Unreleased`

Lee Daghlar Ostadi

ORCID 0009-0001-6380-9558

28 October 2026

Abstract

I designed Ostadix-lang around a direct premise: the evaluator selected for an expression belongs in the expression's syntax. A hosted source file expresses that choice through balanced, language-indexed forms such as `python^(...)_python`. Nested evaluators are structural nodes, and their results cross language boundaries through OValue, a typed interchange algebra. The authoritative Rust runtime parses these forms into a small structural AST, lowers every normal execution through OIR, freezes backend metadata in a validated execution plan, projects the plan into a state-complete HGraph, and coordinates backend actors through canonical CBOR. Persistent language environments, quotation and meta-evaluation, deferred requests, coordination groups, project lifting, and hosted live-system supervision make evaluator composition a complete execution model.

I developed O-core as the native systems lineage of this architecture. It is a statically typed freestanding language whose AST–typed HIR–SSA MIR pipeline emits owned x86-64, AArch64 and RV64GC assembly and ELF objects. A separate native-module pipeline imports original frontend LLVM, verifies the entire normalized module, and emits supported integer/pointer, variadic, atomic, TLS and exception profiles. Compiler-owned source recipes and reusable module sets advance the original-engine lowering objective; complete linked runtime/compiler closure remains a separate, unfinished qualification. Its compiler, runtime modules, capability system, and World-admission machinery carry the architecture from evaluator orchestration into native systems construction. Together, the hosted and native lineages span polyglot expression composition, state-complete execution graphs, freestanding compilation, and authority-bearing World execution.

In this paper I present the language system at a pinned implementation snapshot, with separately dated execution evidence. I explain the hosted grammar and semantics; the OValue interchange algebra; backend actors and authority; OIR, execution plans, and HGraph scheduling; project lifting and route execution; the O-core compiler; and the Live-World and KernelWorld layers. I then trace the current execution constitution through Graph V2, Evidence V6, Admission V6, Why V2, the unified lowercase `o` front door, reciprocal node pairing, Project Mesh, Information, Registry V1, and the canonical World formats. This revision also covers operation realization planning, physical execution profiles, native object ownership, supported actor migration, the shared source-first human/MCP interface, and the fixes that restore hosted edition agreement and the 26-gate portable native aggregate. The central result is a coherent architecture in which evaluator identity is syntax, cross-language state is typed, effects become explicit graph resources, and execution authority is admitted through exact verified records.

Keywords: structural polyglot programming, evaluator composition, typed interchange values, hypergraph execution, capability systems, freestanding systems language, multikernel research.

Core contributions

One language system, eight implemented contributions

1. **Evaluator-indexed syntax.** Every typed expression names its evaluator in the grammar, so heterogeneous source remains structurally visible in one recursive program tree.
2. **Typed inter-language state.** OVALUE gives every evaluator a common algebra for values, requests, groups, scopes, and capabilities; separate judgments describe modeled value crossings and renderer projections.
3. **State-complete graph execution.** OIR and HGRAPH represent values, completion, actor mutation, and world effects as explicit dependencies.
4. **Whole-project composition.** Deterministic project bundles preserve files, routes, guards, artifacts, and alternative policies in a reversible language artifact.
5. **Native compilation and World construction.** Three owned ISA backends, verified original-module import and compiler-owned source recipes connect implementation code to native artifacts. The kernel and KernelWorld extend explicit structure into freestanding services and authority-bearing admission.
6. **Execution constituted by evidence.** Graph V2 binds the solved typed program; Evidence V6 binds its executable environment; Admission V6 rederives and materializes the authority consumed by execution.
7. **Capacity-first project placement.** Project Mesh moves a source-closed project branch with its prerequisites, ranks paired nodes by usable capacity, and records generation-fenced attempts.
8. **Information and World records.** The Information Kernel, eight bounded bridge projections, provenance admission, Registry V1, and the OWIDENT, OWPROTO, OWVALUE, and OWRECEIPT codecs give execution state canonical identities that can cross implementation boundaries.

Audience and reading paths

I wrote this paper for programming-language researchers, compiler and runtime engineers, systems and operating-system researchers, and technical readers evaluating Ostadix-lang as an execution substrate. It assumes familiarity with abstract syntax trees, intermediate representations, runtime processes, SSA, capability systems, and virtual-machine execution. Every Ostadix-specific concept is defined where it first enters the architecture.

The paper develops one continuous argument across three scales. The hosted language makes evaluator identity part of syntax and carries values between evaluators through a typed algebra. The compiler turns that structure into an effect-aware execution graph. O-CORE and the World layers carry the same explicit representation discipline into freestanding and authority-bearing execution.

Every reader can begin with Section 2, which presents the whole system and names its major layers. The paths below then follow each reader's primary technical question.

Reader	Recommended path	Central question
Programming-language researcher	Sections 4–6, Section 8, and Section 16.	How does evaluator-indexed syntax become a typed, compositional semantics?
Compiler or runtime engineer	Sections 3, 6–10, followed by Section 14.	How does source become a validated, scheduled, executable graph?
Distributed-runtime or agent-tool engineer	Sections 3, 9, and 14, followed by Section H.	How do local intent, reciprocal node trust, placement, Information, and World records preserve one execution identity?
Systems or OS researcher	Sections 11–13, followed by Sections 15 and 16.	How does the architecture reach freestanding code, capabilities, and World admission?
Technical adopter or evaluator	Sections 4, 9, 10, and 14, followed by the compact reference and reproduction appendices.	What can the implementation express, build, inspect, and execute today?

How to read the notation, figures, and evidence

Monospaced text denotes source forms, commands, evaluator identities, resource names, and emitted compiler labels. OVALUE, OIR, HGRAPH, `HostWorld`, and `ActorState` name concrete implementation types or compiler concepts. Architecture figures generally flow from source on the left toward execution or publication on the right. The complete graph in Section 9 has its own visual grammar and a seven-pass reading method immediately before the compiler-emitted figure.

Source citations link to implementation files at the exact source commit on the title page. These pinned file references replace the older edition's line intervals, which changed as the implementation evolved. Validation tables distinguish checks performed for this revision, retained dated transcripts, and source-defined gates that require their own execution. This lets a reader move from thesis, to mechanism, to defining code, to executable evidence without changing terminology.

Contents

Audience and reading paths ii

1 Implementation basis and method 1

1.1 Source coordinate . . . 1

1.2 Contribution map . . . 1

1.3 Architectural path through the implementation . . . 2

2 The language system in one view 2

2.1 Hosted orchestration . . . 2

2.2 Project execution . . . 3
2.3 Native systems execution . . . 3
2.4 World layers . . . 3

3 The current execution constitution 3

3.1 Independent `ostadix-api` ownership . . . 3
3.2 The unified lowercase `o` front door . . . 3
3.3 Graph V2, Evidence V6, Admission V6, and Why V2 . . . 5
3.4 A shared native interface for humans and agents . . . 6
3.5 Computation identity and declared realization planning . . . 7
3.6 Physical operation and transfer graphs . . . 8
3.7 Reciprocal LAN pairing and pinned TLS . . . 8
3.8 Capacity-first Project Mesh . . . 9
3.9 Authenticated pure execution Fabric . . . 10
3.10 Checkpoint recovery and actor migration . . . 10
3.11 Information Kernel, Bridge, and Provenance . . . 10
3.12 Registry V1 . . . 12
3.13 World identities, values, protocols, and receipts . . . 12

4 Hosted surface language 13

4.1 Implemented structural grammar . . . 13
4.2 First example: evaluator-indexed expressions . . . 14
4.3 Bindings and dependencies . . . 15
4.4 Persistent and ephemeral evaluators . . . 15

5 Hosted dynamic semantics 15

5.1 Evaluation order . . . 15
5.2 Quotation and meta-evaluation . . . 16
5.3 Demand policies and requests . . . 16
5.4 Coordination groups . . . 17

6 The OValue semantic boundary 18

6.1 Authorable tagged OValue schema . . . 19
6.2 Fidelity . . . 20
6.3 Representation-sensitive identity . . . 21
6.4 Live native objects and enforced value crossings . . . 21
6.5 Wire protocol . . . 22

7 Backends, actors, and authority **22**
7.1 Static registry . . . 22
7.2 Actor process model . . . 23
7.3 Hosted authority model . . . 24

8 OIR, plans, effects, and HGraph execution **24**
8.1 OIR is orchestration IR . . . 24
8.2 Execution plan . . . 24
8.3 State-complete HGraph . . . 25
8.4 Effects, worker execution, and cancellation . . . 25

9 Projects, linking, and route policies **26**
9.1 Project lifting and literal linking . . . 26
9.2 Compiler-emitted Project HGraph from an o-linked codebase . . . 26
9.3 Typed grounding across planning front doors . . . 34
9.4 Bundle model . . . 34
9.5 Routes . . . 35
9.6 Unlinking . . . 37

10 Toolchain and implementation editions **37**
10.1 Public binaries . . . 37
10.2 What olangc targets mean . . . 38
10.3 Runtime payloads and Linux filesystem isolation . . . 39
10.4 Direct WASI and packaged Linux execution . . . 39
10.5 Reference implementations . . . 39
10.6 REPL and notebook state . . . 42
10.7 O-Git . . . 42

11 O-core: the native language **42**
11.1 Language surface . . . 42
11.2 Static semantics . . . 43
11.3 Compiler pipeline . . . 44
11.4 Current native compiler profile . . . 44
11.5 Importing original implementation code . . . 46
11.6 C variadics and explicit execution profiles . . . 47
11.7 Compiler-owned source recipes and module closure . . . 47
11.8 Measured original-source progress . . . 48
11.9 Native computation within the ordinary O graph . . . 49
11.10 Complete external-runtime and compiler objective . . . 49

12 Kernel, capabilities, and native execution **49**
12.1 Freestanding kernel substrate . . . 50
12.2 Keeping native images within their declared bounds . . . 50
12.3 Native capability model . . . 51
12.4 Native Ethernet and a real Linux guest . . . 53

13 Live-World and KernelWorld **54**
13.1 Three different layers . . . 54
13.2 Live-World command and state model . . . 55
13.3 Authorable Live-World schemas . . . 55
13.4 KernelWorld authority chain . . . 58
13.5 Native execution ladder . . . 60
13.6 Mode 20 and Mode 21 . . . 61
13.7 Modes 22 through 34 . . . 61

14 Reproducibility and dated evidence **63**
14.1 Retained checks for the 28 October revision . . . 63
14.2 Evidence classes and retained records . . . 65
14.3 Source inventory and coverage accounting . . . 66

15 Implemented capabilities **66**

16 Design rationale **67**
16.1 Why evaluator identity is syntax . . . 67
16.2 Why graph state is explicit . . . 67
16.3 Why analysis and authority are separate objects . . . 67
16.4 Why the engine owns the types . . . 67
16.5 Why admission is typed across layers . . . 68
16.6 Why placement begins with capacity . . . 68

17 Conclusion **68**

A Compact hosted-language reference **69**
A.1 Exact builtin signatures . . . 69

B Public command reference **70**

C Example atlas **73**

D Source map **73**

E Complete `ostadix-api` module inventory **75**

F Architectural position among neighboring systems **76**

G Reproduction commands **77**

G.1 Build the whitepaper artifact 77

G.2 Pin the source and qualify the Rust workspace 77

G.3 Run the language and unified front door 78

G.4 Qualify the independent engine and MCP 78

G.5 Generate and read the o-linked Project HGraph 78

G.6 Run Project HGraph, Live-World, and World gates 79

G.7 Run compiler targets 79

G.8 Run the Python and C17 editions 80

G.9 Inspect Information, Registry, and node surfaces 80

H Implementation and qualification boundaries **80**

1 Implementation basis and method

1.1 Source coordinate

This revision describes the published canonical OSTADIX commit `f472c983`, package coordinate `0.4.0 Unreleased`, reviewed on 28 October 2026. The title page and all source links pin the full commit. The development repository's `62c7f8f0` and the canonical publication have distinct histories and the identical tracked tree `627afd3ee65c829a3edce3e3bd6ba86db4660017`. Both histories retain prior remote work. Runtime source identity, installed binary identity, and execution evidence retain their own coordinates.

The revision covers three native compiler targets, original LLVM-module import, compiler-owned source recipes, reusable native module sets, variadic ABI support, explicit atomic/TLS/exception profiles, hosted reference-edition repairs, MCP HTTP and OAuth interfaces, and native image repair. It reports the retained 28 October host and portable-native validation separately from older compiler-development evidence. Section 14 gives the source and execution scope of each result.

The Rust workspace has three members. The root `o-lang` package owns compatibility reexports and command-line programs. The independently packageable `ostadix-api` owns the parser, IR, evaluator, graph, evidence, scheduler, project, networking, Information, Registry, World, Live-World, KernelWorld and O-core modules. These two default members share version 0.4.0. The `ostadix-core` member supplies the separately versioned `ostadix` command. The MCP server remains a separately locked package outside the workspace, keeping its protocol dependencies out of the engine's dependency graph.
[Cargo.toml] [crates/ostadix-api/Cargo.toml] [ostadix-core/Cargo.toml]

1.2 Contribution map

Contribution	Implementation mechanism	What it enables
Structural polyglot syntax	Exact evaluator tags, recursive parser nodes, structural splices, and persistent actor identities.	One program tree can contain native Python, SQL, Nix, shell, markup, and other language regions.
Typed exchange	OVALUE, canonical CBOR, renderer contracts, and boundary-specific fidelity judgments.	The exchange boundary preserves tagged meaning; modeled value crossings and renderer projections receive separate judgments.
Graph semantics	OIR, validated plans, hyperedges, completion tokens, and versioned resource states.	Execution order, effects, mutation, and safe parallelism become inspectable program structure.
Project composition	Lossless bundles, materialization, route policies, guards, and artifact collection.	Whole codebases become reversible, executable Ostadix artifacts.
Planning and realization	Canonical computation facets, declared realization tuples, shared typed grounding, and operation/transfer DAGs.	Inspect correspondence and selection, retain decisions, and execute through the applicable native admission and transport contracts.

Contribution	Implementation mechanism	What it enables
Native World lineage	O-CORE HIR/MIR/code generation, kernel capabilities, exact records, and generation-bound admission.	The language architecture reaches from hosted composition to machine-level execution and lifecycle authority.

1.3 Architectural path through the implementation

The paper follows the implementation from source syntax to execution:

1. parse hosted syntax into `ONode`;
2. lower it to OIR, a validated execution plan, and HGRAPH;
3. trace effects, actor identity, scheduling, process reuse, wire values, and authority;
4. trace project bundling and route execution separately;
5. trace O-CORE from lexer through object generation;
6. trace hosted Live-World, native kernel gates, KernelWorld admission, and SVM execution;
7. identify the applicable gate matrix and record which checks were actually executed for each source, artifact, and substrate coordinate.

2 The language system in one view

OSTADIX-LANG is best understood as a family with two compiler lineages and several execution substrates.

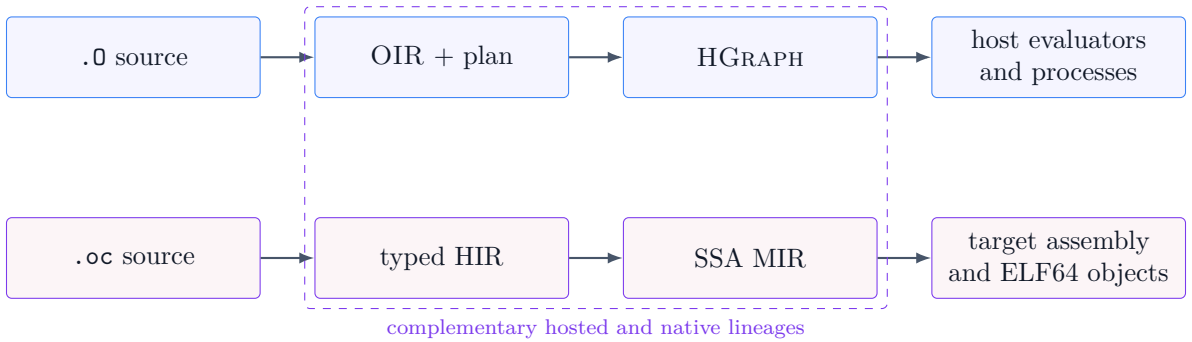


Figure 1: Compiler bifurcation. Hosted `.O` orchestrates evaluators; `.oc` compiles machine computation.

[docs/OCORE.md]

2.1 Hosted orchestration

Hosted O is a compact structural meta-language built from evaluator-indexed blocks, structural nesting and splicing, bindings, orchestration calls, persistent evaluator identities, quotation, explicit scopes, and demand policies. Computation remains native to its selected evaluator while O supplies the shared program structure.

This design makes Python, SQL, Nix, Markdown, shell, and other runtimes direct participants in one structurally visible program tree, with each evaluator retaining its own language semantics.

2.2 Project execution

Project execution lifts a directory into a lossless `ProjectBundle`. A dedicated subprocess coordinator executes its routes, dependencies, guards, artifacts, and alternative policies while the bundle preserves the codebase as one reversible language artifact.

2.3 Native systems execution

O-CORE is a statically typed systems language with modules, functions, structures, enums, explicit unsafe operations, inline assembly, and freestanding ABI constraints. Its output can be linked into the Ostadix kernel. The native lineage gives Ostadix-lang a machine-level language for implementing the runtime substrate that hosted O can construct, package, and coordinate.

2.4 World layers

Hosted Live-World provides executable local-process lifecycle semantics. Native kernel modes realize capability-governed services and virtualized execution. KernelWorld binds exact packages, manifests, policy, architecture, authority, health, and either package-payload image bytes or the declared SHA-256 of a user-supplied image into a verified admission chain. These layers form successive realizations of the World model.

3 The current execution constitution

The current master makes execution a composed constitutional path. Source becomes a solved graph. The analyzer binds that graph to runtime facts. The admission compiler reproduces those facts and materializes the inputs that the coordinator consumes. Local commands, agent tools, and Project Mesh all enter that path through explicit records. Information and World formats preserve the identities needed to inspect, move, and verify the resulting computation.

3.1 Independent `ostadix-api` ownership

I separated implementation ownership from command-line packaging. The publishable `ostadix-api` crate owns the runtime engine and its nominal types. The root `o-lang` package depends on the exact synchronized engine version and reexports the engine modules used by established `o_lang::` callers. A value parsed through the root library and a value handled through `ostadix_api` therefore have one defining type and one runtime implementation.

The ownership boundary is enforced by the workspace and package manifests, the engine module root, and the root's one-way reexport list. [Cargo.toml] [crates/ostadix-api/Cargo.toml] [crates/ostadix-api/src/lib.rs] [src/lib.rs]

3.2 The unified lowercase `o` front door

The lowercase `o` command is the compiled intent and operational front door. Its native dispatcher routes runtime, compiler, project, node, World, device, capacity, and evidence commands to their existing

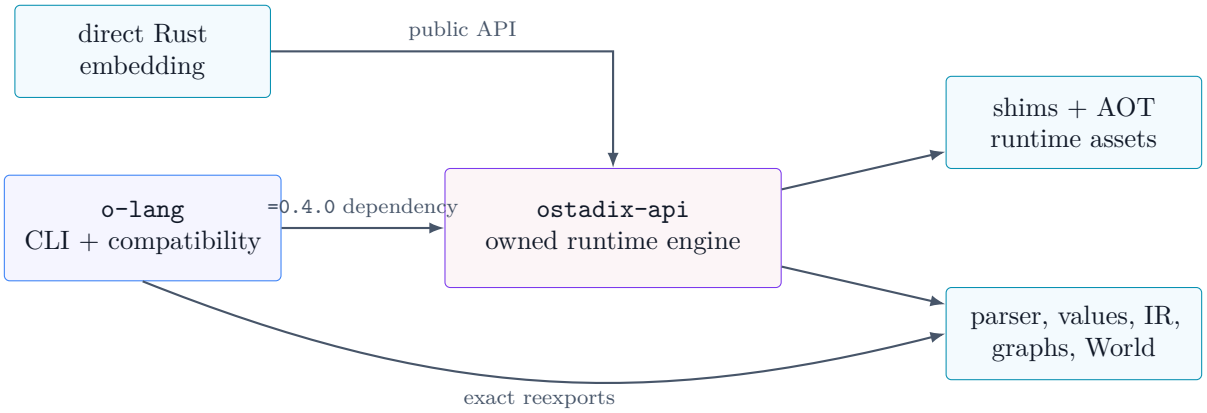


Figure 2: One engine owns runtime behavior. The root package supplies commands and preserves established imports by reexporting the engine's exact types.

owners. Installation provides compiled `o` and `ostadix-evaluator`; the shell script is a compatibility launcher. The front door classifies an input as hosted O source, a directory, a lifted project, or an accepted standalone form, then selects the ordinary evaluator or project engine and renders one stable intent record.

```
o run program.O
o plan program.O
o explain last-run
o inspect last-run

o run ./project --route main
o plan ./project --route main
```

Runs record a durable `RunRecordV1` by default; `-no-record` disables ordinary recording, while marked operation runs require it. Recording errors remain explicit. The record binds a schema, run identity, wall-clock time, resolved intent, engine selection, source identity, execution result, and optional trace attachment. The store uses canonical JSON bytes, validates every record before acceptance, writes through a temporary file, renames atomically, and maintains the selected history root. Record identity is content-bound, so inspection names the artifact that was actually persisted.

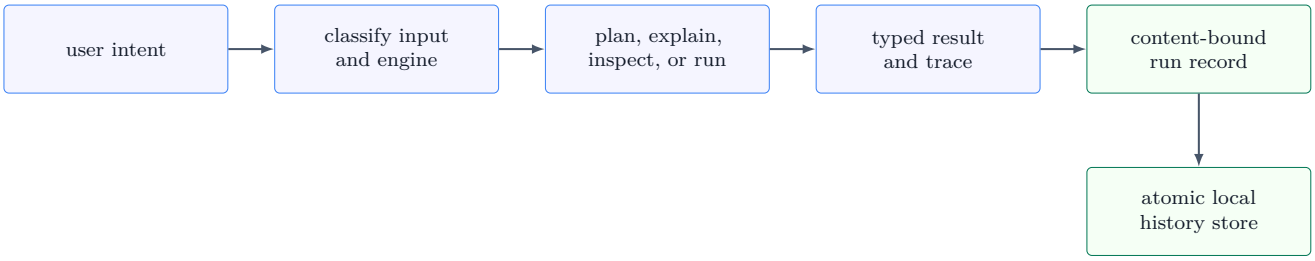


Figure 3: The lowercase front door converts command intent into a resolved execution path and a durable record that can be inspected later.

[src/bin/o-cli/native_dispatch.rs] [src/bin/o-cli.rs] [crates/ostadix-api/src/intent/record.rs] [crates/ostadix-api/src/intent/store.rs]

3.3 Graph V2, Evidence V6, Admission V6, and Why V2

Graph V2 is the executable identity of a solved hosted program. Its canonical digest covers graph nodes, typed values, actor identities, the complete typed fidelity assessment, executable operations, effects, sequence dependencies, roots, and bindings. A semantic change in any of those fields produces a different graph coordinate.

Evidence V6 binds that graph to the complete lowering and execution context. The bundle carries the OIR and plan identities, Graph V2 identity, analyzer coordinate, backend Catalog V6 projection, backend artifact bindings, runtime binding, environment binding, ambient World binding, and one typed assessment for every executable operation. The analyzer first validates the caller graph against a freshly solved Graph V2 before emitting the bundle.

Admission V6 is the executable authority compiler. It checks the schema and analyzer coordinate, re-solves and re-analyzes the graph, matches the complete operation inventory, verifies every hard assessment, materializes admission evidence nodes, validates the admitted graph, and constructs its ready schedule. The coordinator and evaluator accept the resulting `AdmittedExecution` object.

Why V2 is the reader-facing explanation of that authority. For a selected plan node it reports the admitted operation, its inputs and outputs, effect and actor rails, dependency witnesses, typed fidelity decision, and schedule position. It is derived from Admission V6 as an inspection operation.

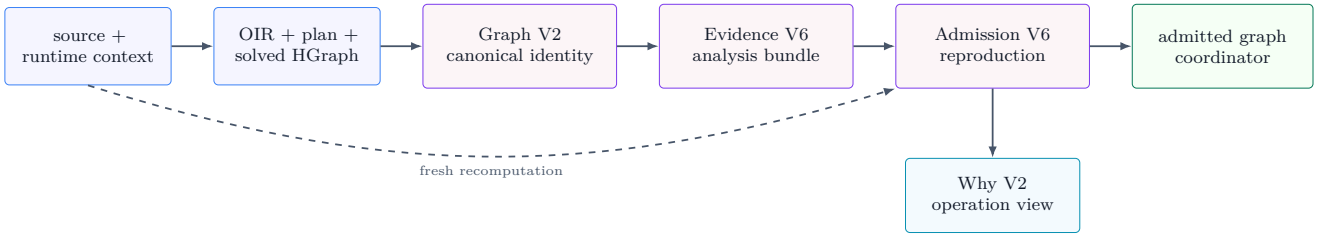


Figure 4: Current hosted execution is a proof-producing and proof-consuming pipeline. Admission reproduces the analyzer result before execution begins.

Coordinate	Primary object	Mechanism
Graph V2	Solved executable HGraph	Canonical identity over executable shape, typed values, effects, actor state, fidelity, roots, and bindings.
Evidence V6	`EvidenceBundleV6`	Analyzer result bound to graph, lowering, catalog, artifacts, runtime, environment, and World.
Admission V6	`AdmittedExecution`	Fresh reproduction, complete per-operation validation, evidence materialization, and ready scheduling.
Why V2	`ScheduleWhyV2`	Stable operation neighborhood projected from the admitted schedule for human and tool inspection.

[crates/ostadix-api/src/evidence/fact.rs] [crates/ostadix-api/src/evidence/analyze.rs] [crates/ostadix-api/src/evidence/admit.rs] [src/bin/olangc.rs]

3.4 A shared native interface for humans and agents

The compiled lowercase `o` front door and the Ostadix MCP expose the same native command owners. A source catalog describes 23 command entries and 11 guide topics, including the aggregate `all` topic. Humans discover them through `o capabilities` and `o guide`; agents use `o_capabilities` and `o_guide`. The expert routes `o tool` and `o_cli` resolve the same catalog entries while preserving each command's argument grammar and admission rules. Discovery describes the source interface; executable availability and support for a particular option must be checked against the selected installation. [src/command_catalog.rs] [src/bin/o-cli/native_dispatch.rs]

The separately locked Rust/`rmcp` stdio and Streamable HTTP server has 21 tool methods in this source snapshot. `setup.sh` builds and installs it, and the repository's `.mcp.json` registers it for MCP clients.

Tools	Native responsibility
`o_execute`, `o_run`	Complete source or existing program/project execution, with explicit static inspection modes.
`o_operation`	Declared operation description, alternatives, planning, execution, retained observation, and replanning.
`o_cli`, `o_eval`	Literal native argv and inline polyglot source, with per-child working directory, environment, and input.
`o_capabilities`, `o_guide`	Shared command discovery and human/agent workflows.
`o_job_list`, `o_job_status`, `o_job_read`, `o_job_write`, `o_job_cancel`	Concurrent session jobs, bounded log pages, lossless base64 reads, interactive input, and owned-process cleanup.
`o_env`, `o_runtimes`, `o_doctor`, `o_smoke`	Resolved installation, backend requirements, diagnostics, and a private fixed Python arithmetic smoke program.
`o_analyze_intent`, `o_execute_intent`	One-use binding of exact source and stable intent before fresh runtime admission.
`o_olangc`, `o_information_inspect`, `o_search_run`	Compiler targets, bounded existing-root Information inspection, and the named search workflow.

[mcp/ostadix_lang_mcp_server/src/main.rs]

`o_execute` accepts exactly one complete source document or existing path. Its `check` action parses; `plan` constructs a static schedule; `compile` explicitly requests a compiler product; and `execute` invokes the native engine. The compatibility `o_run` check mode performs static planning and graph validation. For ordinary source, admitted execution binds the current source and intent before asking the runtime to establish fresh admission. The explicit analyze/execute protocol consumes its expiring handle before validation and binds program, working directory, root, backends, source, and intent. The handle cannot substitute for live runtime authority. [mcp/ostadix_lang_mcp_server/src/unified.rs] [mcp/ostadix_lang_mcp_server/src/main.rs]

Placement retains the native operation's scope. Ordinary source executes on the local HGraph unless one authenticated node is explicitly selected for the complete document. Project routes can use native mesh placement; required mesh placement fails when it cannot be satisfied. Selected-node submission does not partition an ordinary graph. Its client cancellation cannot undo remote effects, and unknown completion must remain distinguishable from a safe-to-repeat failure. [mcp/ostadix_lang_mcp_server/src/selected_node.rs] [mcp/ostadix_lang_mcp_server/src/unified.rs]

Background execution returns a session job with independently retained output. On supported Unix hosts, cancellation cleans the owned session, including nested process groups, and reports observed cleanup. Deliberately detached sessions lie outside that ownership. Jobs and intent handles belong to the MCP session; server shutdown ends jobs, and reconnecting does not restore them. The persistent local client bridge preserves a session for short-lived CLI callers. A job identifier records a start, not successful completion. [mcp/ostadix_lang_mcp_server/src/execution.rs] [scripts/ostadix_mcp_client.py]

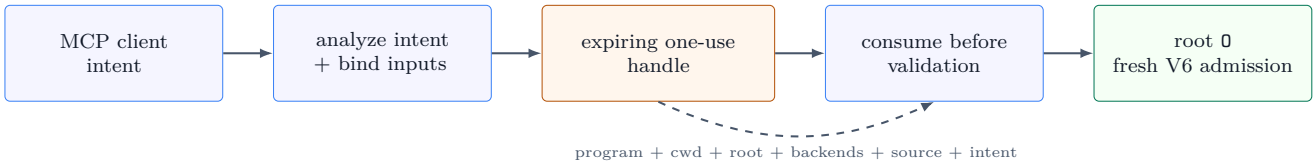


Figure 5: The MCP analyze/execute protocol carries a bounded intent binding into a fresh root-runtime admission.

HTTP transport and client authorization. The same 21-tool implementation is available over stdio and Streamable HTTP. The HTTP endpoint retains the initialized MCP lifecycle and associates jobs and intent handles with a session. Subsequent requests retain the negotiated protocol version and session identifier. Explicit session deletion, inactivity expiry, or server shutdown ends the session and cancels its jobs. The transport accepts configured static bearer authentication, explicit unauthenticated gateway deployment, or the native single-owner OAuth mode. [mcp/ostadix_lang_mcp_server/src/http.rs] [mcp/ostadix_lang_mcp_server/README.md]

The OAuth implementation provides resource and authorization-server metadata, dynamic client registration, S256 PKCE authorization-code exchange, owner-token confirmation in browser consent, and rotating refresh tokens. Persisted grant state binds client identity and token families; replay of a used refresh token revokes its grant family. The consent screen states that an authorized client obtains host tool access, including native commands, program execution, and files. OAuth authorizes access to this MCP service; it does not establish per-tool isolation or replace the runtime's admission, World, capability, or execution-receipt mechanisms. [mcp/ostadix_lang_mcp_server/src/oauth.rs] [mcp/ostadix_lang_mcp_server/src/oauth/model.rs] [mcp/ostadix_lang_mcp_server/src/oauth/handlers.rs] [mcp/ostadix_lang_mcp_server/src/oauth/handlers/exchange.rs] [deployment/mcp/CLIENTS.md]

3.5 Computation identity and declared realization planning

`OComputationManifestV1` relates immutable facets through explicit derivations while domain modules retain their native representations. The ordinary-source builder reparses and lowers the source, recomputes its stable execution intent, and checks the supplied OIR, plan, and solved graph before binding their identities. The project builder similarly verifies the exact bundle, project HGraph, logical projection, and deployment proposal. These builders establish correspondence among supplied semantic products; execution evidence, live capabilities, and admission are established separately. [crates/ostadix-api/src/computation_core.rs] [crates/ostadix-api/src/computation/build_oir.rs] [crates/ostadix-api/src/computation/build_project.rs]

Operation contracts and interfaces describe the logical operation and its ports; realization descriptors and realization sets name supplied alternatives. The V1 operation planner accepts one logical operation and explicit complete tuples of realization, target, port representations, residency, and cost profile. It validates exact closure and port bindings and static target compatibility before ranking. Its implemented objective minimizes the checked sum of predicted compute, startup, conversion, transfer, queue, checkpoint, and uncertainty nanoseconds, with a canonical tuple tie break. Rejected candidates and their reasons remain in `DeploymentPlanV2`. [crates/ostadix-api/src/computation/realization_plan.rs]

Selection is descriptive and grants no reservation or execution authority. An explicitly marked project binds the selected descriptor to exact implementation bytes and a captured route pipeline, then enters the existing project admission and executor. The CLI retains the original request, deployment, and decision before dispatch. `o observe` uses the exact run identifier to inspect content-verified retained records; `o replan -without-target` computes an alternative under supplied exclusions without dispatch. Arbitrary implementation equivalence, live target reachability, and automatic failure detection require separate evidence. [docs/OPERATION_PLANNING_V1.md] [mcp/ostadix_lang_mcp_server/src/operation.rs]

```
o operation ./project
o realizations ./project
o plan ./project --json
o run ./project --json
o observe ./project --run RUN_ID --json
o replan ./project --run RUN_ID --without-target TARGET_ID --json
o computation --help
```

The project must carry the operation marker and exact declaration closure. Use the returned run identity for observation and replanning. The `computation` command builds or inspects computation artifacts through its explicit native subcommands.

3.6 Physical operation and transfer graphs

An embedding API combines operation closures in a `LogicalHGraphV2`. Each logical edge has one supplied transfer offer with exact target and representation coordinates. Planning filters compatible operation offers and derives an executable DAG in which the consumer waits for its transfer, and the transfer waits for its producer. Comparing alternative transfer routes requires separate requests; the planner does not construct a Cartesian product of arbitrary placements. [crates/ostadix-api/src/computation/graph_realization_plan.rs]

A trusted embedding separately registers operation and transfer adapters. Their admission methods check executable and live interaction requirements. Every transfer captures a source observation before moving the payload and checks the destination against it before enabling its consumer. The executor refills bounded worker slots as tasks complete; after failure it suppresses new tasks and drains started work. These checks preserve the adapter's declared observation contract and do not roll back effects already performed. [docs/PHYSICAL_GRAPH_EXECUTION_V1.md]

The automatic local OIR bridge derives this physical graph from a freshly admitted program. Every producer-backed input becomes a transfer task, including value, actor, resource, completion, and control inputs. Its Unix socket transport moves actual serialized inputs and scope to a receiving thread. The bridge independently checks canonical observation and exact typed carrier structure before preparing the consumer, preserving details such as duplicate map entries and raw floating-point bits. Native references retain their existing owner. Empty or literal-only programs have no physical operation plan; the bridge performs ordinary materialization and root commit. This profile is local and does not authenticate remote targets or reconstruct foreign native objects on another runtime. [crates/ostadix-api/src/computation/oir_physical_execution.rs] [docs/OIR_PHYSICAL_EXECUTION_V1.md]

3.7 Reciprocal LAN pairing and pinned TLS

Node trust begins with a reciprocal, one-use, ten-digit pairing code. Each node creates an ephemeral SPAKE2 role and validates the peer transcript. Directional HKDF and HMAC keys bind the code, both node identities, both endpoints, both nonces, and the full request/response transcript. Each destination

then issues the client certificate it will accept for the peer. Private keys remain local; the stored peer record carries the destination CA, destination server name, client certificate, and generation needed for the next TLS session.

The data plane uses TLS 1.3 with the Ostadix ALPN token. Outbound connection validation loads the paired destination CA and checks the stored server name. The node listener validates client certificates against its pairing CA before the application protocol receives a request. Discovery supplies addresses; the paired record supplies identity and cryptographic continuity.

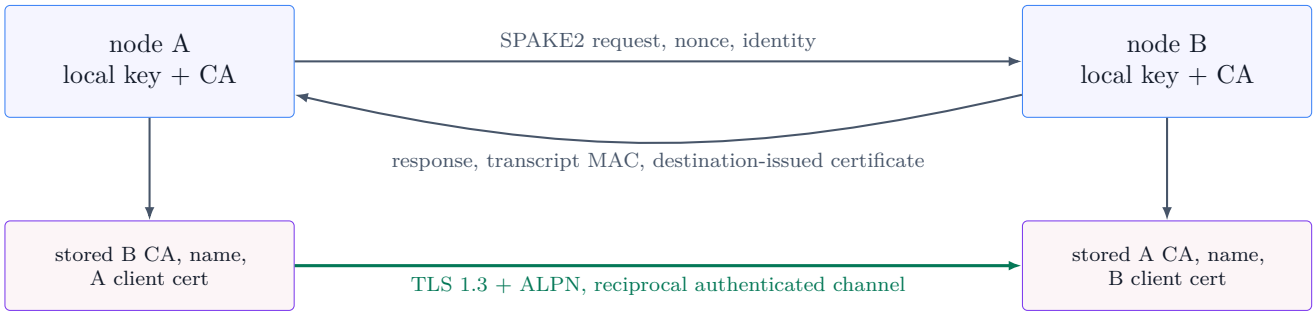


Figure 6: Pairing turns a short shared code into reciprocal destination pins and client credentials. Those persisted facts establish later node channels.

[crates/ostadix-api/src/hosted_remote/pairing.rs] [crates/ostadix-api/src/hosted_remote/lan.rs] [src/bin/o-node.rs] [src/bin/octl.rs] [crates/ostadix-api/src/hosted_remote/tls.rs]

3.8 Capacity-first Project Mesh

Project Mesh distributes a project at the route-branch scale. The planner selects the requested route branch and closes it over prerequisite routes, inputs, declared artifacts, and bundle provenance. Candidate nodes come from paired authenticated discovery and carry current profile, compatibility, and capacity facts. Ranking places usable execution capacity first, then applies the route and policy constraints that preserve semantic compatibility.

The selected node receives a source-closed project payload and a bounded execution request. Generation fences bind discovery, reservation, attempt, and result state. Retry contracts record why an attempt can continue, while concurrent selection policies record all relevant attempts and their terminal settlement. Mesh traces therefore connect placement decisions to the exact project branch, node generation, and returned artifacts.

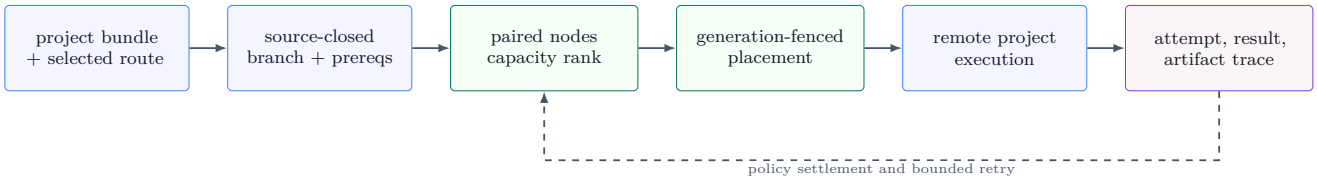


Figure 7: Project Mesh moves a complete route branch, chooses capacity, binds node generations, and returns a causally connected attempt trace.

[crates/ostadix-api/src/hosted_remote/project_mesh.rs] [crates/ostadix-api/src/hosted_remote/mesh.rs]

3.9 Authenticated pure execution Fabric

Fabric V1 adds an opt-in remote boundary for the frozen pure-renderer profile: one source-closed text, HTML, Markdown, or LaTeX operation with explicit input bindings, supported canonical OWVALUE inputs, and bounded output and time. The coordinator sends a capsule; the worker returns a provisional candidate. Candidate validation, publication, and deterministic settlement remain coordinator responsibilities. Physical execution can overlap without granting workers graph mutation or commit authority. [crates/ostadix-api/src/execution_fabric/protocol.rs]

The authenticated profile uses the exact `ostadix-execution-fabric/1` ALPN, paired TLS identity, and a separately signed V3 placement lease binding the exact attempt, source closure, capsule, runtime, target, and generations. A source closure permits reconstruction of the operation; the lease binds the admission digest. The provider retains attempt status in a local replay ledger and signs terminal candidate evidence. The transport grants no authority to publish values, choose a winning attempt, retry an ambiguous submission, or commit external effects. [crates/ostadix-api/src/execution_fabric_authority/protocol.rs] [crates/ostadix-api/src/hosted_remote/fabric/provider.rs]

The repository's two-node harness launches two authenticated node processes on one host with separate identities, state directories, generations, and ledgers. Its contract checks candidate agreement and wrong-node rejection, and requires selected-node failure to remain an infrastructure failure. That harness addresses real process and protocol boundaries; distinct-kernel, physical multinode, and heterogeneous-hardware qualification remain separate evidence requirements. Fabric does not generalize this profile to arbitrary OIR, distributed actors, general effects, or automatic placement. [tests/execution_fabric_two_node.rs] [docs/OIR_EXECUTION_FABRIC_V1.md]

3.10 Checkpoint recovery and actor migration

Local recovery restores settled persistent actors into destination-owned admitted fragments with matching runtime, environment, sandbox, executable, and generation identities. Local migration obtains real backend restore acknowledgments before fencing the source identities and retiring their processes. Failed installation rolls back actors created by that attempt and keeps unrelated destination state; shutdown failures after successful fencing remain explicit in the returned receipt. The supported codecs cover a constrained Python object graph, including aliases and cycles, and SQLite's autocommit main database. Coordinator bindings, in-flight work, process memory, and external resources remain outside this transfer. [crates/ostadix-api/src/eval/migration.rs] [docs/LOCAL_ACTOR_MIGRATION.md]

Hosted V2 adds a distinct authenticated two-node protocol with `Prepare`, `Install`, `Fence`, and `Activate` phases, plus `Abort`/`Cancel`. Exact plans bind both session and node identities, the common principal, actor generations, journal heads, and checkpoint digest. Every phase requires a fresh purpose-bound warrant and placement lease. Signed durable receipts preserve standby restrictions and permanent source fencing through restart; exact retries return the original terminal receipt. The destination becomes executable only after the source's signed fence. A partition can therefore prevent progress while preserving exclusive ownership. Test coverage uses separate local TLS node processes with real Python or SQLite restoration and process-crash injection; it does not establish a cross-host qualification or arbitrary native-resource migration. [crates/ostadix-api/src/hosted_remote/v2/migration_protocol.rs] [docs/HOSTED_ACTOR_MIGRATION.md] [tests/hosted_actor_migration.rs]

3.11 Information Kernel, Bridge, and Provenance

The Information Kernel turns observations into content-addressed information state. An entity descriptor identifies what a fact is about. An Information Atom binds that entity, a typed payload tier, assertions,

and provenance. Snapshots assemble atom identities into one immutable view. Revisions advance named roots through compare-and-swap, so a reader can identify the exact view it observed and a writer can prove the predecessor it replaced.

Payload tiers keep size and meaning explicit. T0 carries bounded public scalars. T1 identifies content held by the managed blob store. T2 identifies external content through canonical schema, media type, digest, and logical length. Canonical CBOR and domain-separated SHA-256 give every atom, blob, snapshot, and revision a stable identity.

Information Bridge V1 defines exactly eight bounded projections:

1. parsed document identity and structural counts;
2. allowlisted public scalar value identity;
3. solved HGraph metadata;
4. exact Evidence V6 metadata;
5. verified Registry V1 profile metadata;
6. verified World receipt identity and semantic digest;
7. logical project graph identity; and
8. Hosted V2 journal-entry identity.

Each projection has a private wire shape, strict schema check, bounded canonical-CBOR decoder, and canonical re-encode check. The bridge chooses the metadata that belongs in Information and hashes identifiers whose native locator vocabulary should remain outside the Information record.

Information Provenance V2 adds a content-addressed sidecar over frozen atom identities. Typed witnesses describe acquisition, producer authentication, procedure, signer authorization, receipt currentness, plan-node attribution, execution and effect fidelity, and morphism fidelity. Admission recomputes the sidecar image, verifies the bound World receipt and execution evidence, then derives a recovery assessment with intrinsic loss, discharged obligations, and unresolved obligations. Exact recovery establishes an origin statement that can be presented to Information readers.

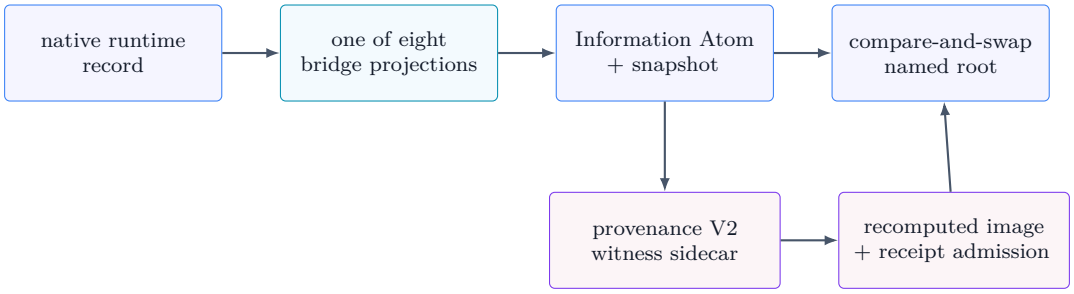


Figure 8: Native facts enter the Information Kernel through explicit bounded projections; provenance sidecars add typed recovery evidence to frozen atom identity.

[crates/ostadix-api/src/information/mod.rs] [crates/ostadix-api/src/information/model.rs] [crates/ostadix-api/src/information/root.rs] [crates/ostadix-api/src/information/provenance.rs] [crates/ostadix-api/src/information_bridge/mod.rs] [crates/ostadix-api/src/information_provenance/mod.rs]

3.12 Registry V1

Registry V1 is an append-only Ed25519 namespace and node-profile chain. A namespace root establishes the trust anchor and validity interval. A root or delegate can authorize a strict descendant namespace. Profile publications bind a namespace, node identity, generation-bearing node profile, issue time, and expiry. Every event carries its sequence, predecessor digest, namespace, signer key, and signed body.

Verification walks the chain from configured anchors, validates signatures and delegation scope, enforces time bounds and profile freshness, and maintains the highest accepted generation for each node. Sequence, predecessor, rollback, and equivocation checks turn a local event file into a verifiable publication history. The resulting `VerifiedRegistryProfileV1` is the profile form consumed by placement and Information projection.

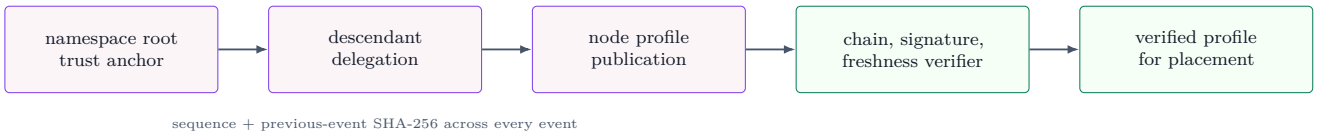


Figure 9: Registry V1 turns signed namespace events into a freshness-checked node profile suitable for placement decisions.

[crates/ostadix-api/src/registry/mod.rs] [crates/ostadix-api/src/registry/model.rs] [crates/ostadix-api/src/registry/crypto.rs] [crates/ostadix-api/src/registry/verify.rs]

3.13 World identities, values, protocols, and receipts

The World layer uses four separate canonical formats. Separation keeps each decoder's vocabulary, size ceiling, and authority meaning explicit.

Format	Primary unit	Mechanism
`OWIDENT V1`	World identity atom	Deterministic identity-only records with strict nonzero generation, term, version, and index rules plus hierarchical stale and logical comparison.
`OWPROTO V1`	Bounded protocol record	Big-endian framing, four fixed record kinds, exact-length and reserved-field validation, and deterministic version and record-limit negotiation.
`OWVALUE V1`	Portable value	Self-framed allowlisted value tree with bounded bytes, depth, and nodes; ordered records and scalar-key maps; root extension envelope with a portable payload.
`OWRECEIPT V1`	Execution receipt	Canonical execution identity, World generations, content digests, descriptive rights, terminal and commit fields, evidence gate, and domain-separated Ed25519 signature envelope.

The Rust and O-core implementations share the wire definitions and canonical corpora. Receipt verification returns a typed verified receipt. Semantic projection removes signer-dependent envelope variation so hosted execution and native Mode 32 can compare the same execution meaning.

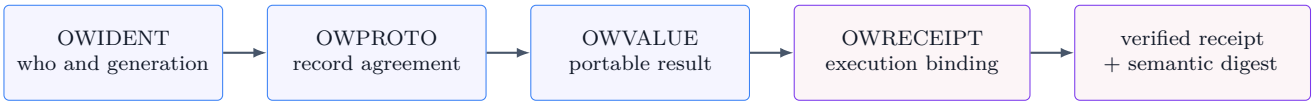


Figure 10: The World formats compose identity, protocol, portable value, and signed execution evidence while retaining separate canonical envelopes.

[crates/ostadix-api/src/world/identity_wire.rs] [crates/ostadix-api/src/world/protocol.rs] [crates/ostadix-api/src/world/codec.rs] [crates/ostadix-api/src/world/value_codec.rs] [crates/ostadix-api/src/world/receipt.rs] [crates/ostadix-api/src/world/receipt_codec.rs]

4 Hosted surface language

4.1 Implemented structural grammar

The implementation AST has five semantic forms: `RawText`, `VarRef`, `LetBinding`, `TypedExpr`, and `Call`. [crates/ostadix-api/src/parser.rs] The following EBNF is an implementation-oriented summary; context rules below are part of the grammar.

```
cli_source   ::= [ shebang newline ] document
shebang      ::= "#!" raw_line
document     ::= sequence
sequence     ::= { comment | raw | var | typed | binding | call }

typed        ::= raw_tag "^(" body ")_" raw_tag
raw_tag      ::= ident [ "[" uint "]" ] [ attributes ]
attributes   ::= "{" attribute { "," attribute } "}"
attribute    ::= ident | ident "=" visible_value
visible_value ::= visible_attr_char { visible_attr_char }

var          ::= "$" ident
binding      ::= "let" ident "=" ( typed | call )
call         ::= ident "(" [ call_arg { "," call_arg } ] ")"
call_arg     ::= var | call
comment      ::= "#" { non_newline_byte } [ newline ]

ident        ::= ( letter | "_" ) { letter | digit | "_" }
uint         ::= digit { digit }
letter       ::= "A"..."Z" | "a"..."z"
digit        ::= "0"..."9"
```

`visible_attr_char` is visible ASCII excluding braces, comma, and equals. `raw` is one or more source bytes up to the next unescaped structural token recognized in the current context; `raw_line` and `non_newline_byte` end at LF or CR/LF. At document level and inside `0` or `quote`, the recognized tokens are comments, bindings, calls, variables, and registered typed openers. Inside every other evaluator body, only variables and registered typed openers are structural. A backslash escapes an otherwise structural opener, closer, or variable splice. The CLI entrypoints strip one leading shebang before invoking the parser.

4.1.1 Exact tag symmetry

The parser closes a typed expression only when its language, environment, and attribute text exactly reproduce the opener. This lexical symmetry makes the extent of every evaluator invocation locally decidable, even when foreign source contains parentheses of its own. A missing environment is represented internally by `u32::MAX` and selects a fresh ephemeral actor. An explicit `LANG[n]` selects a persistent evaluator actor. Language aliases canonicalize in the AST, and reconstruction emits the canonical form. [crates/ostadix-api/src/parser.rs]

4.1.2 Structural versus opaque contexts

`$name` and registered nested typed blocks are recognized throughout foreign bodies. The sequencing forms `let`, calls, and `#` comments are recognized at document level and inside `O` and `quote`. Everywhere else, the parser preserves backend source verbatim. Registry membership therefore controls which apparent tag forms become structural nodes, while ordinary backend syntax retains its native meaning. [crates/ostadix-api/src/parser.rs]

4.1.3 Intentional smallness

O delegates literal construction and ordinary computation to its evaluators. A binding right-hand side is a typed block or call, and a call argument is a variable load or nested call. Every resulting value therefore has an explicit origin in an evaluator or orchestration builtin, which keeps the host grammar small while preserving the full expression systems of its member languages. [crates/ostadix-api/src/parser.rs]

4.2 First example: evaluator-indexed expressions

```
html^(
  <p>The answer is python^(
__oval_result__ = sum(x*x for x in range(10))
)_python.</p>
)_html
```

The inner Python evaluator produces a typed result. Before the outer HTML backend runs, the runtime renders that result according to HTML's splice rules. The splice is a first-class child node with its own evaluator identity, environment, result type, and data dependency. This is the central syntactic move: foreign source remains idiomatic while cross-language composition is visible to parsing, planning, and scheduling.

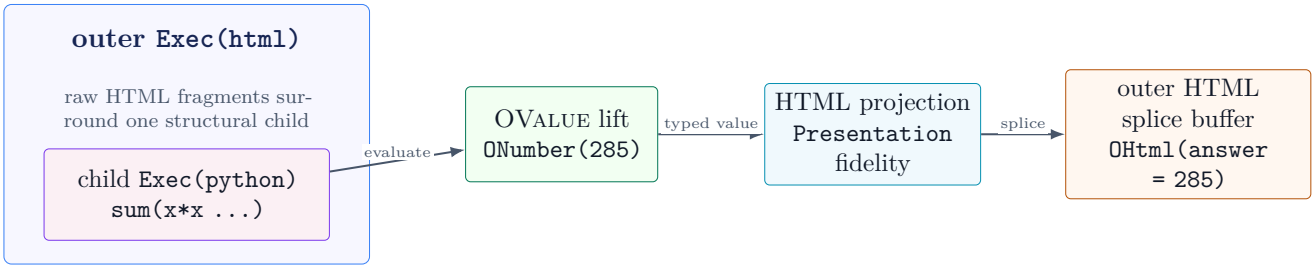


Figure 11: A nested evaluator is a typed child in the program tree. Its result crosses the evaluator boundary as OVALUE, then the receiving inline HTML projection assembles the outer `OHtml` value.

4.3 Bindings and dependencies

```
let answer = python^(
__oval_result__ = 40 + 2
)_python

python^(
__oval_result__ = $answer + 1
)_python
```

`let` lowers to `Store`; `$answer` lowers to `Load`. Plan construction connects the load to the most recent visible store with an explicit data edge. Each shim also receives the complete visible scope. The first mechanism makes dependencies schedulable; the second lets callbacks and backend adapters inspect the same lexical state that the host used to construct the invocation. [crates/ostadix-api/src/ir.rs] [crates/ostadix-api/src/eval.rs]

4.4 Persistent and ephemeral evaluators

```
python[0]^(
x = 40
)_python[0]

python[0]^(
__oval_result__ = x + 2
)_python[0]
```

Both blocks share the same language, explicit environment ID, and process policy, so the process registry reuses one actor and Python state persists. Bare `python^(...)_python` blocks each receive a fresh ephemeral identity and are cleaned up after execution. Different numeric IDs isolate interpreter-local state; ambient `HostWorld` resources remain shared. [crates/ostadix-api/src/process.rs]

Two explicit state domains

Root `Store` nodes commit to the caller's O scope, and nested stores belong to the nested evaluation scope. Persistent evaluator actors retain the backend's own process state under their environment identity. This separation gives O lexical state a graph-visible lifetime while allowing a backend such as Python to preserve its native runtime state across selected invocations.

5 Hosted dynamic semantics

5.1 Evaluation order

The default public path strips a shebang, parses the document, lowers it, and executes the graph. `O_EXECUTOR=serial` selects the reference interpreter; `graph` is the default. The graph executor preserves required data, sequence, completion, and resource-state dependencies. [src/main.rs] [crates/ostadix-api/src/eval.rs]

At document level, evaluation returns the last non-null root result. Raw regions preserve backend source. A typed block:

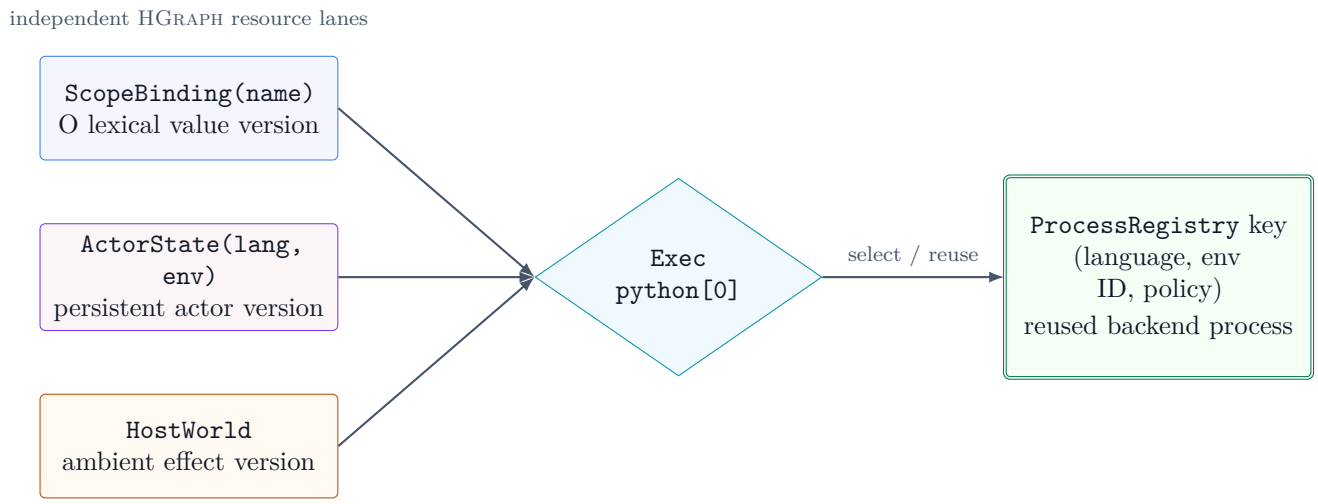


Figure 12: Lexical bindings, persistent actor state, and ambient host effects remain separate graph resources. `ActorState` uses canonical language and environment identity; concrete process reuse additionally includes the sandbox policy.

1. evaluates structural children and loads visible bindings;
2. renders child values using the selected backend's splice renderer;
3. resolves an inline evaluator or an external backend actor;
4. sends source, environment identity, bindings, and policy;
5. decodes the backend result into OVALUE.

5.2 Quotation and meta-evaluation

`quote` returns an unevaluated `OExpr`. A Python shim can invoke `O.eval(expr)`, which reparses and lowers the quoted form under a cloned callback scope. `O.scope(map)` constructs an explicit detached scope, and the two-argument `O.eval(expr, scope)` form chooses it. [crates/ostadix-api/src/eval.rs]

```
python[0]^(
q = quote^(python^(6 * 7)_python)_quote
__oval_result__ = O.eval(q)
)_python[0]
```

The callback path clones or explicitly selects an O scope, reparses the quoted expression, and sends its result back through the waiting actor protocol. Actor identity also provides a deadlock guard: an actor waiting on its own callback rejects recursive re-entry. Deferred shim forcing follows a single-level process path. [crates/ostadix-api/src/eval.rs] [crates/ostadix-api/src/process.rs]

5.3 Demand policies and requests

The evaluator exposes three demand policies:

Policy	Effect	Resolution rule
Eager	Resolve normal calls and blocks immediately.	The ordinary document policy.
Lazy	Construct a request or cache-safe deferred value.	`{lazy}` is accepted only for trusted inline renderers whose fingerprint covers the relevant inputs.
Autonomous	Buffer eligible requests for scheduler resolution.	Strict cache behavior can turn a missing result into an error after flush.

`{defer}` captures an evaluator request for explicit forcing with `now`; its request remains non-cacheable, and an unforced splice is a typed evaluation error. `{lazy}` attaches a cache-safety contract to trusted inline evaluation, so the runtime can force it on demand and reuse the fingerprinted result. [crates/ostadix-api/src/execution_contract.rs] [crates/ostadix-api/src/eval.rs]

Autonomous resolution of Nix-family request values uses a fingerprint-keyed, two-level scheduler. It first collects the transitive request closure and deduplicates the DAG, promotes results through an L1 memory cache and an L2 JSON disk cache, then dispatches uncached ready nodes in topological waves. Default parallelism is the smaller of the available CPU count and eight, with an explicit caller override. The L2 path resolves through `XDG_CACHE_HOME`, then `HOME`, then the platform temporary directory. Each cache entry is written beside its final path and atomically renamed. A missing, unreadable, malformed, or unwritable cache entry is a soft cache event: execution proceeds and preserves the result contract. [crates/ostadix-api/src/scheduler.rs]

The Nix request ladder is explicit:

$$\texttt{NixExpr} \xrightarrow{\texttt{instantiate}} \texttt{Derivation} \xrightarrow{\texttt{realise}} \texttt{StorePath} \xrightarrow{\texttt{activate}} \texttt{System}.$$

`dry_activate` constructs a preview request, `activate(path)` performs the ambient-authority activation, and `current_system` produces a referential system value. Separate request constructors make inspection and state-changing activation distinct in the value algebra and scheduler.

The perform boundary is concrete. `instantiate` wraps the source as `(let v = (BODY); in v.drvPath)`, runs `nix eval -raw -impure -expr`, validates the returned `/nix/store/*.drv` path, and asks `nix derivation show` for its output names, using `out` when output inspection fails. `realise` runs `nix build DRV^out -no-link -print-out-paths` and validates the printed store path. Activation executes the closure's `bin/switch-to-configuration` with `switch` or `dry-activate` and supplies the chosen profile in `NIX_PROFILE`. [crates/ostadix-api/src/eval.rs] [crates/ostadix-api/src/nix_ops.rs] [crates/ostadix-api/src/nixos_ops.rs]

5.4 Coordination groups

Mode	Settlement	Current behavior
`batch`	Collect every outcome in member order; failures become structured error values.	Threadable work respects scheduler parallelism.
`all`	Require every member to succeed; otherwise fail the group.	Threadable work respects scheduler parallelism.

Mode	Settlement	Current behavior
`any`	Return the first success; fail only when all fail.	Probe serial members in source order; if none succeeds, launch the complete threadable cohort.
`race`	Return the first settlement, success or failure.	The first serial member settles the group when one is present; otherwise race the complete threadable cohort.

These request-value groups capture unevaluated arguments, preserving their computation topology until resolution. Their threadable classification covers the Nix request family. Explicit autonomous evaluator groups use the separate worker admission path described in the HGraph execution section. [crates/ostadix-api/src/eval.rs]

Concurrency and settlement

Graph-level parallelism is derived from value, completion, and resource dependencies. Its worker profiles include verified scope reads, trusted inline renderers, and eligible ephemeral backends inside explicit autonomous groups. Request-value concurrency uses `batch`, `all`, `any`, or `race` to select settlement of the threadable Nix request family. Each plane keeps its own admission, ordering, cancellation, and failure rules.

6 The OValue semantic boundary

OVALUE is the cross-evaluator tagged algebra. It carries values, references, control objects, and error information across runtimes. [crates/ostadix-api/src/value.rs]

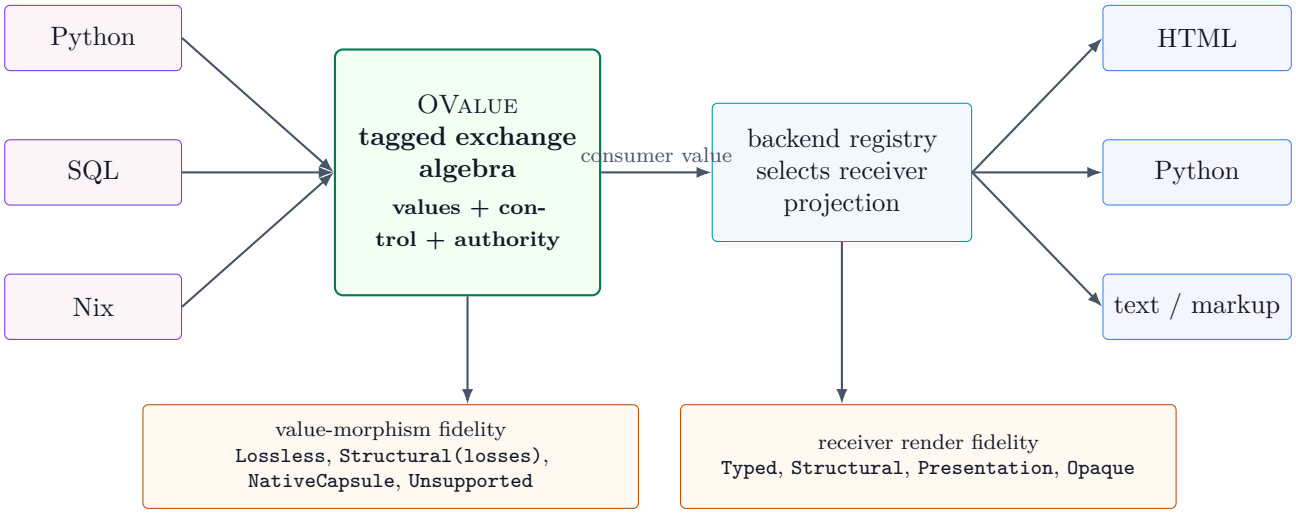


Figure 13: Every producer lifts into one inspectable algebra. The receiving backend then projects that value into its own source domain. Value conversion and source rendering retain distinct boundary-specific fidelity judgments; value-crossing judgments expose the public cross-boundary composition operation, while render-fidelity combination is confined to recursive child folding within one renderer.

Family	Implemented variants
Scalars	Null, Bool, Number, Text, Char, HTML, store paths, and expressions.
Numbers	Arbitrary-precision integers, rationals, decimals including special values, binary floats, arbitrary-precision floats, and complex numbers.
Collections	List, Map, Seq, Object, entry-preserving map, Set, Symbol, and Keyword.
Rich payloads	Blob, Bytes, Graph, and Native capsule.
Execution state	Scope, Snapshot, Thunk, Request, Group, structured Error.
World references	NixExpr, Derivation, System, Capability.

The public subalgebras make collection shape, authority, snapshots, native rehydration, and runtime persistence explicit:

Subalgebra	Variants
`SeqKind`	List, Tuple, Vector.
`SetKind`	Ordered, Unordered.
`CapabilityKind`	File, MemoryRegion, Device, Clock, NetworkEndpoint, Process, Service, SystemActivation, BackendExecution.
`SnapshotKind`	System, Service, Filesystem, Device.
`RequestKind`	Instantiate, Realise, Eval, Activate.
`NativeCodecSafety`	DataOnly, SourceBacked, UnsafeOpaque, LiveHandle.
`NativeBoundary`	Pure, Referential, Effectful.
`RehydratePolicy`	Portable, SameBackend, SameProcess, Never.
`RuntimeBoundary`	Pure, Referential, Effectful.
`Fidelity`	Lossless, Structural, NativeCapsule, Unsupported.
`AnnotationKind`	TypeTag, NumericPrecision, NumericExactness, Encoding, Ordering, Identity, Constraint, Capability, Presentation, and an extensible BackendSpecific label.

6.1 Authorable tagged OValue schema

Every JSON or CBOR OValue is selected by its top-level `t` field. Fields named as values, items, members, bindings, metadata, or state contain OValues recursively.

`t` tag	Fields and nested vocabulary
`null`; `bool`	Null consists exactly of the `t` tag. Bool adds Boolean `v`.
`number`	`v.kind` is `int{v}`, `rational{num,den}`, `decimal{coeff,exp10,special?}`, `binary_float{format,bits}`, `big_float{mantissa,exp2,precision?,special?}`, or `complex{re,im}`. A question mark identifies an optional or nullable field. Canonical big integers are decimal strings; formats are `f32`|`f64`; specials are `nan`, `pos_inf`, `neg_inf`, `pos_zero`, and `neg_zero`.

t tag	Fields and nested vocabulary
text; char; html; store_path; expr	Text has v{utf8,encoding?}; char has scalar; HTML has v; store path has path; expression has src.
list; map	List v is an array; map v is a string-keyed object.
seq; object; entries_map; set	Seq has kind=list|tuple|vector and items; object has string fields; entries-map has an entries array of key/value pairs; set has kind=ordered|unordered and items.
symbol; keyword; scope	Symbol and keyword v each contain optional or nullable namespace and required name; scope has string-keyed bindings.
blob; bytes	Blob has base64 v and mime. Bytes has v{bytes,media_type}, with an octet array and nullable media type.
graph	root is a node ID; nodes contain kind=value with value, or kind=ref with target.
native	v contains lang, nullable implementation/version, type_name, identity{stable?,live?}, codec, optional or nullable byte payload, boundary, safety, capabilities, metadata, and rehydrate. Boundary is pure|referential|effectful; safety is data_only|source_backed|unsafe_opaque|live_handle; rehydration is portable|same_backend|same_process|never.
nix_expr; derivation	Nix expression has body, deps, and fingerprint; derivation has drv_path, outputs, and deps.
request; system	Request has kind, source, and fingerprint. Kind is instantiate, realise, an eval object with lang, env_id, cacheable, nullable authority, and permissions, or an activate object with profile, dry_run, and nullable authority. System has profile_path.
capability; snapshot	Capability has kind, opaque identity, and descriptive metadata. Snapshot has kind=system|service|filesystem|device, identity, and state.
thunk; group; error	Thunk has body, deps, and fingerprint. Group has mode=batch|all|any|race, members, and fingerprint. Error has msg.
Legacy input tags	int{v}, float{v}, and str{v} are accepted and normalize to canonical number or text.

Capability kinds are file, memory_region, device, clock, network_endpoint, process, service, system_activation, and backend_execution. [crates/ostadix-api/src/value.rs]

6.2 Fidelity

Conversions carry one of four fidelity states: Lossless, an accumulated set of structural losses, NativeCapsule, or Unsupported. Composition unions structural loss sets and selects the absorbing severity classes; its idempotence permits the HGraph solver to converge at a fixpoint. [crates/ostadix-api/src/value.rs]

FidelityAssessmentV2 refines the structural case to an interval of definite and possible loss sets, with the checked invariant $D \subseteq P$. Its from_concrete operation is an exact point embedding of a V1 judgment; concretization is exposed as a membership test rather than an exponential powerset. Sequential composition unions both bounds. The structural/lossless path join is the interval hull, while NativeCapsule and

`Unsupported` remain absorbing severity classes rather than cross-class disjunctions. These implemented laws do not by themselves establish a Galois connection. [crates/ostadix-api/src/value.rs]

`RenderFidelity` can be recomputed on demand as a separate description of source projection. It combines classifications only while recursively folding children within one renderer. No conversion, common carrier, or Galois connection relates it to `Fidelity` or `FidelityAssessmentV2`. [crates/ostadix-api/src/eval_core.rs]

`BackendMorphismV1` defines bounded Python, JavaScript, and Rust crossing profiles with explicit input projection, output injection, rejection kinds, two fidelity legs, and a checked lossless round-trip law. Catalog V5 first bound each profile to its canonical backend entry, and Catalog V6 retains those assignments. The HGraph solver exposes the resulting assessment beside its compatibility fidelity result in shadow mode, making profile divergences inspectable while the established admission, rendering, and dispatch paths retain their current authority. [crates/ostadix-api/src/backend_morphism.rs] [crates/ostadix-api/src/hgraph/solve.rs]

6.3 Representation-sensitive identity

- Rational identity preserves the supplied numerator, denominator, and sign representation after rejecting a zero denominator.
- Unordered sets sort canonical item encodings; ordered sets preserve declared order; both preserve encoded multiplicity.
- System identity is referential and based on profile path.
- Snapshot identity uses kind and referential identity, leaving changing snapshot state outside the stable identity key.
- Schema encoding preserves descriptors for capabilities, scopes, native handles, and host effects; their live authority remains bound to the runtime that owns them.

[crates/ostadix-api/src/value.rs]

6.4 Live native objects and enforced value crossings

Python can export an arbitrary live object with `O.native(value)`. The resulting `OValue::Native` carries an opaque descriptor sealed to its exact process owner. Closures, aliases, cycles, generators, and mutable instances retain their Python identity because operations return to that owner. O builtins, Python, and the Unix JavaScript bridge can call, get, set, and release the exported object through the evaluator. The route checks the live owner, session, admitted launch generation, descriptor seal, and argument handles before invoking user code. Mutable or arbitrary results return owner handles; supported immutable scalar results cross as ordinary values.

```
let handle = python[7]^(O.native(lambda value: value + 2))_python[7]
javascript^(console.log(O.native_call(handle, 40)))_javascript
native_release($handle)
```

A persistent owner is required for later use. Owner exit invalidates the descriptor, and replacement actors cannot reconstruct the object. Native operations are effectful, inherit applicable deadlines, and provide no rollback of code already invoked. Outstanding handles pin Python checkpoint state, preventing migration from silently replacing their physical owner. [docs/PYTHON_NATIVE_HANDLES.md] [tests/python_native_handles.rs]

The separate opt-in `python-plain-data-lossless` morphism contract checks actual values crossing a Python adapter. Its carrier consists of exact booleans, null, arbitrary integers, finite bit-preserving f64 values, UTF-8 text, lists, and string-keyed maps arranged as finite trees. Input witnesses are checked before bindings change or source executes; a fresh invocation receipt binds input witnesses and the lifted result. Output rejection follows execution and suppresses successor publication without undoing effects. Live handles, shared or cyclic containers, subclasses, and values outside the carrier require their own contracts. This enforcement gives a concrete conversion guarantee for the admitted adapter and carrier. [docs/BACKEND_MORPHISM_ENFORCEMENT_V1.md] [tests/backend_morphism_enforcement.rs]

6.5 Wire protocol

Backend commands and responses use four-byte big-endian length-prefixed CBOR with a 128 MiB frame limit. Map encoding is canonicalized. Decoding rejects trailing bytes, indefinite encodings, malformed forms, non-text map keys, and non-finite encodings outside the defined value schema. [crates/ostadix-api/src/wire.rs] [crates/ostadix-api/src/canonical_cbor.rs]

The Rust serializer emits canonical `number` and `text` frames. Compatibility shims can emit legacy `int`, `float`, and `str` frames; Rust decoding normalizes them into `ONumber` and `OText` before the value enters the hosted algebra.

The complete shim-extension vocabulary is:

Direction	Tagged CBOR body
Runtime to shim	`cmd=exec` with `code` and a string-keyed `bindings` map of OValues.
Runtime to shim	Fieldless `cmd=cleanup` and `cmd=ping`.
Runtime to shim	`cmd=eval_result` with `value`.
Shim to runtime	`status=ok` with `value`; or `status=err` with `message`.
Shim to runtime	`status=eval_request` with `src` and optional `scope`. The runtime evaluates that O source, returns `eval_result`, and the paused shim resumes until `ok` or `err`.

[crates/ostadix-api/src/value.rs]

7 Backends, actors, and authority

7.1 Static registry

The compile-time Rust registry binds each recognized language tag to its renderer, execution mode, cache-safety classification, and required authority. It declares 30 canonical tags plus six aliases. Seven structural/value handlers execute inline or as builtins; external evaluation is mediated by native adapters and bundled compatibility shims. A single registry entry therefore connects surface syntax to planning metadata, state support, morphism profile, runtime requirements, and runtime dispatch. [crates/ostadix-api/src/backend_catalog.rs] [crates/ostadix-api/src/backend_catalog.inc.rs] [crates/ostadix-api/src/shims.rs]

Class	Registered canonical tags
Structural/value	`O`, `quote`, `nix_expr`, `html`, `markdown`, `latex`, `text`.

Class	Registered canonical tags
Nix and systems tests	`nix`, `nix_store`, `nixos_test`, `ubuntu_vm`.
Data and scripting	`sql`, `python`, `bash`, `shell`, `javascript`, `ruby`, `matlab`, `mathematica`.
Compiled and functional	`haskell`, `ocaml`, `webassembly`, `rust`, `racket`, `csharp`, `c`, `cpp`, `lisp`, `common_lisp`, `java`.

The six aliases are exact parser spellings: `o` maps to `O`; `py` maps to `python`; `md` maps to `markdown`; `tex` maps to `latex`; and `plain` maps to `text`; `ubuntu` maps to `ubuntu_vm`. Block attributes attach execution metadata directly to the evaluator-indexed form. The implemented attribute surface is `cap=name`, the four authority flags `fs_read`, `fs_write`, `network`, and `process`, `lazy`, `defer`, `effects=pure|unknown`, `reads=...`, `writes=...`, and `serial=host`. Effect-resource lists use `+` or `;` as separators. The resource spellings are:

```
project:path            host:path          env:NAME
scope:name              network:endpoint   service:name
actor:language[index]   evaluator          stdio
hostworld
```

These spellings let source describe graph resources in the same vocabulary used by the planner. [crates/ostadix-api/src/parser.rs] [crates/ostadix-api/src/effects.rs]

The repeatable backend-grant syntax is `NAME=LANG[:RIGHT,RIGHT]`. `NAME` begins with an ASCII letter or underscore and continues with ASCII letters, digits, or underscores. `LANG` is nonempty; known aliases canonicalize, and `*` creates a wildcard. Parser dispatch still requires a registered source tag. Rights are `fs_read`, `fs_write`, `network`, and `process`; commas separate them and duplicates collapse. An omitted or empty rights list produces a zero-rights bearer. A block that declares one or more authority flags uses `cap=NAME` as its first bearer candidate. A match must contain every declared right; an absent or insufficient binding follows the evaluator's default-authority path. The release CLI gate executed `runner=python:process`. [crates/ostadix-api/src/eval.rs] [tests/test_cli.sh]

At invocation time, the adapter resolves its host executable and converts the subset of OVALUE accepted by that runtime. This keeps language recognition deterministic while allowing the executable environment to be selected by the host installation.

7.2 Actor process model

Production external evaluation self-spawns the current executable in hidden `-o-backend` mode. The process registry key is

$$(\textit{language},\ \textit{environment ID},\ \textit{sandbox policy}).$$

Persistent keys reuse a process; ephemeral keys are removed after evaluation. Most backends are Rust adapters that invoke local executables. Python and `nixos_test` use legacy Python proxy shims. Python retains this path for the host callback protocol used by `O.eval`. [crates/ostadix-api/src/process.rs] [crates/ostadix-api/src/backend.rs]

Backend stdout is decoded in a defined fallback order: tagged or ordinary JSON, integer/big integer, float, then text. SQL additionally collapses a single-row, single-column result. Environment conversion is narrower than the full OVALUE universe. [crates/ostadix-api/src/backend.rs]

7.3 Hosted authority model

The source contains random 256-bit bearer identities, typed backend rights (`fs_read`, `fs_write`, `network`, `process`), broker checks, revocation, and policy-partitioned process reuse. [crates/ostadix-api/src/capability.rs]

The ordinary CLI executes backends with the ambient authority of the O process. Its launch path realizes that model in five steps:

1. evaluator startup mints a wildcard bearer with all four rights;
2. source `cap=` resolution falls back to that wildcard;
3. normal shim launch policies expand to all rights;
4. the production Python proxy runs under the outer process policy;
5. the macOS profile begins with `allow default` and derives denials from rights omitted by a narrowed embedding policy.

[crates/ostadix-api/src/eval.rs] [crates/ostadix-api/src/process.rs]

Authority is explicit runtime state

Hosted capability machinery supplies random bearer identity, typed rights, revocation, policy-partitioned actor reuse, deferred-request revalidation, and embedding hooks. The public CLI chooses the ambient policy, maximizing backend compatibility and execution capacity. An embedding can supply a narrowed process policy and reuse the same identity and revalidation path.

8 OIR, plans, effects, and HGraph execution

8.1 OIR is orchestration IR

OIR has five node forms:

`Text` `Load` `Store` `Invoke` `Exec.`

These forms model the complete hosted orchestration surface: source fragments, lexical dataflow, binding publication, builtin invocation, and evaluator execution. Lowering from the parser AST is structural; validation then turns that representation into an executable plan. The compact vocabulary makes every hosted construct projectable into the same graph ontology. [crates/ostadix-api/src/ir.rs]

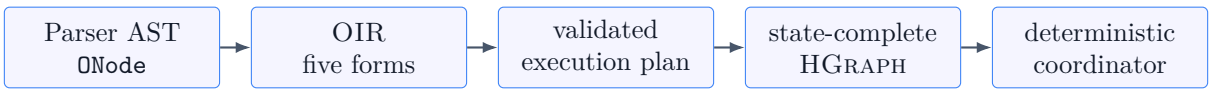


Figure 14: Every ordinary hosted execution is mediated by the same lowering seam; serial execution remains a differential oracle.

8.2 Execution plan

Plan node IDs are preorder-stable. Edges encode child-before-parent structure, sibling sequencing, binding dataflow, capability dependencies, and the full visible scope sent to shims. Each executable node

freezes the registry-derived `BackendInterface`; execution rejects an interface that no longer matches the registry. [crates/ostadix-api/src/ir.rs] [crates/ostadix-api/src/eval.rs]

The validated plan also builds an `ExecutionPlanIndex` for node, incoming-edge, and outgoing-edge lookup. Identity, root, bounds, and cycle validation remain at construction. Indexed adjacency retains original edge order and duplicate edges, preserving the dependency semantics consumed by lowering and scheduling while avoiding repeated full-plan scans. [crates/ostadix-api/src/ir.rs]

8.3 State-complete HGraph

Each ordinary plan node maps to one value node. Every executable hyperedge also consumes required completion tokens and versioned resource states, then atomically produces:

1. one ordinary value;
2. one completion token;
3. successor versions for every accessed resource.

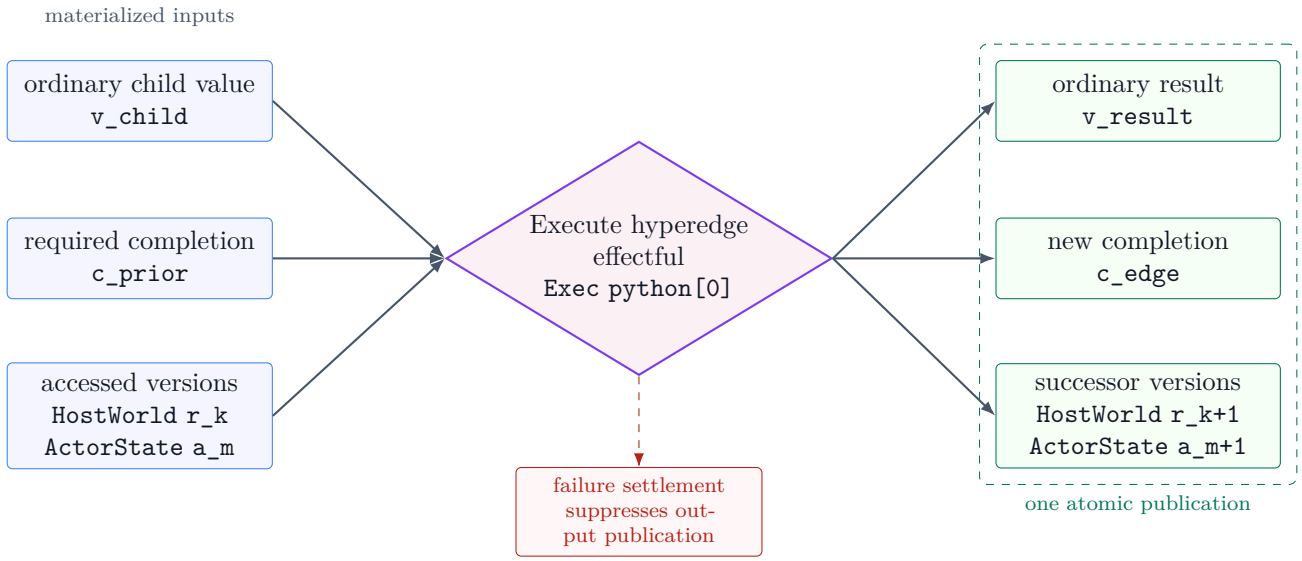


Figure 15: An executable HGraph hyperedge consumes value, completion, and resource state together. This persistent evaluator example advances `HostWorld` and `ActorState(python[0])`. Success publishes the result, completion token, and every successor resource version as one graph transition; failure withholds those graph outputs.

[crates/ostadix-api/src/hgraph/from_oir.rs]

Validation checks unique producers, complete outputs, exact source/plan provenance, sequence preservation, contiguous resource chains, and executable acyclicity. The ready schedule is derived from unresolved inputs and their producers with stable wave and ordinal ordering. [crates/ostadix-api/src/hgraph/graph.rs] [crates/ostadix-api/src/hgraph/schedule.rs]

8.4 Effects, worker execution, and cancellation

The executable HGraph carries values, completion dependencies, and explicit state for evaluators, actors, scope, paths, environment, stdio, services, and the host world. Unknown effects retain conservative

`HostWorld` dependencies. Ordinary fallible shim work stays on the evaluator owner thread; compiler-verified scope reads and pure, deterministic, infallible inline renderers can run on workers. Explicit `autonomous(...)` groups also admit eligible ephemeral backend tasks with owned materialized inputs and fresh live capability checks. Their evidence marks unordered host effects: deterministic outcome selection does not imply serial equivalence or rollback of effects from already-started tasks. [crates/ostadix-api/src/effects.rs] [crates/ostadix-api/src/executor/parallel.rs]

The coordinator refills available workers as dependencies become ready. Eligible provisional results can enable safe dependents before deterministic trace settlement; autonomous publication remains scoped to its admitted group. The default local pool is bounded by host parallelism and the admitted static worker wave. An explicit worker count changes capacity while retaining all dependency and admission constraints. A static schedule's width predicts neither measured overlap nor a machine's available memory or backend readiness. [crates/ostadix-api/src/executor/coordinator.rs] [docs/CLAIMS.md]

Cancellation uses parent-aware tokens shared by embedding requests, backend execution, and competing group members. Cancelling a branch does not cancel its siblings; request cancellation reaches descendants. Tokens request a stop at safe points, while the execution owner performs bounded process cleanup. The coordinator controls acceptance, publication, and settlement, so a late worker completion cannot independently commit graph state. External effects that occurred before cancellation remain part of the execution history. [crates/ostadix-api/src/cancellation.rs] [crates/ostadix-api/src/process.rs] [crates/ostadix-api/src/executor/coordinator.rs]

9 Projects, linking, and route policies

9.1 Project lifting and literal linking

`o-link` has two materially different composition lanes:

- A bare single directory follows the literal lane and runs the linked result. Explicit `-literal` emits the literal linked document; `-literal -run` executes it.
- Explicit `-project` captures a deterministic route-preserving project bundle. `-project -run` selects and executes a route through the project coordinator.

[src/bin/olink.rs]

9.2 Compiler-emitted Project HGraph from an o-linked codebase

The following retained compiler illustration was produced for the August 2026 edition. Its hashes and dimensions identify that historical artifact; the reproduction commands regenerate the view with the selected compiler.

The tracked fixture in `tests/fixtures/project_hgraph/` is a complete project codebase with one input file, a manifest, one prerequisite route, two implementation routes, declared inputs and outputs, an environment guard, and a `verify_equivalent` route set named `main`. I lifted that directory through the project form of `o-link`, asked `olangc -target dot` to render the selected route set, and compared the result with direct directory planning.

```
mkdir -p output/current-master
./target/release/o-link tests/fixtures/project_hgraph --project \
    -o output/current-master/o-linked-project.0
```

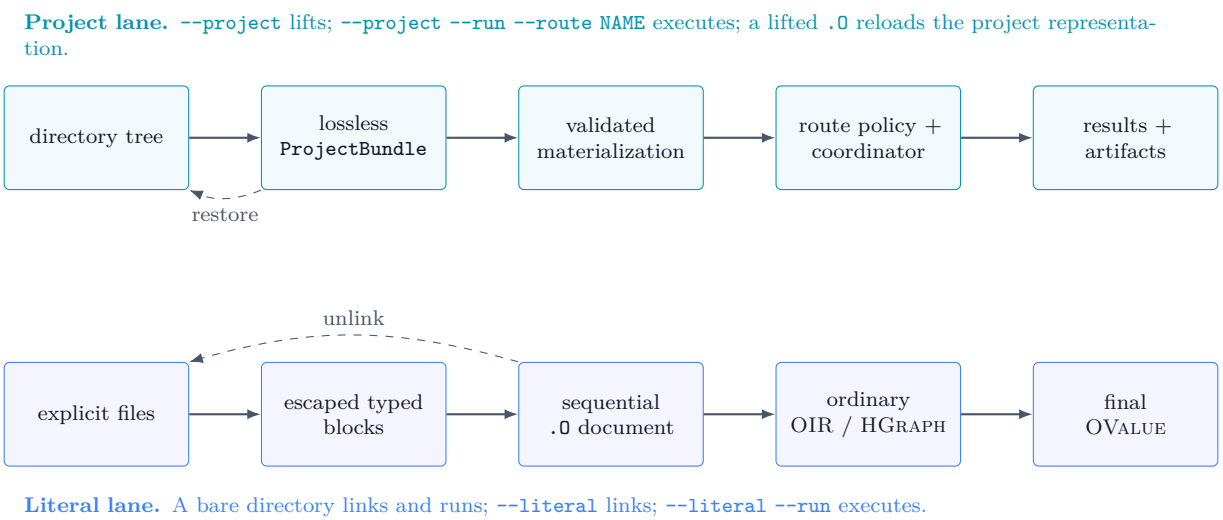


Figure 16: `o-link` preserves two composition semantics. Explicit project lift captures a reversible tree and selects whole-project routes; literal linking constructs one hosted program and a bare single directory runs that program by default.

```
./target/release/olangc output/current-master/o-linked-project.O \
    --target dot --route main --shim-dir backends \
    > output/current-master/o-linked-codebase-hgraph.dot

./target/release/olangc tests/fixtures/project_hgraph \
    --target dot --route main --shim-dir backends \
    > output/current-master/project-hgraph-direct.dot

cmp output/current-master/o-linked-codebase-hgraph.dot \
    output/current-master/project-hgraph-direct.dot

cp output/current-master/o-linked-codebase-hgraph.dot \
    docs/figures/o-linked-codebase-hgraph.dot
dot -Tsvg output/current-master/o-linked-codebase-hgraph.dot \
    -o docs/figures/o-linked-codebase-hgraph.svg
dot -Tpdf output/current-master/o-linked-codebase-hgraph.dot \
    -o docs/figures/o-linked-codebase-hgraph.pdf
```

The lifted artifact is 7,501 bytes with SHA-256 `a6d97fd5b95b829c3e3a754722767960f2963e0644ba8c89da7c15d48d922bb1`. The compiler-emitted DOT is 13,065 bytes with SHA-256 `bf16748f53a377e950a67b9c27d48ff260dcc987a69d86c5f29ca6289df7f00b`. The original direct-directory and lifted-artifact DOT outputs were byte-identical. The SVG is 61,756 bytes with SHA-256 `44d7d33abf7d34ed73e3f4f37ebee77dbe5545f52cfc0077c64c021015310d01`. The included vector PDF is 27,210 bytes with SHA-256 `75c7d219d208a2afddc51a5c9c682c506d26cd9729145828019f88f1c6a8e90e`.

[tests/fixtures/project_hgraph/olang.project.toml] [src/bin/olink.rs] [src/bin/olangc.rs] [crates/ostadix-api/src/project/plan.rs]

How to read the emitted graph

The figure declares 53 semantic vertices and 62 directed edges. Twelve rose `Execute` boxes are project operations. The other 41 vertices are values, completion tokens, and versioned resources. Read it in

seven passes:

1. Begin at `Value N0`. It identifies the exact project bundle consumed by both alternative branches.
2. Find `Execute E0` through `Execute E4`. This is the `impl-a` branch: materialize the project, build and run `prepare`, then build and run `impl-a`.
3. Find `Execute E5` through `Execute E9`. This is the independently materialized `impl-b` branch with its own `prepare` prerequisite and `impl-b` execution.
4. Follow completion diamonds from each build operation into its run operation. A completion edge expresses successful prerequisite settlement; a value edge carries an ordinary result.
5. Follow cyan resource nodes. `HostWorld` advances from `@0` through `@6`; project input, build, and output resources advance through the operations that read or write them.
6. Find `Execute E10`. It compares the two route results required by `verify_equivalent` after both alternatives have completed.
7. Finish at `Execute E11`. It selects the route-set result from the comparison outcome and publishes the final value and completion.

The graph should be read by dependency, then by page position. Arrow labels name the port through which a node participates: `input:value`, `input:completion`, `input:resource-state`, `result`, `completion`, and `resource-state`. Each `P# ordinal #` label binds an execute box to its project plan operation and gives deterministic ordering among simultaneously ready operations. A resource version belongs to the state after the operation that produced it. A completion token belongs to the terminal settlement of its producing operation.

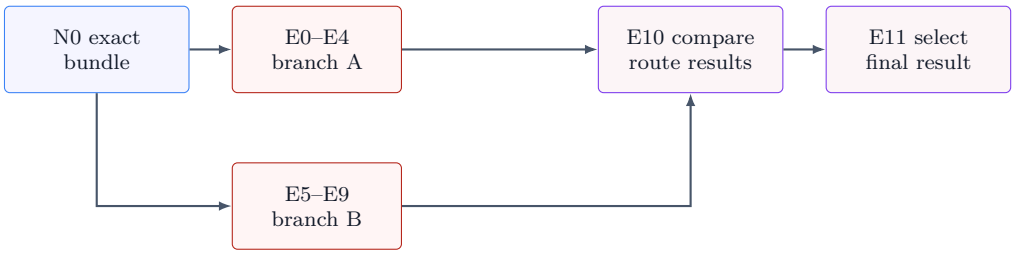


Figure 17: Reading map for the current compiler-emitted project graph. The two source-closed alternative branches converge through comparison and final selection.

The panorama preserves the exact compiler artifact. The four detail figures that follow it enlarge overlapping horizontal windows from that same vector PDF. Read them from left to right. The overlap carries crossing arrows into the next window, so a port that leaves one page can be found again at the matching edge of the following page.

Topology index

Use this panorama to see branch shape and convergence. Read node and port labels in the four enlarged panels that follow.

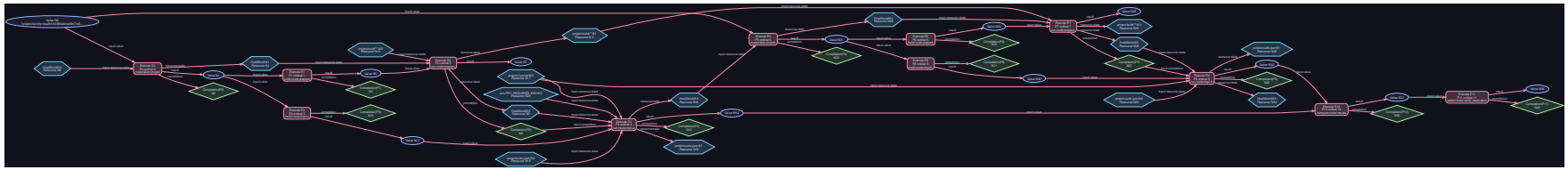


Figure 18: The exact directed Project HGraph emitted by `olangc -target dot -route main` from the o-linked `tests/fixtures/project_hgraph` codebase. The graph contains 12 executable operations, 41 value, completion, and resource vertices, and 62 directed dependency edges.

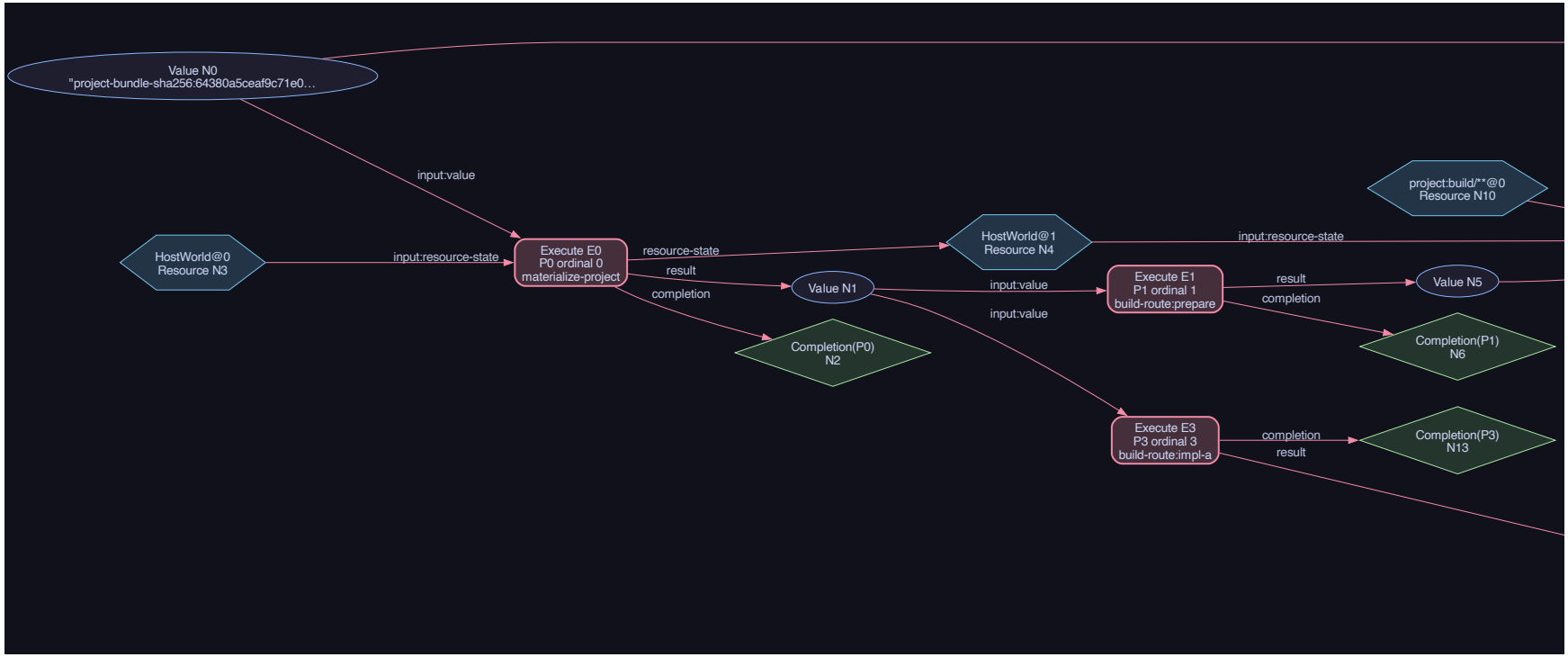


Figure 19: Project HGraph detail 1 of 4. Begin at the exact project identity, then follow materialization and the opening operations of the `impl-a` branch.

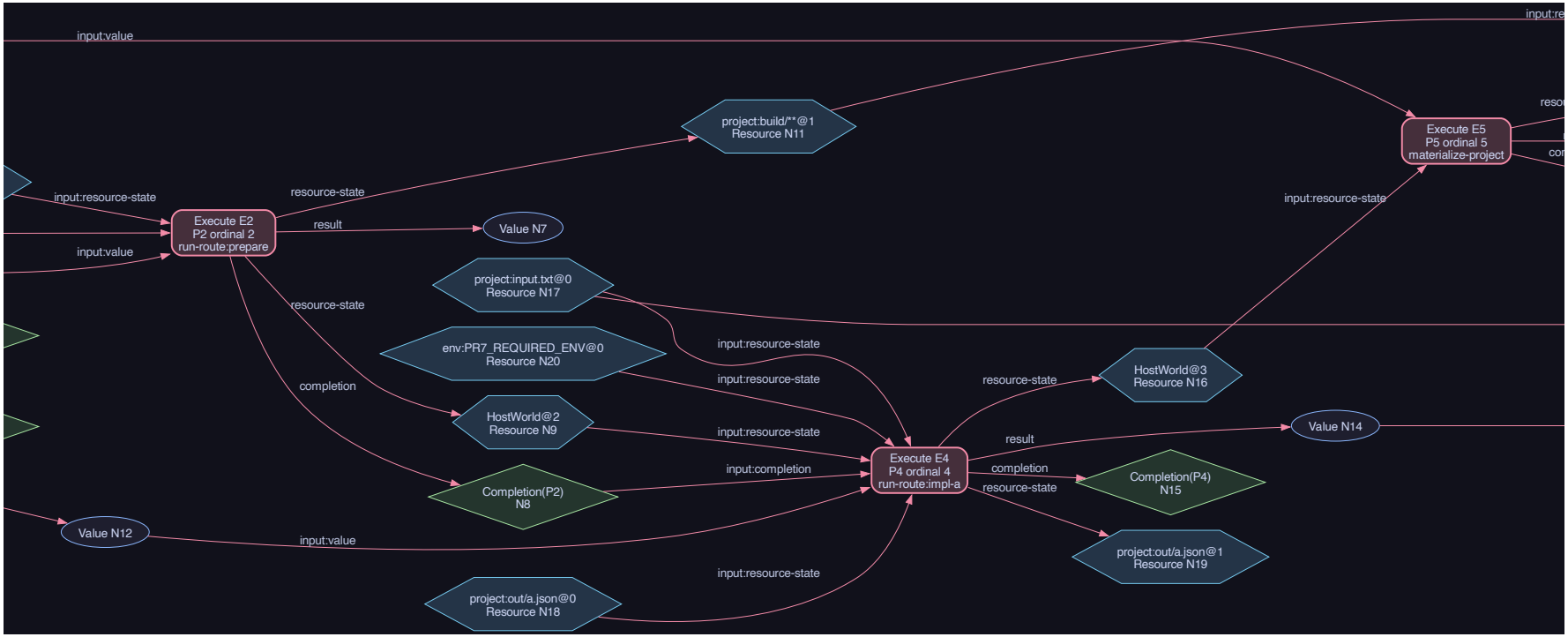


Figure 20: Project HGraph detail 2 of 4. Complete the `impl-a` route, then locate the independently materialized opening of the `impl-b` branch.

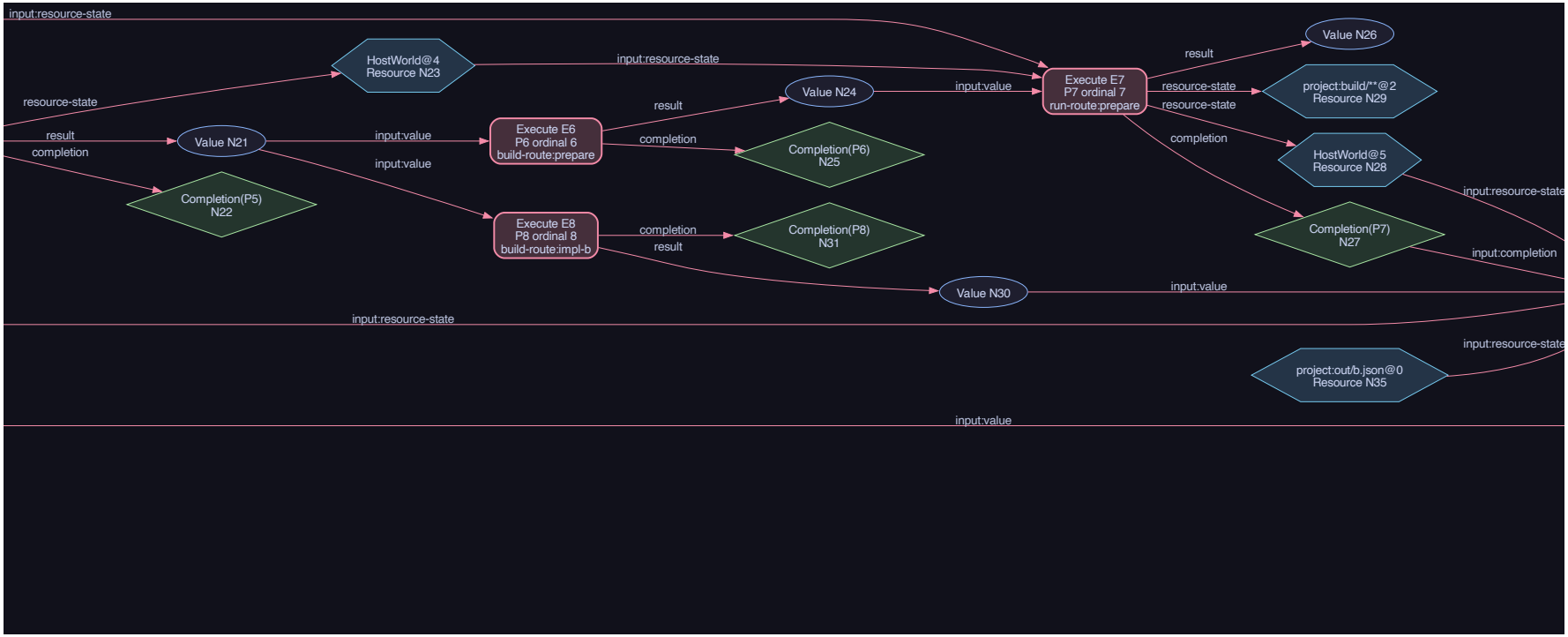


Figure 21: Project HGraph detail 3 of 4. Follow the build and run operations of `impl-b`, including their completion and resource-state ports.

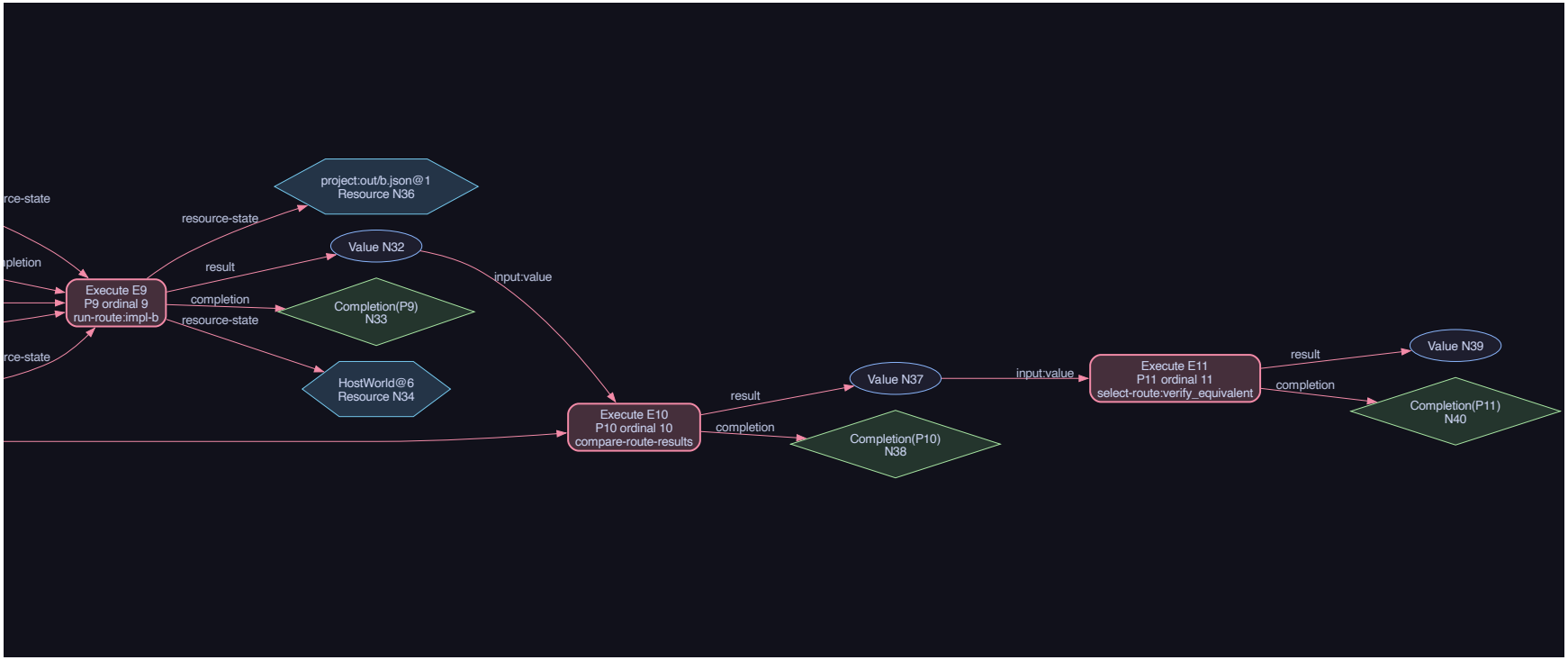


Figure 22: Project HGraph detail 4 of 4. Settle the second branch, compare both route results at E10, and publish the selected route-set result at E11.

The diagram exposes why project planning is a language operation. Directory capture, prerequisite closure, alternative isolation, input and output resources, equivalence comparison, and selection all become typed graph operations. The exact-equality check shows that lifting preserves the project semantics consumed by the compiler.

9.3 Typed grounding across planning front doors

Grounding exposes how an execution graph relates values, effects, actor state, and authority requirements to its execution substrate. Ordinary O and project inputs have separate typed reports, shared by `o plan` and `olangc`. Both are inspection operations.

For ordinary source, the prepared path reconstructs its `GroundingReport` from captured OIR, checks OIR, plan, solved-graph, and execution-intent identities, and reports governed and ambient effects, OValue flows, capabilities, actor affinity, capsules, and residual `HostWorld`. An optional paired `-world-id NAME -world-epoch N` supplies a caller-selected World identity with a nonzero epoch. It performs no live-state lookup or freshness check.

For a selected project, `ProjectGroundingReport` derives the canonical `LogicalHGraphV1` and its hosted-unbound `DeploymentPlanV1`, validates their correspondence, and records their exact digests. Every deployment operation must refer to its matching logical operation and retain the same descriptive authority requirements. The report exposes operation kind, effects, requirements, hosted binding, and residual `HostWorld`. The prepared `o plan` path checks the rebuilt bundle, logical, and deployment identities against captured preflight state before returning.

```
olangc tests/fixtures/project_hgraph --target ir --route main --grounding
o plan tests/fixtures/project_hgraph --route main --grounding
```

Two independent predicates make the report precise. `authority_requirements_absent` describes an empty list of requirements; `non_authorizing` describes absence of live World, snapshot, provider, and task bindings. The inspection grants no authority and dispatches no route. Residual `HostWorld` identifies ambient hosted effects; an empty residual set alone would not prove complete mediation. Project grounding has no World binding and rejects the World identity flags. Grounding is a standalone text view; JSON output, live placement preview, and execution-intent JSON retain separate interfaces.

The regression suites compare actual compiler/CLI report bytes, verify exact identity linkage, and check that inspection creates no execution markers or node state. These tests specify current source behavior. The installed tools checked during this revision predate the project-grounding implementation; Section 14 records that version mismatch. [crates/ostadix-api/src/project/grounding.rs] [crates/ostadix-api/src/intent/mod.rs] [crates/ostadix-api/src/world/grounding.rs] [tests/unified_plan_boundaries.rs] [tests/project_hgraph.rs]

9.4 Bundle model

`ProjectFile` preserves relative path, raw bytes, Unix mode, symlink metadata, evaluator hint, role, and content hash. A project carries route specifications, guards, provenance, effects, and policy. Traversal is deterministic and skips generated/build/cache subtrees. Materialization validates relative paths, ancestor symlinks, and in-root symlink targets. [crates/ostadix-api/src/project/model.rs] [crates/ostadix-api/src/project/bundle.rs] [crates/ostadix-api/src/project/materialize.rs]

The bundle fingerprint covers artifact identity through paths, content hashes, modes, and symlink flags. Route metadata remains a separately selected execution policy. On Unix, materialization reproduces modes and symlinks from the captured model. [crates/ostadix-api/src/project/bundle.rs] [crates/ostadix-api/src/project/materialize.rs]

Six file roles classify the captured tree: `Source`, `Asset`, `Manifest`, `GeneratedRoute` `Metadata`, `Entrypoint`, and `Other`. Route discovery proceeds in a fixed ecosystem order covering Rust, Python, JavaScript, shell, C-family, Java, .NET, Haskell, OCaml, Nix, and generic entrypoints. An `olang.project.toml` manifest can state exact routes and route sets while retaining the captured files as the artifact of record. [crates/ostadix-api/src/project/discover.rs] [crates/ostadix-api/src/project/manifest.rs]

9.5 Routes

The native project coordinator implements `explicit`, `default`, `fallback`, `any_success`, `race_success`, `race_settle`, `all`, `verify_equivalent`, and `benchmark_and_select`. Alternatives run as subprocess routes with isolated or shared workspaces as required; both race policies cooperatively cancel losing route processes. [crates/ostadix-api/src/project/model.rs] [crates/ostadix-api/src/project/runtime.rs]

The project coordinator owns whole-tree materialization, route selection, subprocess lifecycle, result settlement, and workspace sharing. Ordinary OIR/HGraph execution owns expression-level evaluator composition. This division gives project routes enough context to coordinate complete build and run alternatives while preserving the hosted language's value graph. [crates/ostadix-api/src/project/manifest.rs] [crates/ostadix-api/src/project/runtime.rs]

Route model	Variants	Recorded execution facts
Kind	Interpreter command, compiled binary, build target, package entrypoint, shell task, O evaluator, composite.	Identity, command or evaluator, entrypoint, working directory, arguments, environment, prerequisites, inputs, outputs, priority, and default status.
Codec	Text, JSON, bytes.	Defines stdout interpretation and the optional structured JSON result.
Guard	Operating system, command availability, environment variable.	Controls candidate eligibility before launch.
Provenance	Manifest, discovery, CLI override.	Records where executable policy entered the bundle.
Effects	Pure or unknown, plus read and write resources.	Connects project declarations to scheduling and workspace decisions.
Result	One settled route execution.	Route ID, exit status, stdout and stderr bytes, optional JSON, artifacts, duration, command provenance, and workspace provenance.

9.5.1 Authoring project routes

`olang.project.toml` exposes the complete manifest route vocabulary and route-set policy as an authorable format. Manifest routes replace discovered routes with the same ID, and route sets replace sets with the same `provides` key. The reader tolerates additional fields so future metadata can remain attached to a bundle.

Section	Fields
[project]	Optional name and default_route.
[[routes]]	Required id; optional label, command, kind, evaluator, entrypoint, cwd, args, env, depends_on, inputs, outputs, provides, result_codec, priority, default, guards, and pure.
[routes.guards]	Optional os, requires_command, and requires_env gates on the preceding route.
[[route_sets]]	Required provides and alternatives; optional policy: explicit:ID, default, fallback, any_success, race_success, race_settle, all, verify_equivalent, or benchmark_and_select.

The repeatable -route-decl override uses semicolon-separated key=value fields. id and cmd are required; the remaining recognized keys are label, cwd, args, env, provides, depends, outputs, codec, kind, priority, and default. Commands are whitespace-split, list values are comma-separated, and env contains comma-separated K=V pairs. [crates/ostadix-api/src/project/manifest.rs]

```
[project]
name = "project-policy-matrix"
default_route = "a"

[[routes]]
id = "a"
command = ["/bin/echo", "42"]
provides = ["value"]
result_codec = "json"
priority = 2
default = true

[[routes]]
id = "b"
command = ["/bin/echo", "42"]
provides = ["value"]
result_codec = "json"
priority = 1

[[route_sets]]
provides = "verify-case"
alternatives = ["a", "b"]
policy = "verify_equivalent"
```

The nine public selectors encode distinct settlement algorithms. explicit chooses a named route; default chooses the declared default; fallback tries routes sequentially in descending priority order; any_success tries declaration order; race_success settles on the first success; race_settle settles on the first completion; all returns every sequential result, including nonzero exits; verify_equivalent runs alternatives concurrently, requires success, and compares structured results; and benchmark_and_select retains every parallel result while placing the fastest success last as the effective selection. Route sets are declared explicitly in the project manifest. A route's provides metadata describes capability publication and remains independent of route-set membership.

The tracked tests/fixtures/project_hgraph/olang.project.toml supplies a compact authoring example. Its main route set chooses verify_equivalent over impl-a and impl-b; both alternatives

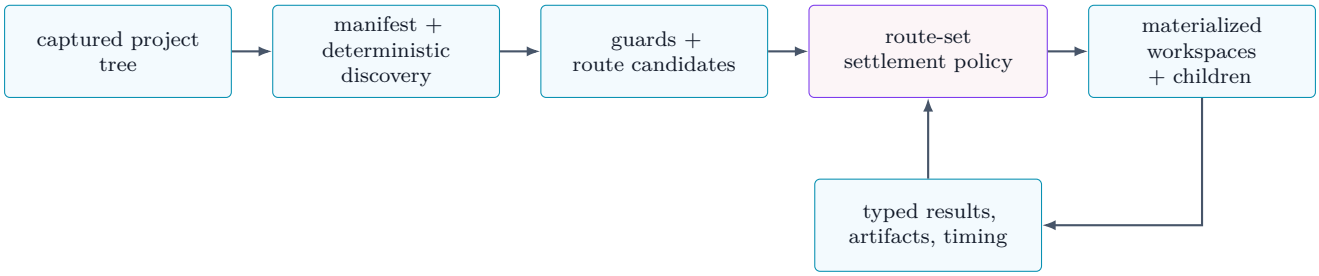


Figure 23: Project capture and policy settlement. A route set observes concrete results until its policy can settle, cancel, compare, or rank candidates.

depend on `prepare`. The compiler graph in Section 9.2 shows the resulting isolated branches, comparison operation, and final selection directly.

```
./target/release/o-link tests/fixtures/project_hgraph --project \
  -o output/current-master/o-linked-project.O
./target/release/olangc output/current-master/o-linked-project.O \
  --target ir --route main --shim-dir backends
./target/release/olangc output/current-master/o-linked-project.O \
  --target dot --route main --shim-dir backends
```

9.6 Unlinking

`o-unlink` restores both embedded project bundles and literal linked documents. Project restoration validates relative paths, ancestor symlinks, and in-root symlink targets before writing. Literal restoration decodes the escaped file boundaries and rejects absolute paths and parent traversal. [src/bin/ounlink.rs]

10 Toolchain and implementation editions

10.1 Public binaries

The committed root Cargo package declares fifteen binaries. The separately locked MCP server adds its own executable.

Binary	Implemented role
`O`	Hosted interpreter, REPL, executor selection, compatibility backend grants, and hidden backend worker mode.
`o-cli`	Compiled intent and operational dispatcher, shared command discovery, operation planning and observation, and durable run records.
`olangc`	Hosted packaging/compiler front end: binary, WASM, script, IR, and DOT.
`ocorec`	O-CORE source and verified native-module compiler: inspect source IR, import supported LLVM and emit owned ISA assembly or objects.
`o-link` / `o-unlink`	Literal linking and safe project bundle lift/restoration.

Binary	Implemented role
`o-notebook`	Feature-gated local notebook server with per-process O scope and typed result rendering.
`ogit`	Bounded semantic-receipt and policy-aware diff demonstration.
`o-live-host`	Hosted Live-World lifecycle oracle.
`o-node` / `octl`	Hosted node service, discovery, reciprocal pairing, paired TLS operation, and node-control client.
`o-registry` / `o-info`	Registry V1 event operations and Information-root inspection.
`ostadix-device`	Native Android/Termux device control, diagnostics, deployment, and supported device operations.
`ocore-kernel-world-record`	Verify a strict package/payload pair and encode its deterministic native admission record.

[Cargo.toml]

10.2 What olangc targets mean

Target	Implementation	Execution model
`binary`	Generates a temporary Cargo project containing embedded source, runtime modules, shims, and a lockfile; builds a host executable.	The generated executable embeds the hosted runtime, reparses its source at startup, and evaluates through the ordinary graph path.
`wasm`	Direct generated-runtime `wasm32-wasip1`, or explicit Linux-image packaging with paired builder/runtime image flags.	Direct inline execution or a packaged emulated Linux guest, according to the selected profile.
`script`	Parses and invokes the normal evaluator inside `olangc`.	This is the direct compiler-driver entry into hosted graph evaluation.
`ir`	Prints OIR, the execution plan, and HGraph.	It exposes structural lowering, frozen interfaces, dependencies, and resource state in one inspection stream.
`dot`	Solves and renders directed graph operation ports.	It visualizes executable operations and the value/resource ports connecting them.

[src/bin/olangc.rs]

Project inputs compile through binary, WASM, and script targets. The generated project runtime restores the captured tree and executes its selected subprocess route, carrying whole-project structure

into the packaged artifact. Project directories and lifted bundles also enter the IR and DOT targets, which build the exact logical Project HGraph for the selected route or route set. A generated project WASI module contains the project runtime; its route operations use the process facilities supplied by the execution substrate.

10.3 Runtime payloads and Linux filesystem isolation

For an ordinary native O binary, `-runtime-bundle DIR` embeds a supplied relocatable runtime tree, retaining regular files, internal file symlinks, empty directories, permissions, and a digest inventory. Startup verifies and privately extracts the payload, then confines runtime command lookup to its `bin/` before evidence and admission. The default host profile still depends on the host kernel and each supplied library/service closure. It does not infer arbitrary dynamic dependencies or certify every catalog runtime. This option does not apply to project binaries or WASI. [docs/EMBEDDED_RUNTIME_BUNDLES.md]

The explicit `linux-rootfs-v1` profile enters fresh user, mount, PID, network, IPC, and UTS namespaces before evaluator or multicall dispatch. It detaches the original root, makes the payload read-only, clears ambient environment and extra descriptors, drops capabilities, and installs `no_new_privs` and a mount syscall filter. Private temporary filesystems, loopback, threads, subprocesses, and local sockets remain available. A namespace init reaps descendants and preserves evaluator exit status. Entry failure stops evaluation. The image identity and namespace conditions are checked on multicall reentry; an environment marker alone cannot authorize entry. Linux x86-64/AArch64 support depends on available namespace facilities, and the host kernel and initial loader remain trusted. [docs/EMBEDDED_RUNTIME_BUNDLES.md] [tests/linux_runtime_rootfs.rs]

The Linux collector resolves declared ELF loaders and `DT_NEEDED` dependencies without executing the input programs. Runtime-loaded modules, data, services, and computed library names still require explicit inputs and qualification of the resulting executable. [scripts/collect_runtime_rootfs.py] [docs/LINUX_RUNTIME_ROOTFS.md]

10.4 Direct WASI and packaged Linux execution

The direct `wasm32-wasip1` target compiles the generated O runtime. Inline-only plans can run locally; shim-backed or effectful browser plans require an explicit whole-program provider. A separate opt-in route accepts digest-pinned runtime and builder images, builds the native evaluator inside Linux, and packages that environment through container2wasm into a WASI module containing a Linux guest and CPU emulator. Supplied foreign runtimes execute inside that guest. The two routes have distinct browser bundle schemas and hosts; packaged Linux execution does not make Python a direct-WASI backend. [src/bin/olangc.rs] [docs/OLANGC_WASM.md]

Tracked documentation records a 12 September 2026 Python fixture build and execution under Wasmtime 47.0.3 and Wasmer 7.2.1, with success/failure statuses 0/1, followed by headless Chrome 152 execution of the retained artifact. The recorded artifact is 156,555,045 bytes. Those dated observations qualify that fixture and image, and were not rerun for this whitepaper revision; the browser receipts are recorded as retained outside the checkout. The optional interactive Guix browser profile is present in source but still lacks a fresh build and installation/persistence qualification. [docs/OLANGC_WASM.md]

10.5 Reference implementations

The Rust edition is authoritative and implements the full hosted graph, project, Live-World, and O-CORE surfaces. The Python edition identifies itself as the original/reference implementation at version

0.4.0. It has a compact recursive evaluator and uses environment 0 as persistent by default. The C17 edition supplies a portable hosted interpreter and hosted AOT path. Together, these editions provide independently executable semantic references around the authoritative Rust graph runtime. [o_lang/README.md] [o_lang/__init__.py] [o_lang/evaluator.py] [c_cpp/README.md]

Edition	Parser-visible evaluator registry	Implementation surface
Rust 0.4.0 Unreleased	30 canonical tags and six aliases listed in Section 7.	OIR, plans, HGraph, both schedulers, projects, native and WASI generation, REPL, notebook, O-Git, Live-World, KernelWorld records, and O-CORE.
Python 0.4.0	`python`, `markdown`, `html`, `latex`, `text`, `0`, `quote`, `nix`, `nix_store`, `nixos_test`, `common_lisp`, `javascript`; aliases `py`, `md`, `tex`, `plain`, `o`.	Recursive evaluation, AST and output projections, a small typed embedding API, and stateless Common Lisp/JavaScript shim crossings.
C17 interpreter and AOT	One registry shared by the interpreter, meta-evaluator, and generated C: 22 shipped shim tags, together with `0`, `html`, `latex`, `markdown`, `text`, `quote`, and `nix_expr`.	Hosted evaluation and generated-C AOT; AOT packaging includes every advertised shim and both imported support modules. Runtime discovery and value support remain distinct from parser registration.

The Python reference evaluator retains its in-process Python environments, but environment lifetime is backend-specific. Its added `common_lisp` and `javascript` adapters invoke the shipped framed-CBOR shims in a fresh process for each block, including blocks with numeric environment labels. They supply executable coverage for the nested Python–Common Lisp–JavaScript greeting without claiming the Rust runtime's native-object ownership or admitted-execution protocols. [o_lang/backends/__init__.py] [o_lang/backends/shim_backend.py] [tests/test_python_shim_backend.py]

The C17 shim registry contains `python`, `nix`, `nix_store`, `nixos_test`, `bash`, `shell`, `rust`, `racket`, `javascript`, `common_lisp`, `lisp`, `cpp`, `csharp`, `haskell`, `java`, `mathematica`, `matlab`, `ocaml`, `ruby`, `sql`, `ubuntu_vm`, and `webassembly`. The `OLANG_SHIM_BACKENDS` definition constructs the parser registry and the AOT shim-file inventory. This removes the earlier divergence in which an evaluator could implement a backend while the parser did not recognize a typed binding on its right-hand side. The registration test uses deterministic wire peers, then removes the original source and shim directory before running the packaged executable; it tests routing and packaging independently of installed language engines. [c_cpp/include/backends.h] [c_cpp/src/main.c] [c_cpp/src/eval.c] [c_cpp/src/olangc.c] [c_cpp/tests/test_backend_registration.py]

The Python shim adapter accepts one response frame and rejects trailing data, malformed responses, unsupported value variants, and failed child processes. Its lifting subset includes null, booleans, arbitrary-size integers, IEEE-754 binary64 values, strings and HTML, lists, string-keyed maps, blobs, and store paths. Binary64 decoding preserves finite bit patterns and signed zero; no conversion to arbitrary binary formats or native handles is implied. Child values use the generic source-splice representation; the adapter does not infer a JavaScript or Lisp quoting context. A bounded subprocess timeout terminates the owned POSIX process group, including an engine descendant, without silently retrying a failed computation. [o_lang/backends/shim_backend.py] [tests/test_python_shim_backend.py]

The Python package root exposes its complete small embedding vocabulary:

API family	Public names
Parse and evaluate	`run`, `parse`, `evaluate_document`, `EvalContext`, `pretty`.
Syntax tree	`Document`, `ExpressionNode`, `TextPart`.
Values	`OValue`, `ONull`, `OBool`, `OInt`, `OFloat`, `OStr`, `OHtml`, `OStorePath`, `OList`, `OMap`, `OScope`, `OBlob`, `OExpr`.
Conversion and rendering	`from_python`, `to_python`, `render_plain`, `to_json_str`.

[o_lang/__init__.py]

The C17 edition publishes embedding headers and a shared backend registry:

Header	Public integration surface
`backends.h`	Shared parser registry and shim inventory for the interpreter, meta-evaluation and generated-C AOT.
`value.h`	Tagged OValue union, constructors, reference counting, maps, content identity, JSON, length-prefixed CBOR frames, SHA-256, and base64.
`parser.h`	AST storage, registered-language sets, parser lifecycle, tree destruction, and source reconstruction.
`eval.h`	Eager, lazy, and autonomous policy; evaluator lifecycle, document and source evaluation, rendering, and shim lookup.
`process.h`	Backend processes, reentrant evaluation steps, and the persistent $(language, environment)$ process registry.
`scheduler.h`	Disk cache, configurable autonomous scheduler, batch callbacks, and transitive-request collection.
`nix_ops.h`	Typed instantiate, realise, activate or dry-activate, and current-system transitions.

[c_cpp/include/value.h] [c_cpp/include/parser.h] [c_cpp/include/eval.h] [c_cpp/include/process.h] [c_cpp/include/scheduler.h] [c_cpp/include/nix_ops.h]

Demonstration and infrastructure fixtures. The public `examples/hello.O` now constructs a greeting through Python, Common Lisp, and JavaScript bindings. It exercises actual language composition and therefore requires the corresponding hosted engines. Setup verification and the MCP `o_smoke` tool instead materialize a private, fixed Python computation of $1 + 1$, whose expected result is 2; those checks remain stable when the public example changes. The hosted boot harness still runs the public greeting. It compares captured graph and serial output bytes, including trailing newlines, and reports an installed reference edition's execution failure as a failure rather than a successful skip. These fixtures separate engine-composition regressions from infrastructure readiness. [examples/hello.O] [setup.sh] [boot-and-test.sh] [mcp/ostadix_lang_mcp_server/src/main.rs]

Editor execution paths. The editor installer distributes locally packaged syntax support and, on macOS and Linux Code-family editors, a managed *Ostadix: Run Current File* launch configuration. The launch selects the runtime and backend paths from the checkout that ran the installer and passes the active filename through an environment variable to the terminal launcher. Optional Code Runner

mappings use that extension's own filename substitution. Installation preserves unrelated settings and JSONC comments, backs up changed files, and supports read-only check and dry-run modes. This integration invokes the native runtime; it supplies neither language-level breakpoints nor stepping, and installing syntax alone does not build the executable. [tools/syntax/install.py] [tools/syntax/vscode_settings.py] [tools/syntax/README.md]

10.6 REPL and notebook state

The interactive surfaces expose two deliberate state lifetimes. The REPL retains O lexical bindings and evaluator-owned backend actors across entries. Its `:r` command replaces the O lexical scope while leaving persistent actor processes available. The notebook binds `127.0.0.1:8888` and places one `Session` behind a process-wide mutex. Every browser client therefore observes the same O scope and evaluator registry. A POST to `/reset` replaces that complete session, clearing both lexical bindings and actor state. GET `/` serves the embedded interface, while POST `/eval` returns tagged null, text, HTML, or image output.

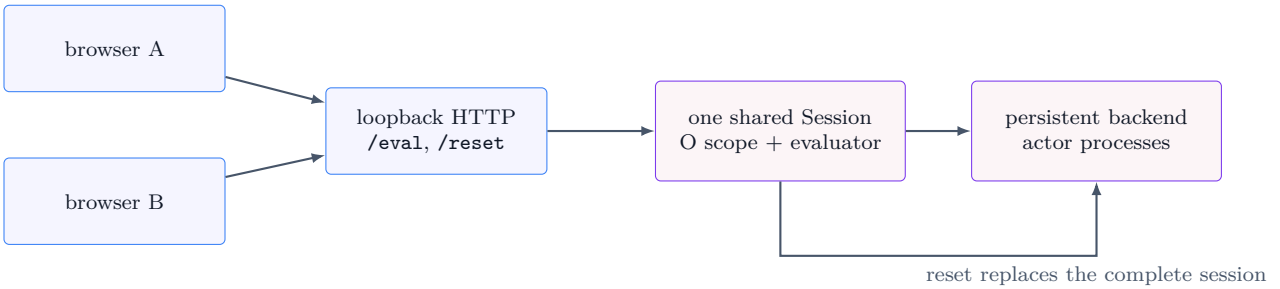


Figure 24: The local notebook has one process-wide interactive state. Cell evaluation preserves both O bindings and backend actors; reset creates a new scope and evaluator together.

10.7 O-Git

O-Git demonstrates semantic receipts and policy-aware diffs. A receipt records source and target paths, group declarations, output, and qualitative properties classified as preserved, lowered, erased, or assumed. The command turns semantic transformation choices into a machine-readable artifact that can be compared alongside ordinary repository history.

[src/bin/ogit.rs] [src/bin/ogit.rs]

The checked-in `examples/group_pipeline/main.O` and `main.eager.O` are receipt inputs that expose a lazy-to-eager policy change. O-Git lowers and executes its generated C target, then records the semantic transformation. Direct hosted O evaluation encounters their unforced `{defer}` splice, so an executable hosted variant would place `now(...)` at that force point. This is the exact semantic distinction the receipt is designed to preserve.

11 O-core: the native language

11.1 Language surface

Every `.oc` unit begins with a module declaration. The lexer and AST cover imports and visibility; functions, unsafe and extern declarations; structures, enums, statics, and constants; conditional and loop control; arrays and aggregate construction; direct calls; field and index access; casts; volatile operations;

and inline assembly. [crates/ostadix-api/src/ocore/lexer.rs] [crates/ostadix-api/src/ocore/parser.rs] [crates/ostadix-api/src/ocore/ast.rs]

Supported foreign interfaces include `extern "sysv64"`, `extern "C"`, and `extern "ocore"`. A `let` requires an initializer, assignment is right-associative, and binary precedence is encoded in the parser. Inline assembly has explicit syntax and operands. [crates/ostadix-api/src/ocore/parser.rs]

The checked-in minimal unit illustrates the source accepted by the AST, HIR, MIR, assembly, and ELF object pipeline:

```
module examples::minimal;

@export
pub fn add(left: u64, right: u64) -> u64 {
    return left + right;
}
```

[ocore/examples/minimal.oc]

The module declaration names the unit, `@export` fixes external visibility, `pub` admits cross-module lookup, the signature fixes every machine type, and the explicit return becomes typed SSA before ISA lowering. Structures, enums, imports, statics, constants, extern declarations, unsafe blocks, volatile and atomic intrinsics, and `asm!` use the same direct item-and-expression grammar across the 149 tracked O-core source files. The complete tracked language inventory also contains 86 hosted O files; the current parser check accepts all 235 without executing them. [docs/OCORE.md]

11.2 Static semantics

Type checking proceeds in dependency-aware passes:

$$\text{module scopes} \rightarrow \text{type names} \rightarrow \text{layouts} \rightarrow \text{signatures} \rightarrow \text{constants/statics} \rightarrow \text{bodies}.$$

Unsafe operations require an explicit unsafe context. Attributes are whitelisted; unknown ones are rejected. Interrupt and naked functions, aggregate calls, and floating-point ABI crossings receive explicit defensive checks. [crates/ostadix-api/src/ocore/typeck.rs]

11.2.1 Types, aggregates, and layout

The scalar set is `bool`; unsigned `u8`, `u16`, `u32`, `u64`, and `usize`; signed `i8`, `i16`, `i32`, `i64`, and `isize`; `f32`, `f64`, `void`, and `never`. Compound types are fixed arrays `[T; N]`, immutable and mutable raw pointers, declaration-ordered structures, tagged enums, and function-pointer types. Structures support construction, fields, locals, statics, and aggregate copy. Arrays support literals, repetition, indexing, locals, statics, and explicit address operations. Enum layout combines the smallest adequate `u8`/`u16`/`u32` tag with an aligned payload region.

Type class	Size (bytes)	Alignment
`bool`, `u8`, `i8`	1	1
`u16`, `i16`	2	2
`u32`, `i32`, `f32`	4	4
`u64`, `i64`, `usize`, `isize`, `f64`, pointers	8	8
`void`, `never`	0	1

Natural padding preserves declaration order. `@packed` selects alignment 1 and removes inter-field padding; `@align(N)` raises alignment to a power of two. System V scalar calls use RDI, RSI, RDX, RCX, R8, and R9, followed by stack arguments, with RAX as the scalar result and 16-byte call alignment. `@interrupt` functions return with `iretq`. [docs/OCORE.md]

11.2.2 Name, conversion, storage, and call semantics

One `ocorec` invocation forms a compilation unit from one or more source files, each with a unique module name. Unqualified lookup proceeds through local bindings, current-module items, explicit imports, and predeclared intrinsics. Cross-module functions receive deterministic mangled symbols; `@export` and `@no_mangle` state the external symbol contract.

An unsuffixed integer literal inherits an expected integer type. Every other numeric or pointer conversion is written with `as`. Raw pointer arithmetic and dereference, pointer/integer conversion, mutable-static access, inline assembly, and privileged intrinsics require an explicit unsafe context. Values use deterministic destruction-free storage. Stack objects, statics, raw pointers, and explicitly linked runtime allocators state every storage lifetime at the freestanding boundary. Heap allocation occurs through an explicitly linked runtime allocator, keeping lifetime management explicit.

Direct scalar calls lower through the selected ABI, and function-pointer types carry call signatures through parsing and type checking. Aggregate parameters and aggregate returns are rejected across direct and extern call boundaries; authors pass pointers explicitly. Floating-point values have fixed storage layouts and bit-preserving transport. [docs/OCORE.md] [crates/ostadix-api/src/ocore/typeck.rs]

11.2.3 Machine operations and linkage

The volatile/atomic surface is `volatile_load`, `volatile_store`, `atomic_load`, `atomic_store`, `atomic_exchange`, `atomic_compare_exchange`, and `atomic_fetch_add`. Orders are `relaxed`, `acquire`, `release`, `acq_rel`, and `seq_cst`; the type checker validates operation width, mutability, and legal order combinations before code generation. Hardware intrinsics cover byte/word/dword port input and output, interrupt enable/disable, halt, page invalidation, `syscall0` through `syscall6`, and Intel-syntax `asm!` with explicit register operands and memory/stack options.

The linkage attributes are `@export`, `@no_mangle`, `@link_section("name")`, `@align(N)`, `@used`, `@packed`, `@interrupt`, and `@naked`. Together they let O-core source state symbol visibility, exact symbol names, ELF placement, retention, layout, and entry/exit discipline directly. `@unsafe_linkage` explicitly admits a writable static placed in an executable section. [docs/OCORE.md]

11.3 Compiler pipeline

MIR is typed SSA, while mutable locals and statics remain explicit memory places. In the x86-64 kernel profile shown here, the code generator emits Intel-syntax assembly and gives every local and SSA value stack storage. Clang assembles with `x86_64-unknown-none-elf`, freestanding settings, and the stack protector disabled. [crates/ostadix-api/src/ocore/mir.rs] [crates/ostadix-api/src/ocore/codegen.rs] [crates/ostadix-api/src/ocore/driver.rs]

11.4 Current native compiler profile

O-CORE has owned assembly emitters for `x86_64-unknown-none`, `aarch64-unknown-none`, and `riscv64-unknown-none` (RV64GC). All three produce little-endian ELF64 objects with an LP64

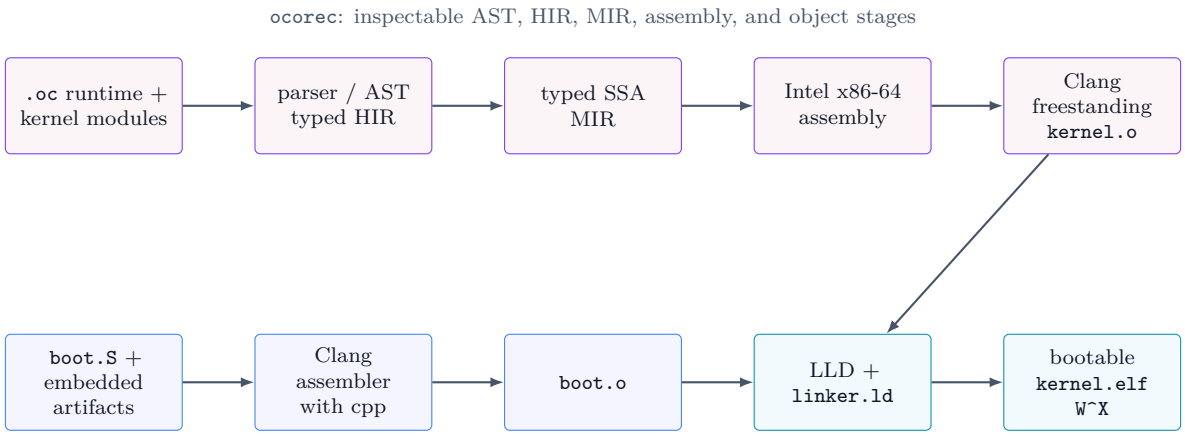


Figure 25: The x86-64 kernel build has two feeders. `ocorec` compiles O-core modules into a freestanding object; boot assembly, embedded mode artifacts, and the linker script complete the Multiboot2/Xen-PVH kernel image.

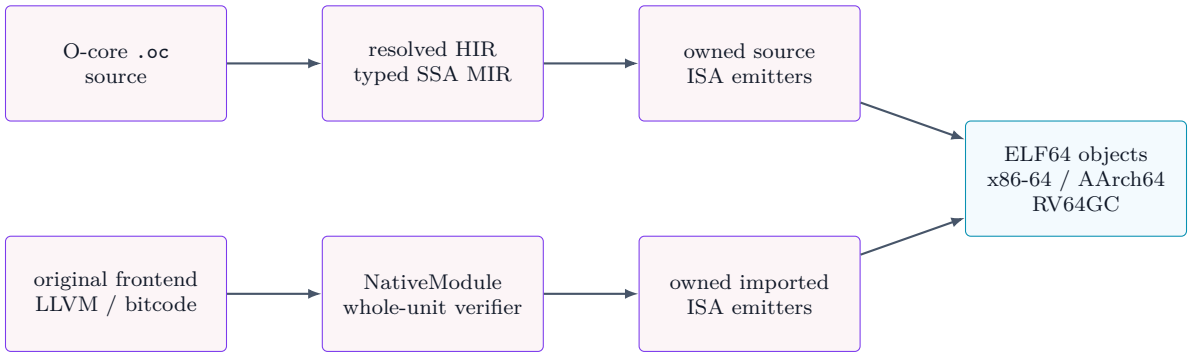


Figure 26: Two native inputs retain distinct invariants. Source HIR/MIR and imported NativeModule have separate verification and ABI boundaries; both use O-core-owned instruction emission. OIR remains the orchestration layer.

data model. The source route is `.oc` → parsed AST → resolved, typed HIR → SSA MIR → target assembly → object. Initial source type checking precedes MIR lowering. The assemblers translate O-core's emitted assembly; they do not select the imported implementation's machine instructions. [crates/ostadix-api/src/ocore/driver.rs] [crates/ostadix-api/src/ocore/codegen.rs] [crates/ostadix-api/src/ocore/codegen_aarch64.rs] [crates/ostadix-api/src/ocore/codegen_riscv64.rs]

The three `.oc` backends share typed HIR and MIR and use stack-spill code generation, without an optimizing pipeline or general register allocator. They implement scalar locals and pointers, integer operations and casts, direct and unsafe indirect calls with stack arguments, branches and parallel phi transfers, volatile memory, supported integer atomics, aligned local frames, and overlap-safe aggregate copies. Aggregate layout, construction, fields, indexing, locals and statics are implemented; `.oc` call boundaries still pass aggregates through explicit pointers. Floating-point computation and full C ABI classification remain outside this source profile. [docs/OCORE.md]

`extern "C"` selects the target's scalar integer C convention: System V AMD64, AAPCS64, or RISC-V LP64. The default, compiler-versioned `extern "ocore"` uses the corresponding scalar register and stack placement. Explicit `extern "sysv64"` remains AMD64-only. AArch64 and RISC-V now support their own inline assembly, operand-free naked functions, interrupt-mask operations, wait instructions, translation invalidation and syscall interfaces. They reject x86 port I/O and the x86 interrupt-entry ABI. A RISC-V compiler backend does not by itself establish a RISC-V kernel port. [docs/OCORE.md]

The same-source native-equivalence harness compiles the real `.oc` CLI path for all three targets, links an independent freestanding C oracle, and compares values, traps, memory operations and C crossings. Its declared corpus covers 4,096 vectors over three arithmetic functions, twelve-argument direct and indirect calls, packed and aligned memory, large local storage, volatile operations, supported atomic widths and six fault programs per ISA. The harness also compares objects emitted in separate output directories. These are scoped compiler-conformance checks; execution through QEMU user emulation does not establish physical-machine behavior. [scripts/check_ocore_native_equivalence.py] [docs/OCORE.md]

11.5 Importing original implementation code

A second native route accepts original frontend LLVM text or bitcode and normalizes it into `ocore-native-module-v1`. This representation is separate from both orchestration OIR and the `.oc` source AST/HIR. It records target layout, types, functions, globals, aliases, COMDAT groups, initializers, relocations, control-flow graphs, SSA values, execution contracts and source/tool provenance. The importer uses a digest-pinned LLVM parser and a bitcode-record audit; an independent Rust verifier checks the complete normalized module before any owned ISA emitter receives it. Verification includes unused functions and cold blocks, type and signature consistency, dominance, exact phi predecessors, layout and relocation contracts. A verification receipt binds the module bytes. It establishes structural validity, with code-generation coverage and execution checked separately. [scripts/ocore_llvm_import.py] [crates/ostadix-api/src/ocore/native_ir.rs] [crates/ostadix-api/src/ocore/native_verify.rs]

The implemented integer/pointer profile includes widths 1–64, typed structure and array values, ordinary memory, static entry allocations, switches, parallel phi moves, indirect calls, memory fill/copy/move and supported intrinsics. Fixed C and LLVM `fastcc` calls preserve the frontend's aggregate decomposition, argument attributes, register placement, stack arguments and result-storage contract. Typed `sret` retains its original pointer parameter and target placement, including AArch64's `x8`. This imported ABI support is broader than the `.oc` aggregate-call surface. It does not reconstruct a source-language ABI merely from a structure's byte size. [crates/ostadix-api/src/ocore/native_abi.rs] [crates/ostadix-api/src/ocore/native_codegen.rs] [docs/OCORE.md]

Imported execution uses the explicit `llvm_defined_behavior_v1` contract. Original integer widths and operation predicates remain meaningful; `.oc` division and shift rules are not silently substituted.

The contract records deterministic refinements of poison and undef and conservatively erased optimizer promises. The nonoptimizing emitter does not infer purity from LLVM attributes or from a runtime's name. Unsupported operations, ABI features or hidden bitcode fields reject the whole unit. Each ISA requires input with the matching original frontend triple, DataLayout and ABI; changing the requested output ISA does not retarget an x86 LLVM module to AArch64 or RISC-V. [docs/OCORE.md] [crates/ostadix-api/src/ocore/native_verify.rs]

11.6 C variadics and explicit execution profiles

Imported C variadic declarations, definitions and direct or indirect calls preserve the fixed signature prefix and the actual trailing operand types. Call-site attributes are checked independently, including attributes after the fixed prefix. The implemented profile uses integer and address-space-zero pointer arguments in little-endian ELF LP64; default argument promotions remain the original frontend's responsibility. Canonical `llvm.va_start`, `llvm.va_copy` and `llvm.va_end` preserve their operands and context. Target prologues save the incoming register banks, and the emitted cursor operations preserve register-save areas, stack overflow positions, fixed-prefix offsets and independent copied cursors. RV64 LP64 and LP64D frontend ABI metadata and object flags remain distinct. [crates/ostadix-api/src/ocore/native_variadic_codegen.rs] [crates/ostadix-api/src/ocore/native_abi.rs] [tests/test_ocore_native_variadic_abi.py]

The retained 20 October variadic qualification records seven test methods without skips across x86-64, AArch64, RV64 LP64 and RV64 LP64D. Four combinations of reference/owned caller and callee per profile compare 11,136 integer/pointer results. Eight additional opaque-`va_list` forwarding pairings compare 1,344 results, including floating-point register overflow and AArch64/RV64 binary128 transport. Saving raw incoming vector registers lets an owned integer wrapper forward an opaque list to a foreign fixed-signature consumer; it does not introduce floating-point SSA or an owned floating-point caller. Of the 12,480 total comparisons, 9,024 include at least one owned component. These are retained Linux development-carrier results, with QEMU for nonhost ISAs. General `byval`, first-class floating-point calls, generic LLVM `va_arg` and exception execution through owned variadic frames remain separate qualifications. [evidence/native_variadic_integration_20261020.json]

Other imported operations require named execution profiles. Coherent-RAM atomics preserve ordering and supported integer/pointer operations while excluding device memory; RV64 masked subword operations additionally require accessible containing words and LR/SC eventuality. The static, nonpreemptive Linux ELF TLS profile preserves original TLS model metadata and realizes current-thread addresses through local-exec accesses. The static `itanium_dwarf_v1` exception profile preserves `invoke`, normal and unwind edges, landing pads, catch/filter order, selector queries and `resume`, together with call-frame and LSDA metadata. The implementation has independent malformed-object controls and Linux execution witnesses for TLS and exception consumers. Their original musl, unwind and C++ ABI providers remain explicit reference dependencies. These witnesses do not establish O-core lowering of those providers, native OSTADIX platform services, dynamic loading or JIT publication. [crates/ostadix-api/src/ocore/native_atomic_codegen.rs] [crates/ostadix-api/src/ocore/native_exception_codegen.rs] [tests/test_ocore_native_eh_execution.py] [tests/test_ocore_native_tls_execution.py] [docs/OCORE.md]

11.7 Compiler-owned source recipes and module closure

`olangc -native-runtime` accepts a pinned original-source recipe, a capture report, or an already verified native module-set package. With `-native-runtime-source`, the compiler runs the declared original build in a new output directory, retains its generated unit inventory and frontend inputs, independently re-admits the capture, and invokes native lowering. The original source revision, build configuration, producer tools, selected units and emitted artifacts remain bound by hashes. The source is not rewritten

to remove unsupported bodies. The generated inventory must account for the complete configured output graph, including required archive members and extension roots; required recipe entries are a minimum inventory, not permission to omit additional build-generated units. [src/bin/olangc/native_runtime/source_recipe.rs] [scripts/ocore_runtime_capture.py] [tools/ocore-runtime/README.md]

The driver distinguishes capture from independently admitted capture and from successful lowering. Its stage receipt retains failure diagnostics as well as successful artifact identities. Declared dependencies are supplied as verified `ocore-native-module-set-v1` packages. The loader checks embedded module hashes and structural validity, reconstructs component and unit identities, recomputes symbol/ABI/-COMDAT closure, and requires matching target and execution profiles. Private symbols remain unit-scoped; identical transitive components are shared, and complete cyclic dependency graphs are permitted. Missing packages, unresolved source-identity constraints, conflicting components and forged closure summaries reject. Reloading a successful self-contained package needs no original source or LLVM files, while object emission still requires the owned compiler and an assembler. [src/bin/olangc/native_runtime/artifact.rs] [tools/ocore-runtime/README.md]

The resulting module-set establishes native-module dependency closure for its included profile. The current source route does not perform the final runtime link, attach an ordinary `.O` program, supply platform services or qualify a complete native runtime. Accordingly, its receipts keep `runtime_complete`, whole-runtime dependency closure and native execution qualification false. These fields prevent a successful source capture or a collection of emitted objects from being mistaken for a linked language engine. [src/bin/olangc/native_runtime/source_recipe.rs]

11.8 Measured original-source progress

The following observations are dated retained development evidence. They qualify the listed input and host profiles, and were not recreated by the whitepaper's source review.

Original source	Recorded result and remaining boundary
CPython, 18 October	The RV64 capture retained 876 LLVM units and accounted for all 360 configured implementation units. The whole-unit baseline independently verified 72 units and emitted 63 owned objects. A separate explicit coherent-RAM atomic retry verified and emitted the nine object-stage failures. The baseline and retry are distinct reports. Other units and provider implementations remain unfinished; the original hosted CPython smoke is reference evidence.
chibicc, 20 October	The variadic follow-up attempted all nine unchanged captured units. Six independently verified and emitted owned objects; three first rejected double-precision or binary128 values. This advances the earlier nine-unit variadic rejection result, without qualifying a linked or executed complete compiler or its providers.
adler2 2.0.1	The pinned original configured Rust library and its emitted CGU were freshly re-imported, verified and emitted. Its concrete `adler32_slice` API ran on Linux RV64 LP64D against original-object and independent C/Python controls for twelve cases, including a 1 MiB payload. This does not establish all future generic instantiations, Rust standard-library ownership, complete provider closure or `rustc`/Cargo compilation.

[evidence/original_python_c_rust_expansion_20261018.json] [evidence/cpython_atomic_profile_followup_20261018.json] [evidence/chibicc_variadic_followup_20261020.json] [tools/ocore-runtime/rust-crates/README.md]

11.9 Native computation within the ordinary O graph

The eager `ocore_i64` builtin connects a complete verified native module to ordinary OIR and HGraph through a literal descriptor and an explicit adapter. The admitted entry has fixed `i64` C parameters and result. Static plan inspection performs no assembly, linking or execution. Fresh graph admission audits the whole module against the separate `linux-elf-i64-acyclic-v1` profile, including unused functions and unreachable blocks. This profile permits acyclic scalar arithmetic and control flow but rejects memory, pointers, calls, globals, aliases, TLS, atomics, exception handling and unresolved providers. It has no arbitrary graph, module or instruction-count ceiling. [crates/ostadix-api/src/native_leaf.rs] [docs/OCORE.md]

On a matching Linux ISA, the owned emitter and a libc-free wrapper produce a statically linked process using an exact binary input/output protocol. Signed integer arguments enter the native computation and its result returns as an O integer. Traps and malformed output fail the operation without a hosted fallback. This is an implemented bridge from a bounded native operation to the hosted graph; it does not lower the entire graph coordinator or implement a complete runtime-bearing OIR artifact. [crates/ostadix-api/src/native_leaf.rs] [tests/test_ocore_native_hgraph_cli.py]

11.10 Complete external-runtime and compiler objective

The larger compiler objective is automatic preservation and lowering of existing runtime and compiler implementations, including interpreters, AOT compilers and JITs. The intended complete pipeline makes `olangc` own source/build recipes, frontend invocation, import, verification, ABI adaptation, dependency resolution, linking and artifact construction. A resulting binary or complete IR artifact must contain the required native implementation, `.O` orchestration, scripts, resources, adapters and declared dependency closure. Included native references must resolve within that closure. Merely embedding an original executable, staging a manually rewritten interpreter in `.oc`, or interpreting unchanged imported IR does not establish this source-lowering objective. [docs/OCORE.md]

Completeness includes functions, globals, initializers, relocations, libraries, compiler helpers, modules and platform interfaces. Compiling a compiler's own implementation and routing the programs that compiler generates through O-core are distinct required capabilities. A JIT additionally needs an explicit supported generated-code lowering and publication path. The current source recipes, reusable module-sets and scalar graph leaves implement parts of this architecture; complete runtime-bearing OIR artifacts and full original engine acceptance remain work in progress.

Cross-ISA equivalence is qualified per execution profile, comparing observable values, faults and required effects under common inputs. Pure deterministic profiles require explicit numerical and library behavior; clocks, entropy, filesystems and scheduling do not become replayable merely because a runtime was compiled. Full runtime acceptance also requires actual O-core-generated engine execution with its complete dependency closure and native OSTADIX services on each ISA, after the ambient original engine is unavailable. Linux carriers, reference provider objects and QEMU development tests retain their narrower evidential scope. [docs/OCORE.md]

12 Kernel, capabilities, and native execution

12.1 Freestanding kernel substrate

The kernel build links a complete set of `.oc` runtime modules with boot assembly and a linker script. It uses Rust/Cargo `ocorec` on the host, Clang for assembly, and LLD for ELF linking. The image contains Multiboot2 and Xen PVH metadata, `W^X` program headers, fixed user RX and RW-NX regions, guard-page checks, and fail-closed initialization of memory, process/domain, scheduler, capability, mapping, and IPC state. [ocore/kernel/build.sh] [ocore/kernel/linker.ld] [ocore/kernel/main.oc]

12.2 Keeping native images within their declared bounds

The 28 October repair addresses three distinct failure modes visible in the Linux portable aggregate: linked kernels exceeded probe headroom; linked personality files exhausted the bounded OVFS image; and rebuilt payloads no longer matched the previously admitted identities. These are failures at three different layers: linked reachability, file representation, and content identity. The repair retains the existing kernel and OVFS limits.

Reachability and kernel headroom. For the affected probe modes, the compiler places emitted functions in individually collectable ELF sections and the linker uses `-gc-sections`. Reachable code and explicitly retained user sections remain in the image. Unreachable helpers from the shared runtime no longer consume each probe's reserved area. The linker assertions continue to enforce the original memory boundary, including mode 18's `0x380000` headroom limit. This is reachability-based removal; it preserves execution semantics and the existing kernel-size constraint. [src/bin/ocorec.rs] [crates/ostadix-api/src/ocore/codegen.rs] [ocore/kernel/build.sh] [ocore/kernel/linker.ld]

[ocore/kernel/boot.S]

File padding and loaded semantics. The M6A and live M6B personality builders use the linker's `-nmagic` layout to avoid unnecessary file-alignment padding. The resulting native OVFS images again fit their unchanged 64 KiB admission bound. A synthetic regression fixture compares the padded and compact linked forms by entry point, mapped file bytes, zero-filled memory and page permissions. The repaired real payloads are then qualified by their native QEMU gates. Let $L(E)$ denote those load-visible properties of an ELF file E. The regression requires

$$L(E_{\text{compact}}) = L(E_{\text{padded}}), \qquad |E_{\text{compact}}| < |E_{\text{padded}}|.$$

A smaller file is therefore not interpreted as a reduction in the program's required memory or capability surface. The test compares complete load segments, their permissions and zero-fill as well as the executable entry point. [tests/test_ocore_payload_layout.py] [ocore/kernel/build-m6-artifacts.sh] [ocore/kernel/build-m6b-live-artifacts.sh]

Explicit artifact identities. After rebuilding and checking the payloads, the corresponding fixed SHA-256 pins and path records are updated together. Host preflight recomputes the actual image digest and checks it against the native admission record before boot. Negative controls reject a changed build digest and drift in the embedded path records. Runtime admission still compares exact bytes; the repair does not learn or trust an arbitrary digest during boot. [tests/test_ocore_ovfs_pins.py] [ocore/kernel/smoke-loader-qemu.sh]

Rebuilt image	Bytes	Admission consequence
M4 loader	57,960	Refreshed image and native path identities.

Rebuilt image	Bytes	Admission consequence
M5 live system	57,960	Refreshed immutable package image identity.
M6A personality	58,320	Compact payloads fit the 65,536-byte bound.
M6B live personality	61,576	Compact payloads fit the same bound.
Live Linux personality	60,648	Refreshed exact embedded OVFS identity.
M7 Linux/Plan 9	97,512	Refreshed identity within its 98,304-byte profile.
M7B logical read	82,400	Refreshed identity within its 98,304-byte profile.

The complete portable run subsequently passed all 26 required gates. That aggregate executes the repaired receipt, project-receipt, loader, live, personality and logical-read paths alongside the previously passing native checks. Section 14 records its exact source coordinate and preserves the distinction between emulated execution and physical hardware.

12.3 Native capability model

Native handles encode

$$(generation \ll 32) \mid slot,$$

while CSpace identity remains kernel-private. Validation checks the current CSpace, slot, occupancy, generation, object kind, and rights. Reservation and close are explicit transactions; generation exhaustion retires a slot. Memory-view and delegated-resource authority is deliberately nontransferable. [ocore/runtime/x86_64/capability.oc]

The kernel syscall contract fixes authority checks to seventeen numbered operations. A caller supplies generated handles; the dispatcher derives the current PCB and CSpace, validates object kind, generation, and rights, and then performs the operation.

No.	Operation	Contract
0	`debug_write`	Capability-gated bounded copy from readable user memory.
1	`cap_close`	Close one generation-tagged CSpace entry.
2	`cap_copy`	Prepare an attenuation-only transfer ticket to an authorized endpoint's receiver.
3	`page_alloc`	Allocate a typed page object and return a kernel-selected capability.
4	`yield`	Record or perform a cooperative scheduler transition.
5	`ticks`	Return the diagnostic timer count.
6	`exit`	Enter the configured trusted lifecycle continuation.
7	`sleep`	Block until a scheduler-managed tick deadline.
8	`endpoint_create`	Create and return a generation-safe endpoint.
9	`endpoint_send`	Send two words plus an optional transfer ticket.
10	`endpoint_receive`	Receive one bounded 32-byte message.
11	`endpoint_cancel`	Cancel by endpoint and correlation identity.

No.	Operation	Contract
12	`serial_read`	Read one nonblocking byte through the exact control capability.
13	`control_submit`	Submit one checked command of at most 192 bytes.
14	`personality_call`	Invoke the M6A scalar route with a deadline.
15	`personality_reply`	Complete an exact M6A request through a reply capability.
16	`personality_supervise`	Apply an M6A policy action to an exact generation and subject.

The message path uses fault-aware bounded copy, FIFO endpoint queues, real TCB block/wake epochs, request correlation, lifecycle cancellation, and exact attenuation. The scalar personality calls use distinct call, reply, and supervision capability kinds, so operation role is carried by authority rather than a caller-provided mode bit. [docs/OCORE.md]

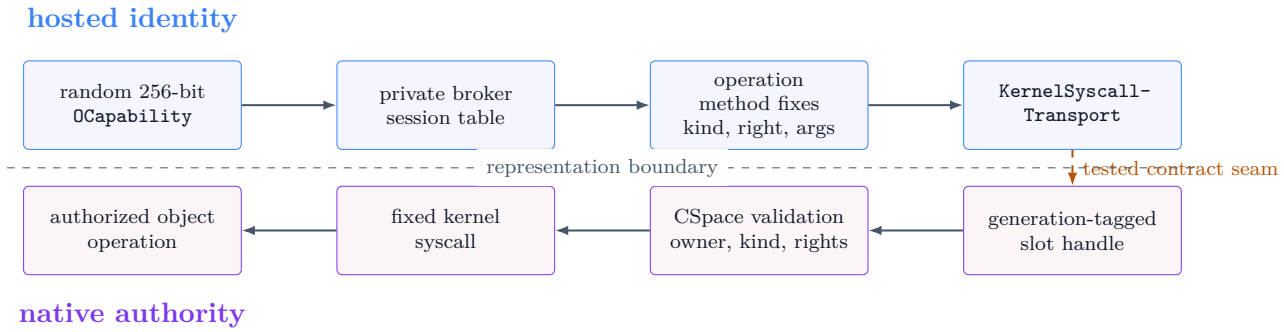


Figure 27: The bridge preserves hosted bearer identity and kernel-owned native authority. Operation-specific contracts cross the transport seam; CSpace validation resolves generation, object kind, and rights before execution. A private session table owns bearer-to-handle bindings, while recording and failure transports verify the exact calls.

[crates/ostadix-api/src/ocore/capability_bridge.rs]

12.4 Native Ethernet and a real Linux guest

The native Ethernet profile boots two independent x86-64 O-core kernels with RTL8139 devices. Their native driver transfers a boot-provisioned chain of two through four dependent additions. HMAC-SHA256 binds packet bytes, nodes, generations, session, and graph; the worker checks operands and predecessor digests against its independently provisioned delegation. Exact duplicate requests use retained results, altered duplicates are rejected, and bounded retry/drain protocols precede NIC shutdown. The verifier injects packet faults, loss, replay, and permanent partition while checking intermediate results and native quiescence. The host harness does not compute the graph operations. [ocore/kernel/native-cluster/README.md] [docs/NATIVE_DISTRIBUTED_LINUX_STATUS.md]

The AArch64 profile places an unmodified Linux 6.12.43 kernel behind an O-core EL2 monitor with separate guest stage-two RAM. Linux uses its standard virtio-MMIO block driver. During withdrawal the monitor returns a real block I/O error, waits for guest consumption and interrupt acknowledgment, then removes guest RAM mappings and contains the resulting stage-two fault. The broker presently executes at EL2; the full G7 contract's separate trusted host-EL1 broker remains implementation work. These software profiles have a dedicated required-CI job, separate from portable component gates and the append-only Alpha ledger. No fresh native run or remote CI result is asserted by this document update. General native ProjectBundle lowering, replicated commit/recovery, physical device isolation/reset, and the integrated physical multinode scenario remain outstanding; G4, G7, G8, and G10 remain unqualified. [docs/NATIVE_DISTRIBUTED_LINUX_STATUS.md] [.github/workflows/ci.yml]

13 Live-World and KernelWorld

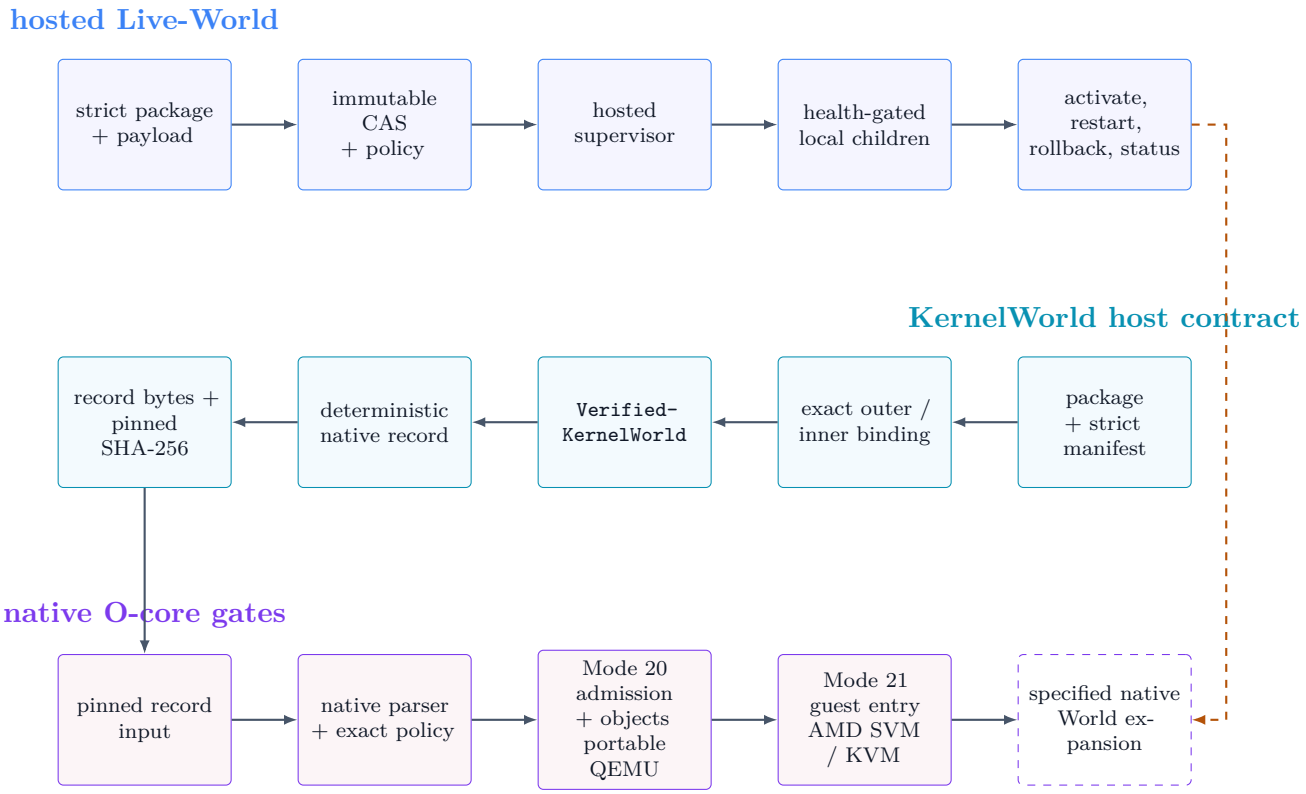


Figure 28: World semantics, host verification, and native execution occupy separate substrates. The record path is a solid authority chain into native admission. The dashed path marks differential comparison between hosted lifecycle behavior and the specified native continuation.

13.1 Three different layers

Layer	Implemented substrate	Architectural role
Hosted Live-World	Local child processes, immutable content store, strict manifests, policy, health checks, rollback/restart/reconstruction, and private bearers.	Executable lifecycle semantics and a differential oracle for native World stages.
Native M5 slice	Separately loaded x86-64 ELF services and CSpaces under QEMU, a serial control loop, and generation-based crash/restart behavior.	Machine-level service isolation, capability routing, crash recovery, and generation replacement in the M5 execution gate.
Target architecture	Independent unprivileged endpoint-RPC services, durable reconstruction, and a native build service.	The specified expansion path from the implemented native service substrate to a complete World lifecycle.

[docs/HOSTED_LIVE_REFERENCE.md] [docs/LIVE_SYSTEM.md]

Hosted advisory locks and revision compare-and-swap serialize lifecycle transactions over the local immutable content store. Supervisor code implements generation rotation, bearer issuance, exact live-binding invocation, restart, and stale-generation denial. [crates/ostadix-api/src/live_system/supervisor.rs]

13.2 Live-World command and state model

The public `o-live-host` surface is a complete local lifecycle. A package enters an immutable content-addressed store, activation creates a health-checked service generation, invocation resolves the exact active binding, composition passes OVALUE between worlds, and every mutation records enough state for reconstruction.

Command	State transition	Resulting state
`pack`	Canonical manifest and payload → exact package digest.	Emits the content identity calculated over canonical manifest semantics and the exact captured payload tree.
`install`	Verified package → immutable CAS object.	Writes the object and strict alias metadata into the content-addressed store.
`activate`	Policy admission + health → active generation.	Publishes a health-checked service generation and its exact policy grants.
`invoke`	Active bearer + ABI + operation → tagged result.	Resolves the exact live binding and returns a bounded tagged OVALUE.
`compose`	Invoke chain → OVALUE transfer between worlds.	Passes each typed result into the next declared service operation.
`upgrade`	Candidate health + generation rotation → new active digest.	Retains the prior healthy root and publishes fresh service generations.
`rollback`	One retained prior healthy root ↔ current package, with fresh service generations.	Swaps the retained and active roots after health and policy validation.
`restart`	Selected service → fresh process and bearer generation.	Rotates the service generation and invalidates the prior live binding.
`status`	Revisioned `active-set.json` + CAS + policy → reconstructed and health-checked active set.	Reconstructs exact package digests, services, generations, and policy grants from those three state sources.
`demo`	Fixed acceptance scenario → nine lifecycle markers.	Exercises immutable CAS, policy denial, health, composition, rollback, stale-bearer rejection, crash isolation, restart, and reconstruction.

13.3 Authorable Live-World schemas

The lifecycle is driven by four strict public data surfaces. These are the formats a package author and operator write directly.

Surface	Required shape
Package manifest, TOML	`schema = "ocore.package/v1"` plus required `name`, `version`, `architecture`, `payload_sha256`, `runtime`, `services`, `capability_requests`, `health`, and `build`. Runtime contains `kind`, `entry`, and `abi`; services contain `name` and `protocol`; requests contain `kind`, `rights`, and `purpose`; health contains `protocol` and `timeout_ms`; build contains `source_sha256` and `builder`.
Activation policy, TOML	`schema = "ocore.hosted-activation-policy/v1"` plus `grants`. Admission selects the exact (*package*, *kind*, *purpose*) grant and requires its rights set to contain every requested right.
Runtime program, TOML	Required `schema = "ocore.runtime-program/v1"`, `world`, `health.status`, and at least one operation, where status is `healthy` or `unhealthy`. The kinds are fieldless `identity` and `crash`, `wrap.field`, `prefix.text`, `int_pair.lhs/rhs`, `sum_fields.lhs/rhs`, and `fail.message`.
Composition plan and input, JSON	A nonempty array of `{service, protocol, operation}` steps receives one tagged OVALUE JSON input. Each result becomes the next step's input.

Package verification establishes the general `ocore.package/v1` contract. Hosted activation and reconstruction then require `architecture` to equal the host architecture, `runtime.kind` to equal `native_test_runtime`, `runtime.abi` to equal `ocore.runtime-service/v1`, and `health.protocol` to equal `ocore.health/v1`. The runtime entry is an absolute path inside the captured payload, such as `/bin/live`. Manifest health timeouts range from 1 through 60,000 ms; the public CLI's default supervisor admits at most 5,000 ms.

```
schema = "ocore.runtime-program/v1"
world = "example.source"

[health]
status = "healthy"

[operations.produce]
kind = "int_pair"
lhs = 20
rhs = 22
```

```
schema = "ocore.hosted-activation-policy/v1"

[[grants]]
package = "runtime/example"
kind = "endpoint"
purpose = "request channel"
rights = ["send", "receive"]
```

```
[
  {"service":"world.source","protocol":"ocore.runtime-service/v1",
   "operation":"produce"},
  {"service":"world.sum","protocol":"ocore.runtime-service/v1",
   "operation":"compute"}
]
```

The plan's first input can be `{"t":"null"}` or any other tagged OVALUE JSON value. The worker protocol carries the same algebra in bounded canonical-CBOR frames, preserving type and structure between processes. Unknown manifest, policy, runtime-program, and composition fields fail strict deserialization before lifecycle mutation. [crates/ostadix-api/src/live_system/manifest.rs] [src/bin/o-live-host.rs]

[crates/ostadix-api/src/live_system/protocol.rs] [crates/ostadix-api/src/live_system/supervisor.rs] [crates/ostadix-api/src/live_system/manifest.rs] [crates/ostadix-api/src/live_system/supervisor.rs]

The runtime owns three further strict record families that connect installed bytes to reconstructed services and typed process exchange.

Runtime-owned format	Exact shape
`ocore.store-object/v1` and `ocore.store-alias/v1`, TOML	An object records `schema`, `algorithm`, and `digest`. An alias adds `alias`; both select `sha256`.
`ocore.hosted-active-set/v1`, JSON	`schema`, monotonic `revision`, `active`, and `rollback`. Each package record carries `package_name`, `digest`, and `services`; each service carries `name`, `protocol`, and `generation`. One prior healthy root can be retained per active package.
`ocore.runtime-service/v1`, canonical CBOR	Requests tagged by `type`: `health{nonce}`, `invoke{service, operation, input}`, and `shutdown`. Responses are `healthy{nonce, world}`, `unhealthy{nonce, message}`, `result{value}`, `error{message}`, and `stopped`.

[crates/ostadix-api/src/live_system/store.rs] [crates/ostadix-api/src/live_system/supervisor.rs] [crates/ostadix-api/src/live_system/protocol.rs]

Exact package identity is constructed from canonical semantics and captured bytes. The manifest first becomes canonical identity bytes. Payload entries are normalized and sorted, then encoded with a length-prefixed relative path, Unix executable bit, and file contents. Separate `ocore.payload-tree/v1` and `ocore.package-object/v1` SHA-256 domains distinguish the tree hash from the complete package hash. On Unix, payload traversal is descriptor-relative and applies `O_NOFOLLOW` at every component. The scanner retains opened regular-file descriptors through capture and compares metadata around the read. The resulting digest therefore names the canonical manifest and the exact files opened for capture, and that identity becomes the immutable CAS key used by activation, rollback, and reconstruction. [crates/ostadix-api/src/live_system/manifest.rs]

Layer	Implemented fixed capacities
Hosted Live-World	64 KiB manifest; 128-byte logical name; 64-byte identifiers; 256-byte protocol; 512-byte purpose; 4,096-byte runtime entry; 32 services; 64 capability requests with 16 rights each; 8,192 payload entries including 4,096 files; 32 path components of 255 bytes; 64 MiB per file; 256 MiB per package; 4 KiB store metadata.
Hosted worker protocol	1 MiB runtime program and frame; 256 operations; 4,096-byte operation messages.
Hosted supervisor	The public CLI defaults to a 5-second health ceiling, 5-second invocation timeout, and 64 KiB structural-value limit. Configurable maxima are 60 seconds and 512 KiB; public input files have a separate 512 KiB read ceiling. Fixed bounds are a 1 MiB active-set file, 64 active packages, 256 active services, 64 rollback roots, 4,096 live bearers, 256 composition steps, 255-byte operation names, and a 64 KiB activation policy with 256 exact grants.

Layer	Implemented fixed capacities
KernelWorld host contract	64 KiB manifest; 64 exports; 64 requests; 32 requirements; 16 rights per request; 128-byte identifiers; 512-byte text fields; 256 vCPUs; 1,048,576 MiB declared memory; 65,536 outstanding requests; 1,000,000 requests per generation; 1 TiB shared memory; 256 devices; health timeout from 1 through 60,000 ms; 16 KiB native record.
Native admission	The native record admits four exports and eight requests. Runtime admission holds two worlds, four exports and eight requests per world, 16 policy rules, and 512-byte policy fields. Mode 20 seals two VM, four vCPU, and eight guest-page objects in its executed gate.

[crates/ostadix-api/src/live_system/supervisor.rs] [src/bin/o-live-host.rs]

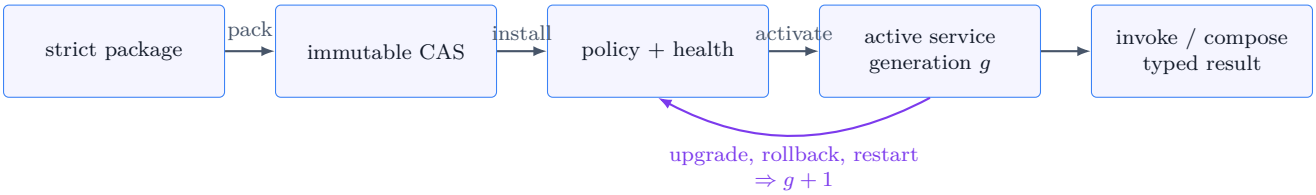


Figure 29: Admission and health create an executable Live-World generation. Upgrade, rollback, and restart return through the gate and allocate a new generation, making stale bearer identity explicit.

13.4 KernelWorld authority chain

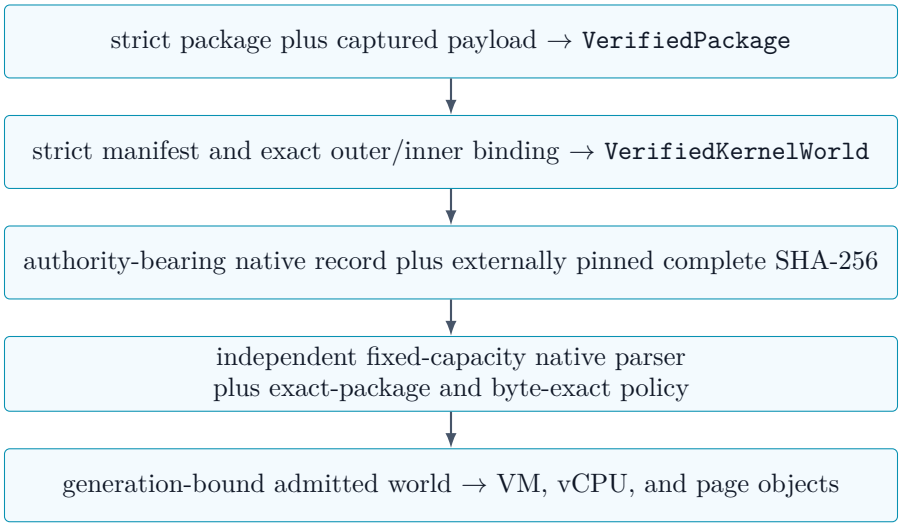


Figure 30: KernelWorld verification and native admission.

Host verification joins the outer `ocore.package/v1` object to the inner world manifest through exact equality. The outer runtime declares `kind = "kernel-world"`; its ABI is `ocore.kernel-world-control/v1`. Its absolute payload entry names a non-executable UTF-8 manifest file. Outer and inner name, version, architecture, health protocol, health timeout, service name/protocol set, and normalized capability-request set match exactly. For a `package_payload` image, the declared SHA-256 equals the digest of the exact image bytes opened from the verified payload. These equalities produce

one `VerifiedKernelWorld` bound to the immutable package digest. That verified object creates the authority-bearing native record, whose native codec checks every encoded bound exactly and rejects overflow. [crates/ostadix-api/src/kernel_world.rs]

The authorable KernelWorld manifest is strict TOML under `ocore.kernel-world/v1`. Its required top-level fields are `schema`, `name`, `version`, `integration`, `image`, `machine`, `lifecycle`, `quotas`, `exports`, `capability_requests`, and `license`.

Field group	Public vocabulary and cross-field contract
Integration and image	`source_integrated` uses a `package_payload` image, `paravirtual` execution, and `direct` firmware. `binary_contained` uses `hardware_virtualized` execution and requests `vm.machine run` authority. Image kinds are `package_payload` with `path`/`sha256`, or `user_supplied` with `expected_sha256`.
Machine	`guest_architecture`, `profile`, execution `paravirtual`|`hardware_virtualized`, firmware `direct`|`uefi`, bounded min/max vCPUs and memory, and requirements.
Lifecycle and quotas	`health_protocol`, `health_timeout_ms`, restart `never`|`on_failure`|`always`; outstanding requests, requests per generation, shared-memory bytes, and device ceilings. `max_requests_per_generation` is at least `max_outstanding_requests`.
Exports and authority	Both lists are nonempty. Each export has `name`, plane `device`|`abi`|`semantic`, and `protocol`. Each capability request has `kind`, one or more `rights`, and `purpose`. A device-plane export requires a `device.*` request and a nonzero device quota.
License	Redistribution is `redistributable`|`user_supplied_only`, paired with `external_acceptance_required`. A user-supplied image selects `user_supplied_only` and requires external acceptance.

[crates/ostadix-api/src/kernel_world.rs]

The same public host contract includes `KernelWorldInstance`, a bounded semantic oracle for lifecycle and request settlement. Its state path is `Installed` → `Starting` → `Healthy`, with explicit `Failed` and `Stopped` transitions. The manifest's `never`, `on_failure`, or `always` restart policy decides which terminal state may begin a fresh generation. Healthy export resolution returns immutable package, world, integration-mode, generation, and export provenance.

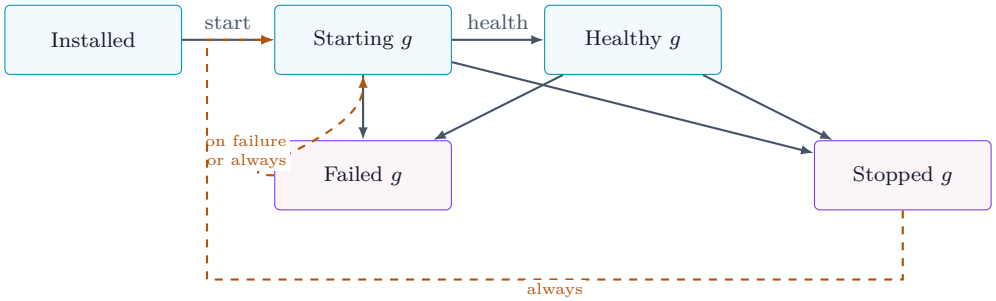


Figure 31: The host-side KernelWorld lifecycle oracle. Restart policy controls generation replacement, while health controls export publication.

Every `RequestId` is the pair (*generation*, *sequence*). Outstanding and per-generation quotas gate admission before insertion. Reply, cancellation, deadline expiry, world failure, and world stop settle requests exactly once as `Replied`, `Cancelled`, `DeadlineExceeded`, `WorldFailed`, or `WorldStopped`. Failure and stop drain all in-flight requests into the corresponding terminal record; a replacement generation resets the sequence and makes every earlier request ID stale. [crates/ostadix-api/src/kernel_world.rs]

Native admission consumes the deterministic normal form under an independently registered immutable policy. It validates exact package identity and byte content by kind and purpose, then creates a generation-bound grant after every rule and cross-field constraint succeeds. The grant remains `ADMITTED` until exact-generation revocation. [ocore/runtime/x86_64/kernel_world_admission.oc]

13.5 Native execution ladder

The native gates form one dependency-ordered execution ladder. Modes 0–20 establish the x86-64 bootstrap, isolation, process, scheduler, IPC, service, personality, and KernelWorld prerequisites consumed by the later modes. Each row names the new machine fact introduced at that point.

Mode	Mode contract
0	Long-mode bootstrap, O-core kernel entry, serial output, allocator, IDT, PIC/PIT, syscall entry, and one linked CPL3 personality.
1–8	Fatal isolation probes for divide, invalid opcode, non-present access, supervisor-page read, guard-stack access, NX instruction fetch, noncanonical address, and invalid syscall return.
9	Recoverable guarded `copy_from_user`/`copy_to_user` fault path.
10	Normal process exit, exact lifecycle teardown, stale-handle denial, sibling survival, and reclamation.
11	Contained process-fault teardown with the unrelated sibling continuing.
12	Four-TCB scheduler across two processes, preemption, yield, sleep/wake, lifecycle, hostile saved-state containment, and one million identity switches.
13	Kernel-side endpoint and capability-transfer foundation.
14	Public CPL3 IPC with four-message FIFO blocking/wake, correlation, attenuated transfer, death cleanup, and personality containment.
15	Deterministic OVFS, static ELF loading, BSS/stack construction, `W^X` mappings, and service namespace.
16	Four loaded service ELFs, serial activation, contained package-daemon fault, health-gated restart, and final reclamation.
17	Immutable package semantics, exact rollback, stale-generation denial, restart, and control-parser behavior.
18	Four-principal scalar personality RPC, unprivileged supervisor, timeout/death settlement, restart, and exact capability rebind.
19	Four request-scoped bounded-copy views, five lease classes, written-prefix commit, terminal ordering, and delegated revocation.
20	Exact KernelWorld record parsing, default-deny admission, and generation-bound VM, vCPU, and guest-page object construction.
21	AMD SVM/NPT synthetic-vCPU entry, controlled exit, injection, denied GPA, teardown, and restart through the dedicated hardware gate.

The implemented capacities make each native proof repeatable: 3,072 reclaiming frames cover the fixed 4–16 MiB pool; M2 uses four TCBs, two processes, and one CPU; M3 exposes four FIFO messages and 16 transfer tickets; M6A uses four principals with 16 tombstones and nine retained identities; M6B provides four views of 128 bytes each under a 256-byte aggregate charge and five lease classes; Mode 20 constructs two VMs, four vCPUs, and eight guest pages.

The native artifact builders domain-separate OVFS, package, manifest, record, and image identities. Each build gate recomputes those identities from the exact current inputs and passes them to an independent validator before QEMU consumes the artifact. Reproducers should retain the hashes printed by their current-master run beside the gate transcript. Each source coordinate produces its own artifact identities.

[ocore/kernel/build-m4-artifacts.sh] [ocore/kernel/build-m5-artifacts.sh] [ocore/kernel/build-m6-artifacts.sh]

13.6 Mode 20 and Mode 21

Gate	Implemented claim	Substrate	Execution significance
Mode 20	Exact record validation, default-deny admission, generation-bound nonexecuting VM/vCPU/pages, quota/overlap/stale/reclaim behavior.	Portable x86 QEMU gate.	Establishes the admission and object-construction stage before guest entry.
Mode 21	Real AMD SVM/NPT entry of a two-page synthetic real-mode guest, injected vector, known computation, `VMMCALL`, denied unmapped GPA, teardown and restart.	Writable `/dev/kvm`, AMD SVM/NPT, QEMU KVM `-cpu host`.	Defines the hardware execution gate for admitted objects and its required execution substrate.

[ocore/kernel/smoke-kernel-world-qemu.sh] [ocore/kernel/smoke-kernel-world-execution-qemu.sh] [ocore/runtime/x86_64/svm_execution.oc]

13.7 Modes 22 through 34

Modes 22 through 34 extend the portable evidence ladder through supervised KernelWorld execution, bounded foreign-personality services, canonical World records, causal fallback, hosted-to-native receipt convergence, challenged boot information, and four-CPU startup. All thirteen modes are required gates in the current evidence registry.

Mode	Implemented mechanism	Required gate
22	Health-gated typed export publication, reset intent, failure withdrawal, exact VM-graph revocation, generation-2 restart, stale-generation denial, and unrelated-service survival.	`smoke-kernel-world-live-qemu.sh`

Mode	Implemented mechanism	Required gate
23	TCG-emulated SVM/NPT entry, VMEXIT-derived health, validated virtual PIO, NPF quiescence, execution-pin release, generation rebind, and cross-World authority rejection.	`smoke-kernel-world-execution-device-qemu.sh`
24	Four digest-pinned CPL3 bounded-personality services, terminal lifecycle matrix, contained daemon fault, generation replacement, and request-wide authority cleanup.	`smoke-live-bounded-personality-qemu.sh`
25	Exact static Linux x86-64 ELF execution in CPL3 through a four-call personality, stdout/stderr results, exact `-ENOSYS`, exit status, replacement, and resource reclamation.	`smoke-live-linux-personality-qemu.sh`
26	Native Linux 9P2000 provider and Plan-9-style client over the exact `/srv/linux/status` exchange, namespace withdrawal, provider replacement, stale denial, and reclamation.	`smoke-live-linux-plan9-qemu.sh`
27	Shared Rust and O-core World identity types, byte-identical OWIDENT V1 corpus, nonzero coordinate validation, and hierarchical stale and logical comparison.	`smoke-world-identity-qemu.sh`
28	OWPROTO V1 big-endian records, exact bounds and fields, deterministic version negotiation, and Rust/O-core byte convergence.	`smoke-world-protocol-qemu.sh`
29	OWVALUE V1 portable allowlist, bounded recursive shape, canonical ordering, authority-bearing value rejection, stable hashing, and cross-language byte convergence.	`smoke-world-value-qemu.sh`
30	OWRECEIPT V1 canonical receipt and signing preimage, hosted Ed25519 sign/verify, structural signature-envelope validation, World generation and commit fields, and Rust/O-core byte convergence.	`smoke-world-receipt-qemu.sh`
31	One LogicalRead across generation-distinct providers, terminal A failure, A-route withdrawal, fresh B-local session reconstruction, immutable object hash validation, causal attempt state, cleanup, and witness survival.	`smoke-m7b-logical-read-qemu.sh`
32	Hosted project execution through a caller-supplied World view, live signed uncommitted receipt emission, native full decode and exact re-encode, signing preimage reconstruction, and signer-independent semantic digest equality.	`smoke-world-project-runtime-qemu.sh`
33	Challenged UEFI/Multiboot2 handoff into bounded kernel-owned BootInfo, memory-map-selected allocator window, retired inspection aperture, CPL3 lifecycle, and transcript challenge binding.	`smoke-x86_64-boot-info-qemu.sh`

Mode	Implemented mechanism	Required gate
34	Exact four-CPU ACPI/MADT topology, PIT progress, x2APIC INIT/SIPI, distinct AP stacks, trampoline `W^X` lifecycle, one atomic BSP/AP barrier, post-barrier timer, and one-CPU rejection control.	`smoke-x86_64-smp-qemu.sh`

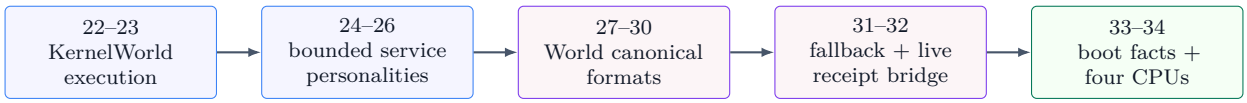


Figure 32: Modes 22 through 34 move from supervised execution through portable World records and causal recovery into challenged platform initialization.

The evidence registry declares its schema, required count, supplemental count, portable aggregate command, per-gate tool requirements, positive claims, and expected transcript markers. The validator checks registry structure, required membership, script presence, evidence classes, milestone uniqueness, marker discipline, and the release projection consumed by the portable aggregate. Mode 32 additionally wires a focused hosted project execution test into the native receipt gate so the receipt being checked originates from an executed Project HGraph path.

[evidence/gates.toml] [scripts/release_evidence.py] [ocore/kernel/smoke-world-project-runtime-qemu.sh] [tests/project_world_runtime.rs]

14 Reproducibility and dated evidence

The implementation described by this edition is the source tree `627afd3ee65c829a3edce3e3bd6ba86db4660017`, published as `f472c983` in the canonical OSTADIX repository and as `62c7f8f0` in the development repository. These commits have distinct history but identical tracked trees. A tree identity relates the published sources; it does not retroactively change the source coordinate of a test. The portable native run below records `d9af173f`; the final Linux host suite was run at `62c7f8f0`. The Mac logs record the 28 October repair validation, and are reported separately instead of being assigned an unrecorded exact commit.

A defining source, a successful build, a parser check, a runtime transcript, and an emulated native gate establish different facts. The counts below retain those distinctions. Source-defined optional tests may return without executing an external runtime when their prerequisite is absent; the Cargo passing count alone therefore does not establish execution of every language engine or external backend.

14.1 Retained checks for the 28 October revision

Check	Observed result	Scope and coordinate
Portable native aggregate	26 required gates present; 26 passed; zero failed. Per-gate validation accepted 202 required exact transcript markers.	Linux A51, QEMU/TCG with OVMF where selected; source `d9af173f`. Includes bounded x86-64 and AArch64 gates.

Check	Observed result	Scope and coordinate
Final Linux Rust workspace	2,359 passed; zero failed; five ignored.	A51 at `62c7f8f0`; the three workspace packages. The aggregate script then repeated two parser property tests and one reproducibility test, producing 2,362 passing test invocations in the complete log.
Linux example manifest	42 passed; zero failed; ten skipped.	The skips preserve interactive, opt-in guest/NixOS, manual, and missing WebAssembly-tool prerequisites; they are not runtime passes.
Mac Rust root package	921 passed; zero failed; one ignored.	Separate `o-lang` package run on the development Mac during the 28 October repair validation.
Mac Rust engine package	1,414 passed; zero failed; three ignored.	Separate `ostadix-api` run, including 1,392 library tests and 22 integration tests. Homebrew LLVM supplied the exercised compiler path.
Mac combined package result	2,335 passed; zero failed; four ignored.	Sum of the preceding two independently retained package logs, not a new whole-workspace run or proof that every optional runtime executed.
CLI suite	108 passed; zero failed.	Current debug command binaries on the Mac; the isolated arithmetic fixture includes interpreter and AOT front-door checks.
Live MCP stdio smoke	PASS.	A private installed-layout staging directory used freshly built server and command binaries. The harness's historical "release smoke" message does not establish a release-optimized build.
Tracked language-source check	235 sources passed: 86 hosted `.O` and 149 native `.oc`; zero programs executed.	The retained JSON identifies every source digest and both parser binaries. This is complete tracked parser coverage, not runtime coverage.
HTML exports	101 declared exports; whole-byte check reported zero drift.	Generated with the pinned `marked` 18.0.13 renderer. Document parity is independent of runtime and native qualification.

The portable registry also declares one supplemental hardware-KVM gate, `kernel-world-mode21-svm-kvm`, outside the 26-gate aggregate. No result for that supplemental gate is implied by the portable pass. The exact four-vCPU x86-64 SMP gate is an emulated startup/barrier experiment; it is not a general

SMP scheduler or physical-machine qualification. The AArch64 G2 gate retains its bounded MMU-off execution profile. The separate World-alpha registry has fourteen gates with its own evidence classes and dependencies; the portable aggregate does not discharge that larger contract. [evidence/gates.toml] [evidence/world_alpha_gates.toml]

The final Linux example skips comprised one interactive program, two opt-in group-pipeline programs, three opt-in Guix/WASM or guest programs, two NixOS programs, one manual Plan 9 browser example, and one program requiring unavailable WebAssembly tools. The five ignored Linux Rust checks comprise two Nix/network tests, one NixOS test, one Linux-namespace root-filesystem test, and one retained-artifact packaging hook. The Mac count omits the Linux-only root-filesystem test. Prerequisite-sensitive tests remain separately scoped. No installer-wide rerun, physical-machine boot, arbitrary-topology SMP qualification, or universal external-runtime lowering result is inferred from these records.

Portable host-test construction. The test phase defaults to debug-information level zero and disables incremental compilation for its build subprocesses, while respecting explicit profile overrides. This reduces the isolated workspace's disk requirement; it neither removes test targets nor imposes a worker-count limit. The retained A51 run selected two Cargo build jobs and used the current debug evaluator. The complete workspace tests, parser/reproducibility reruns and example manifest still executed. Setup's read-only check and dry-run modes also suppress Python bytecode cache creation. Platform tests resolve canonical filesystem paths and qualify the actual compiler features they require. [boot-and-test.sh] [setup.sh] [src/bin/olangc/native_runtime/source_recipe.rs]

14.2 Evidence classes and retained records

1. **Defining source** establishes the implemented mechanism at the published tree and the cited source coordinate.
2. **Generated structure** instantiates that mechanism for a named program and compiler. Section 9.2 retains the August project graph as a dated illustration.
3. **Registry validation** checks declared gate membership and the required release projection without substituting for gate execution.
4. **Execution transcripts** establish behavior on the actual binaries, artifacts, toolchain, prerequisites, and execution substrate.

The accompanying `evidence/whitepaper_revision_20261028.json` records the source census, retained-log byte lengths and SHA-256 identities used by this edition. It is a summary of retained observations, not a fresh gate run. The raw 28 October records are in `target/validation-logs/`. This directory is local build evidence, not a tracked source publication. Its important records are `a51-final-smoke-20261028.log`, `a51-host-tests-20261028.log`, `mac-cargo-root-20261028.log`, `mac-cargo-api-20261028.log`, `mac-cli-20261028.log`, `mac-mcp-live-20261028.log`, and `language-source-check.json`. Repository source links identify committed implementation files; the companion inventory identifies these local logs by their byte lengths and SHA-256 digests. Reproducing an observation requires the recorded execution substrate as well as the named source.

The September 17 G0 continuity attestation and transcript retain their dated repository-conformance observation; they are not relabelled as October execution. Native and distributed Linux status retains separate qualifications for embedded guests, hypervisors, hardware, and distributed execution profiles. [evidence/world/g0-attribution-history-continuity-2026-09-17.toml] [evidence/world/transcripts/g0-attribution-history-continuity-2026-09-17.log] [docs/NATIVE_DISTRIBUTED_LINUX_STATUS.md]

14.3 Source inventory and coverage accounting

The workspace has three members: the root `o-lang` package, `ostadix-api`, and `ostadix-core`. The first two are the default members and share version 0.4.0; the third declares the package `ostadix` at version 0.1.0. The root manifest declares fifteen binaries, including the feature-gated notebook command; the third package also provides its own `ostadix` binary. The MCP server remains a separately locked package outside the root workspace.

The engine module root declares 51 top-level modules: 43 public and eight nonpublic. The root library reexports 42 engine modules and declares two local CLI helper modules. These are module declarations, not counts of runtime implementations. The shared catalog contains 23 commands and eleven guide topics; the MCP server declares 21 tools. Catalog membership does not prove installation, health, admission, or execution of a capability. [Cargo.toml] [ostadix-core/Cargo.toml] [crates/ostadix-api/src/lib.rs] [src/lib.rs] [src/command_catalog.rs] [mcp/ostadix_lang_mcp_server/src/main.rs]

```
# Capture the actual source and tracked tree before testing:
git rev-parse HEAD HEAD^{tree}
python3 scripts/check_language_sources.py validate
python3 scripts/check_language_sources.py check \
  --o-bin target/debug/O --ocorec target/debug/ocorec
python3 scripts/release_evidence.py validate
python3 scripts/render_docs.py check
# Execute separately on prepared host/native substrates:
./boot-and-test.sh hosted
./boot-and-test.sh tests
./boot-and-test.sh smoke
```

The reproduction appendix gives build and execution procedures, not promised results. Record the actual revision, binary identities, complete output, skipped prerequisites, and substrate for each new run. Same-source trees, identical artifact bytes, semantic agreement across editions, and cross-ISA execution are separate reproducibility properties and require their own comparisons.

15 Implemented capabilities

- A structural polyglot syntax with exact language-tagged nesting, persistent environments, bindings, calls, quotation, and scopes.
- A rich typed interchange algebra, separate explicit crossing-fidelity assessments, and a canonical CBOR wire protocol capped at 128 MiB per frame.
- Uniform hosted lowering through OIR, a validated plan, and a state-complete HGraph.
- Deterministic graph settlement with explicit resource/effect modeling, trusted worker profiles, and explicit autonomous groups.
- Computation identity, retained realization decisions, typed grounding, physical operation/transfer DAGs, and a bounded authenticated pure-renderer Fabric.
- Live Python object ownership, checked plain-data crossings, acknowledged checkpoint migration, and owned-process cancellation.
- Deferred requests, Nix/OS values, and four first-class group topologies.
- Explicit route-preserving project lifting, reversible project bundles, logical Project HGraphs, executable route policies, and capacity-first Project Mesh placement.

- Hosted packaging to native host executables and WASI modules, plus separate Python and C17 reference/subset editions.
- Owned x86-64, AArch64 and RV64GC O-core compilers; whole-module LLVM import, explicit ABI/execution profiles, and compiler-owned source capture with reusable native module sets.
- Native x86-64/AArch64 kernel substrates, including challenged x86-64 BootInfo and bounded four-CPU startup; the compiler's third ISA does not imply a third kernel port.
- Hosted Live-World supervision and a strict KernelWorld record/admission contract.
- An independently packageable `ostadix-api` engine, an exact root compatibility shell, a unified lowercase `o` command, and a source-first local MCP with managed jobs.
- Graph V2 identity, Evidence V6 analysis, Admission V6 authority, Why V2 explanation, and one-use agent intent handles.
- Reciprocal SPAKE2 pairing, destination-issued credentials, pinned TLS, Registry V1 profiles, and generation-fenced mesh execution.
- Content-addressed Information, eight bounded native projections, typed provenance recovery, and canonical World identity, protocol, value, and receipt formats.

16 Design rationale

16.1 Why evaluator identity is syntax

I made evaluator identity part of syntax because the composition point is a semantic boundary. Structural recognition preserves native backend source while giving the host a complete heterogeneous program tree. The notation therefore carries both the foreign program and the information required to compose it.

16.2 Why graph state is explicit

Execution semantics include values, failures, ordering, actor mutation, and world effects. I represent those dimensions with ordinary values, completion tokens, and versioned resource nodes. The strict execution profile preserves the represented ordering constraints; explicitly autonomous groups admit unordered effects under their declared policy. The graph makes the chosen parallel frontier and its settlement obligations inspectable.

16.3 Why analysis and authority are separate objects

I separate Graph V2 identity, Evidence V6 analysis, and Admission V6 authority because each stage answers a different technical question. Graph V2 states which solved computation exists. Evidence V6 states why each operation is executable in one exact context. Admission V6 proves that the context and analysis still reproduce at the execution boundary. Why V2 exposes the resulting dependency neighborhood in a stable reader-facing form. This separation gives compilers, local commands, agents, and placement systems one shared execution constitution.

16.4 Why the engine owns the types

I made `ostadix-api` the implementation owner so embedding is a primary runtime surface. Parser nodes, OVALUE, plans, graphs, evidence, admission, projects, and World records can be used directly

without importing a command package. The root crate reexports those exact identities, preserving existing programs while keeping one evaluator and one value universe.

16.5 Why admission is typed across layers

I use one typed admission architecture across hosted and native execution. Hosted Live-World executes the lifecycle semantics and supplies a differential oracle for native World stages. Native M5 realizes an isolated service lifecycle on its own substrate. The connected authority path begins when Kernel-World binds verified package bytes, manifest claims, and policy into a deterministic record. The native parser verifies that record, Mode 20 constructs generation-bound admission objects from it, and Mode 21 consumes an admitted object at guest entry. This path carries authority through explicit verification products while the hosted and native lifecycle implementations share the same semantic vocabulary.

16.6 Why placement begins with capacity

Project Mesh begins with usable execution capacity because placement exists to increase the amount of work the language can execute. It ranks paired node profiles by capacity, then applies compatibility, generation, route, and policy constraints to the source-closed branch. Admission records, reciprocal TLS identity, and causal attempt traces make that capacity dependable at the moment it is used.

17 Conclusion

I designed Ostadix-lang around one proposition: evaluator identity belongs in the structural syntax of a typed expression. That decision gives the parser a heterogeneous program tree, gives OValue a typed boundary between independent runtimes, and gives OIR/HGraph the information required to represent dataflow, completion, actor state, and world effects explicitly. The nesting is the interface because the nesting survives every stage from source recognition to runtime settlement.

The project layer extends the same idea from expressions to complete source trees. Deterministic bundles preserve artifacts, route policies select whole execution alternatives, and the toolchain exposes their complete logical Project HGraph. Project Mesh closes a selected branch over its prerequisites, ranks paired nodes by capacity, and binds every attempt to node generations and returned artifacts.

Computation facets and declared realizations make the relation between meaning, implementation, representation, and target explicit. Grounding checks the identities connecting those descriptions; admitted physical execution moves the actual values under declared transfer observations. Supported native objects retain their live owners, checkpoint protocols move supported actor state, and cancellation follows the ownership of already-started work.

O-core carries the architecture to the machine boundary. Typed source lowers through HIR and SSA MIR into owned x86-64, AArch64 and RV64GC objects. Whole-module import brings original frontend output into a separate verified native representation, while source recipes and module sets make the implementation inventory and dependency closure explicit. Existing kernel profiles compose native code into generation-tagged capabilities, isolated services, strict KernelWorld records, and staged VM admission and entry. Hosted Live-World supplies executable lifecycle semantics for immutable packages, service generations, revocation, rollback, and reconstruction. The October repairs restore the portable kernel aggregate without enlarging its memory limits or treating stale artifact hashes as valid. The remaining compiler objective is complete original-runtime and compiler lowering, linking, platform closure and qualified execution across all three ISAs.

The result is one language system spanning recursive polyglot programming, state-complete graph execution, whole-project composition, native compilation, capability-bearing kernel services, Information, Registry, and World construction. The independent engine, unified `o`, and shared human/MCP catalog make the same runtime available to embedders, operators, and agents. Graph V2, Evidence V6, Admission V6, and Why V2 carry one continuous representation of evaluator choice, typed value, effect, explanation, authority, and execution substrate across all of those scales.

A Compact hosted-language reference

Form	Meaning
`LANG^(body)_LANG`	Fresh ephemeral evaluator expression.
`LANG[n]^(body)_LANG[n]`	Persistent evaluator actor keyed by language, numeric ID, and process policy.
`LANG{lazy}^(...)_LANG{lazy}`	Cache-safe deferred inline evaluation; auto-forces on splice.
`LANG{defer}^(...)_LANG{defer}`	Non-cacheable deferred evaluation; requires explicit `now`.
`let x = expr`	Root binding store; right-hand side is a typed block or orchestration call.
`$x`	Load and splice the visible binding.
`quote^(...)_quote`	Produce an unevaluated `OExpr`.
`O.eval(expr[, scope])`	Shim callback that reparses and evaluates quoted O source.
`scope()`, `O.scope()`	Capture the visible O lexical environment as an `OScope`.
`lazy(x)`, `autonomous(x)`, `now(x)`	Demand-policy and forcing operations.
`batch/all/any/race(...)`	First-class group construction with distinct settlement rules.
`instantiate`, `realise`, `activate`	Move through Nix expression, derivation, store path, and system rungs.
`dry_activate`, `current_system`	Construct an activation result without mutation; inspect the current system value.
`cap=`, authority flags, `effects=`, `reads=`, `writes=`, `serial=host`	Attach capability, effect, resource, and serialization metadata to a block.

A.1 Exact builtin signatures

Signature	Value transition
`instantiate(expr)`	`ONixExpr` to `ODerivation`.
`realise(drv)`	`ODerivation` to `OStorePath`.
`now(request_or_group)`	Force one request chain or settle one coordination group.
`dry_activate(path[, profile])`	Construct the dry activation result for one store path and optional profile.
`activate(path[, profile])`	Activate with ambient system authority.

Signature	Value transition
`activate( system_activation_capability, path[, profile])`	Validate the typed activation capability and profile guard, then activate.
`current_system()`	Return the current referential `OSystem`.
`scope()`	Capture visible O bindings as a detached `OScope`.
`lazy(expr)`	Evaluate one expression under Lazy policy.
`autonomous(expr)`	Evaluate one expression under Autonomous policy and flush admitted work.
`batch(a,...)`, `all(a,...)`, `any(a,...)`, `race(a,...)`	Capture an ordered member list with the selected settlement topology.

B Public command reference

Command	Principal forms	Result
`O`	`O [-executor serial|graph] [-workers N] [-backend-grant SPEC] [-json] [-check] FILE [shim-dir]`; `O -eval SOURCE` or `O -e SOURCE`; `-require-source-sha256 HEX`; `-require-execution-intent-sha256 HEX`; `O -repl [shim-dir]`; no arguments in an interactive terminal; `-i` aliases `-repl`; `O -version`; `O version [-json]`; `-h`/`-help`; REPL commands `:scope`, `:r`, `:?`, `:q`	Evaluate one document or enter and control the persistent interactive session.
`o` native dispatcher	`run`, `plan`, `explain`, `inspect`, `why`, `kernel`, `node`, `node-host`, `registry`, `info`, `live`, and `receipt`; `capabilities`, `guide`, `tool`, `operation`, `realizations`, `observe`, `replan`, and device/capacity families; ordinary source enters the evaluator	Route the compiled lowercase command surface into the native intent owner, compiler inspection, kernel, node, Registry, Information, Live-World, O-Git, or ordinary evaluator.
`o-cli`	`o run INPUT [execution options]`; `o plan INPUT [-route ID] [-live|-grounding]`; `o explain [last-run|RUN] [-json]`; `o inspect [last-run|RUN] [-trace] [-json]`	Resolve one hosted or project intent, plan or execute it, and retain a durable run record for explanation and inspection.
`olangc`	`-target binary|wasm|script|ir|dot`; `-o`/`-output`; `-shim-dir`; `-keep-build-dir`; repeatable `-backend-grant`; `-route`; `-routes-policy`; `-project-trace-out`; `-grounding`; `-explain-schedule`; `-format`; `-execution-intent-json`; `-why`; `-workers`; `-world-id`; `-world-epoch`	Package, execute, or inspect the hosted graph. Script target executes immediately inside the compiler process.

Command	Principal forms	Result
`ocorec`	One or more input modules; `-emit ast|hir|mir|asm|obj`; `-target x86_64-unknown-none[-elf]` or `aarch64-unknown-none[-elf]` or `riscv64-unknown-none[-elf]`; `-input-kind`; `-verify-native`; `-native-map`; `-o`/`-output`; `-keep-asm`; `-V`/`-version`	Inspect source HIR/MIR or import, verify and emit a supported native module; see Section 11.5.
`o-link`	Bare single-directory literal run; explicit `-literal` link; explicit `-project` project lift; `-o`/`-output`; `-stdout`; repeatable `-lang`; `-verbose-skips`; `-no-validate`; `-run`; `-shim-dir`; `-backend-grant`; `-shebang`; `-list-routes`; `-route`; `-routes-policy`; repeatable `-route-decl`; `-project-trace-out`; `-parallel[=verified]`; `-parallel-required`; `-explain-parallel`; `-mesh[=prefer|required]`; `-mesh-retries`; `-mesh-local-fallback`; `-mesh-discovery-timeout-ms`; `-mesh-no-lan-discovery`; `-mesh-peer-root`; `-mesh-trace-out`; `-explain-mesh`	Create reversible project or literal artifacts and invoke selected routes.
Generated project binary	No argument for its selected default; `-list-routes`; `-route ID`; `-routes-policy POLICY`	Restore the embedded project and execute its route coordinator.
`o-unlink`	`o-unlink input.O (-o|-output-dir) dir`; `-dry-run`	Restore either embedded representation with format-specific validation.
`o-notebook`	`o-notebook [shim-dir]`	Serve the embedded loopback notebook and its shared evaluator session.
`ogit`	`demo semantic-receipt`; `diff-semantic A B`	Generate a semantic receipt or compare group policy and intent.
`o-live-host`	`pack`, `install`, `activate`, `invoke`, `compose`, `upgrade`, `rollback`, `restart`, `status`, `demo`	Drive the complete hosted lifecycle state machine.
`o-node`	`start`, `stop`, `status`, `restart`, `pair`, `pki`, `identity`, `admin`, `profile`, `doctor`, `serve`	Run the node service, establish reciprocal identity pins, and administer its bounded hosted execution state.
`octl`	`node list`, `node use`, `node profile`, `node doctor`, `node run`, `node session`, `node authority`	Discover and select paired nodes, invoke exact source, operate durable sessions, and issue development-authority leases.

Command	Principal forms	Result
o-registry	init, profile-local, publish-profile, verify, list, export, import	Operate the local signed Registry V1 event chain.
o-info	init, keygen, record, verify, import, head	Create, sign, verify, import, and inspect bounded content-addressed Information state.
ocore-kernel-world-record	-manifest; -payload; -o/-output; -V/-version	Verify one package/payload pair, encode its deterministic native admission record, and print its bytes, path, record hash, and package digest.
Python 0.4.0	python3 -m o_lang FILE; -as auto|text|html|markdown|latex|json; -o/-output; -dump-ast; -V/-version; typed root API above	Execute the source-tree reference edition or consume typed Python API values.
C17 edition	c_cpp/O INPUT.O [shim-dir]; c_cpp/olangc INPUT.O [-o OUT] [-shim-dir DIR] [-keep-build-dir]	Run or AOT-compile the standalone C17 hosted edition.
Setup and aggregate gates	setup.sh with -m/-minimal, -f/-full, -y/-yes, -v/-verify, -no-wrappers, -dry-run, -h/-help; boot-and-test.sh phases setup, hosted, ocore, kernel, smoke, tests, quick, full	Build, install, verify, or execute the repository-wide layer gates.
fuzz/parser	cargo fuzz run parser	Feed arbitrary bytes through the hosted parser under libFuzzer.

[src/main.rs] [src/bin/o-cli/native_dispatch.rs] [src/bin/olangc.rs] [crates/ostadix-api/src/ocore/driver.rs] [src/bin/olink.rs] [o_lang/cli.py]

Fourteen root binaries expose -h/-help at their public command boundary. The notebook serves its positional loopback interface at 127.0.0.1:8888. Use the shared catalog to discover each command's supported help invocation.

Every stateful o-live-host command accepts global optional -policy FILE. Its exact subcommand arguments are:

```
pack --manifest MANIFEST --payload PAYLOAD
install --state STATE --manifest MANIFEST --payload PAYLOAD [--alias ALIAS]
activate --state STATE DIGEST
upgrade --state STATE DIGEST
invoke --state STATE SERVICE PROTOCOL OPERATION INPUT
compose --state STATE --plan PLAN --input INPUT
rollback --state STATE PACKAGE
restart --state STATE SERVICE
status --state STATE
demo --state STATE
```

C Example atlas

The repository contains 47 tracked top-level `examples/*.O` files. This atlas groups representative programs into nine teaching families. Companion fixtures in `examples/group_pipeline/`, `tests/fixtures/project_hgraph/`, and `tests/fixtures/project_hgraph_exec/` form the tenth through semantic receipts, logical project graphing, and bounded project execution.

Family	Representative files	Language fact
Core structure	`hello`, `bindings`, `nested_splice`, `trailing_expr`	Typed blocks, O bindings, structural nesting, and backend result extraction.
HTML and typed rendering	`html_basic`, `html_python_html`, `html_escape`, `html_raw_roundtrip`, `python_html_python`	Receiving evaluator projection, trusted HTML, escaping, and three-language nesting.
State and documents	`computed_plot`, `literate_report`, `persist`, `env_split`, `ephemeral`, `script`	Images, literate output, actor identities, fresh environments, and shebangs.
Meta-evaluation	`meta_eval`	Quotation, O expressions, callbacks, and explicit lexical scope.
Shell and JavaScript	`bash_*`, `shell_hello`, `js_*`	Host-language execution, binding injection, JSON, and multiline programs.
SQL	`sql_create_insert_select`, `sql_select`, `sql_aggregation`, `sql_python_sql`	Persistent SQLite actors and SQL/Python value circulation.
Nix lattice	`nix_basic`, `nix_python_html`, `nix_storepath*`, `instantiate_realise_basic`	Nix expressions, derivations, store paths, and cross-language projection.
Requests and groups	`lazy_defer_attrs_basic`, `lazy_request_basic`, `coordination_groups`	Demand policy, forcing, cache safety, and four settlement topologies.
Operating systems	`os_as_participant_basic`, `nixos_test*`	OSystem values, activation boundary, and NixOS test records.
Composition fixtures	`examples/group_pipeline/`, `tests/fixtures/project_hgraph/`, `tests/fixtures/project_hgraph_exec/`	Semantic receipts, the exact published Project HGraph, prerequisite and alternative branches, and bounded project execution settlements.

D Source map

Concern	Primary source	Why it is authoritative
Hosted grammar	`crates/ostadix-api/src/parser.rs`	AST, exact tag parser, contexts, aliases, escapes, and parser tests.
Hosted execution	`crates/ostadix-api/src/eval.rs`	Policies, requests, groups, lowering entry, dispatch, scopes, authority, and rendering.
Cross-language values	`crates/ostadix-api/src/value.rs`, `crates/ostadix-api/src/wire.rs`	Tagged algebra, fidelity, identities, requests, runtime boundaries, and CBOR.
OIR and plan	`crates/ostadix-api/src/ir.rs`	Lowered forms, dependencies, backend interfaces, registry, and renderers.
HGraph	`crates/ostadix-api/src/hgraph/`	Hypergraph ontology, plan projection, solving, validation, and schedule.
Effects and coordination	`crates/ostadix-api/src/effects.rs`, `crates/ostadix-api/src/executor/`	Resources, conservatism, worker eligibility, atomic settlement.
Backend process model	`crates/ostadix-api/src/process.rs`, `crates/ostadix-api/src/backend.rs`	Self-exec actors, sandbox policy, adapter invocation, and decoding.
Projects	`crates/ostadix-api/src/project/`, `src/bin/olink.rs`	Bundle model, materialization, routes, policies, lifting, and restoration.
Hosted compiler/tooling	`src/bin/olangc.rs`, `Cargo.toml`	Declared binaries and exact behavior of every target.
O-core	`crates/ostadix-api/src/ocore/`, `docs/OCORE.md`	Lexer through object generation and explicit compiler boundaries.
Native kernel	`ocore/kernel/`, `ocore/runtime/x86_64/`	Boot/link/runtime code and substrate-specific execution gates.
Hosted Live-World	`crates/ostadix-api/src/live_system/`, `docs/HOSTED_LIVE_REFERENCE.md`	Executable supervisor and explicit hosted-oracle boundary.
KernelWorld	`crates/ostadix-api/src/kernel_world.rs`, `ocore/runtime/x86_64/kernel_world_admission.oc`	Strict host record, independent native parser/policy, and generation-bound admission.
Runtime ownership and intent	`crates/ostadix-api/src/lib.rs`, `src/lib.rs`, `src/bin/o-cli.rs`	Independent engine ownership, exact compatibility reexports, unified intent, and durable run records.

Concern	Primary source	Why it is authoritative
Execution constitution	`crates/ostadix-api/src/evidence/`	Graph V2, Evidence V6, Admission V6, Why V2, and current coordinator authority.
Nodes and Project Mesh	`crates/ostadix-api/src/hosted_remote/`	Pairing, pinned TLS, node profiles, capacity ranking, placement, retry, and attempt traces.
Information and provenance	`crates/ostadix-api/src/information/`, `crates/ostadix-api/src/information_bridge/`, `crates/ostadix-api/src/information_provenance/`	Content-addressed state, eight native projections, and typed recovery admission.
Registry and World	`crates/ostadix-api/src/registry/`, `crates/ostadix-api/src/world/`	Signed namespace/profile chains and the canonical World wire families.
Computation and physical execution	`crates/ostadix-api/src/computation/`, `crates/ostadix-api/src/computation_core.rs`	Canonical facets, realization records, retained decisions, operation/transfer planning, and admitted local physical execution.
Grounding and cancellation	`crates/ostadix-api/src/project/grounding.rs`, `crates/ostadix-api/src/cancellation.rs`	Validated inspection identities and parent-aware request cancellation.
Agent and native evidence	`mcp/ostadix_lang_mcp_server/`, `evidence/gates.toml`	Source-first agent execution, managed jobs, and the required portable native gate registry.

E Complete `ostadix-api` module inventory

The independent engine declares 48 modules. Forty-two are public and are reexported by the root compatibility library. Six are crate-internal implementation modules: `backend_catalog`, `canonical_cbor`, `capability`, `dispatch_model`, `eval_core`, and `placement_protocol`. The table accounts for every declaration in the engine root exactly once.

Concern	Declared modules
Language and data model	`api`, `parser`, `syntax_dialect`, `value`, `version`, `wire`, `canonical_cbor` (internal).

Concern	Declared modules
Hosted execution and backends	`backend`, `backend_catalog` (internal), `backend_morphism`, `backend_state`, `capability` (internal), `dispatch_model` (internal), `cancellation`, `effects`, `environment`, `eval`, `eval_core` (internal), `execution_contract`, `execution_fabric`, `execution_fabric_authority`, `executor`, `hgraph`, `ir`, `process`, `runtime_exec`, `scheduler`, `shims`.
Evidence, intent, placement, and projects	`computation`, `computation_core`, `evidence`, `intent`, `placement`, `placement_protocol` (internal), `project`.
Distributed state and information	`hosted_remote`, `information`, `information_bridge`, `information_provenance`, `registry`, `resource_identity`, `world`.
Lifecycle and native systems	`boot_objects`, `kernel_world`, `live_system`, `nix_ops`, `nixos_ops`, `ocore`.

[crates/ostadix-api/src/lib.rs] [src/lib.rs]

F Architectural position among neighboring systems

The complete contribution joins expression-granular evaluator selection, recursive evaluator nesting, independently implemented runtimes, typed OVALUE exchange, OIR/plan/HGraph lowering, V6 evidence-based admission, capacity-first project placement, content-addressed Information, canonical World records, and an O-core path from typed native source into capability-bearing World execution.

Neighboring family	Its organizing unit	Ostadix-lang organizing unit
Racket `#lang` and language-oriented programming	One declared module language in a common host substrate.	A recursively nested evaluator-indexed expression dispatched to an independently implemented runtime.
Polyglot notebooks and literate tools	A top-level cell or document block.	An AST expression at arbitrary nesting depth, including Python inside HTML inside Python.
Staged metaprogramming	Code represented as data inside a staged language system.	`OExpr`, `quote^`, lexical `OScope`, and `O.eval` re-enter heterogeneous evaluator structure.
String-embedded DSLs	A string whose integration is defined by its host language.	A parsed typed-expression boundary whose result is an OVALUE consumed by the receiving evaluator.
Workflow engines	Tasks and edges in an external workflow model.	Requests, groups, policies, values, and effects inside the language's value and graph model.
Eco, PyHyp, and GraalVM	Nested editor boxes, an engineered language pair, or a shared polyglot runtime substrate.	Registry-extensible expression tags, independent runtimes, typed exchange, and whole-program state-complete graph lowering.

Neighboring family	Its organizing unit	Ostadix-lang organizing unit
Freestanding systems languages	Native compilation and kernel construction.	O-core's typed HIR/SSA MIR pipeline inside a hosted-to-native World authority chain.

Primary reference points. The neighboring organizing models are documented in Felleisen et al., *A Programmable Programming Language*; the Jupyter messaging specification; the Org Babel language manual; Diekmann and Tratt, *Eco: A Language Composition Editor*; Barrett et al., *Fine-grained Language Composition: A Case Study*; and the GraalVM Polyglot embedding reference. These sources define the comparison units used in the table: module language, notebook or document block, language box, engineered language pair, and shared polyglot runtime.

The implementation makes every junction explicit. The parser preserves evaluator identity; OVALUE preserves cross-language meaning; the plan freezes backend interfaces; HGRAPH exposes value and resource state; project policies settle whole-tree alternatives; and exact records carry verified host intent into native admission.

G Reproduction commands

The commands below begin at a prepared checkout of the pinned runtime source. The hosted greeting requires Python, a supported Common Lisp engine and Node.js. The document-build helper is distributed with this paper revision. They are reproduction procedures; successful execution must be recorded separately. Use an isolated checkout when local development changes the workspace or lockfile.

G.1 Build the whitepaper artifact

The distributed `scripts/build_whitepaper.py` invokes Tectonic with the source directory as its working directory, normalizes the result with qpdf, and retains the title, subject, keywords and Lee Daghlar Ostadi author field. It removes tool identity, timestamps, XMP and imported source-path metadata. Build products stay under `target/whitepaper/`; installation copies the checked PDF to both published artifact locations. Tectonic and qpdf must be available on PATH. Its first build may fetch TeX bundle resources; use `-only-cached` after populating the cache for an offline build.

```
python3 scripts/build_whitepaper.py --install
qpdf --check Ostadix-lang_Technical_Whitepaper.pdf
pdfinfo Ostadix-lang_Technical_Whitepaper.pdf
```

G.2 Pin the source and qualify the Rust workspace

```
git rev-parse HEAD HEAD^{tree}
# Runtime source basis: f472c983deb83d260d4d52e77aa4dd8740710dc5
# Tree: 627afd3ee65c829a3edce3e3bd6ba86db4660017

cargo fmt --all --check
cargo check --workspace --all-targets --all-features --locked
cargo clippy --workspace --all-targets --all-features --locked -- -D warnings
CARGO_PROFILE_DEV_DEBUG=0 CARGO_PROFILE_TEST_DEBUG=0 \
```

```
CARGO_INCREMENTAL=0 OSTADIX_TEST_RUNTIME_POLICY=required \
  cargo test --workspace --all-targets --all-features --locked
cargo build --release --workspace --bins --all-features --locked
```

G.3 Run the language and unified front door

```
./target/release/O examples/hello.O backends
./target/release/o-cli run examples/hello.O
./target/release/o-cli plan examples/hello.O
./target/release/o-cli explain last-run
./target/release/o-cli inspect last-run

for bin in O o-cli olangc ocorec o-link o-unlink ogit \
  o-live-host o-node octl o-registry o-info ostadix-device \
  ocore-kernel-world-record
do
  "./target/release/$bin" --help
done
```

The notebook owns a positional launch surface and can be exercised through its loopback API:

```
./target/release/o-notebook backends
curl -fsS http://127.0.0.1:8888/
curl -fsS -X POST http://127.0.0.1:8888/eval \
  -H 'content-type: application/json' \
  --data '{"code":"text^(42)_text"}'
curl -fsS -X POST http://127.0.0.1:8888/reset
```

G.4 Qualify the independent engine and MCP

```
cargo test -p ostadix-api --all-targets --all-features --locked

cargo build --release --locked \
  --manifest-path mcp/ostadix_lang_mcp_server/Cargo.toml
cargo clippy --all-targets --locked \
  --manifest-path mcp/ostadix_lang_mcp_server/Cargo.toml -- -D warnings
cargo test --locked \
  --manifest-path mcp/ostadix_lang_mcp_server/Cargo.toml
python3 scripts/smoke_ostadix_mcp.py
rg -n '^\s*#\[tool\(' mcp/ostadix_lang_mcp_server/src/main.rs
```

G.5 Generate and read the o-linked Project HGraph

```
mkdir -p output/current-master
./target/release/o-link tests/fixtures/project_hgraph --project \
  -o output/current-master/o-linked-project.O

./target/release/olangc output/current-master/o-linked-project.O \
  --target ir --route main --shim-dir backends \
```

```
  > output/current-master/o-linked-project.ir
./target/release/olangc output/current-master/o-linked-project.O \
  --target dot --route main --shim-dir backends \
  > output/current-master/o-linked-codebase-hgraph.dot
./target/release/olangc tests/fixtures/project_hgraph \
  --target dot --route main --shim-dir backends \
  > output/current-master/project-hgraph-direct.dot
cmp output/current-master/o-linked-codebase-hgraph.dot \
    output/current-master/project-hgraph-direct.dot
cp output/current-master/o-linked-codebase-hgraph.dot \
  docs/figures/o-linked-codebase-hgraph.dot
dot -Tsvg output/current-master/o-linked-codebase-hgraph.dot \
  -o docs/figures/o-linked-codebase-hgraph.svg
dot -Tpdf output/current-master/o-linked-codebase-hgraph.dot \
  -o docs/figures/o-linked-codebase-hgraph.pdf
```

G.6 Run Project HGraph, Live-World, and World gates

```
./scripts/smoke-project-hgraph.sh
./scripts/smoke-project-hgraph-exec.sh
./scripts/smoke-hosted-live-reference.sh

python3 -m unittest -v \
  tests.test_release_evidence \
  tests.test_source_release \
  tests.test_example_manifest

python3 scripts/release_evidence.py validate
./boot-and-test.sh smoke
```

The aggregate reads the current 26-gate portable projection from `evidence/gates.toml`. Each registered script emits the markers attached to its gate entry.

G.7 Run compiler targets

```
./target/release/olangc examples/hello.O --target script \
  --shim-dir backends
./target/release/olangc examples/hello.O --target ir \
  --shim-dir backends > output/current-master/hello.ir
./target/release/olangc examples/hello.O --target dot \
  --shim-dir backends > output/current-master/hello.dot
./target/release/olangc examples/hello.O --target binary \
  --shim-dir backends -o output/current-master/hello-host
./output/current-master/hello-host
./target/release/olangc examples/hello.O --target wasm \
  --shim-dir backends -o output/current-master/hello.wasm
# Direct WASI execution uses an inline-only program:
./target/release/olangc examples/wasm_hello.O --target wasm \
  -o output/current-master/inline.wasm
wasmtime run output/current-master/inline.wasm
# Hosted effects require an explicit provider or the Linux-image route.
```

G.8 Run the Python and C17 editions

```
python3 -m tests.test_parser
python3 -m tests.test_evaluator
python3 -m compileall -q o_lang

make -C c_cpp
make -C c_cpp test
make -C c_cpp olangc-test
make -C c_cpp warnings-as-errors
cmake -S c_cpp -B target/c17-cmake
cmake --build target/c17-cmake
ctest --test-dir target/c17-cmake --output-on-failure
```

G.9 Inspect Information, Registry, and node surfaces

```
./target/release/o-info --help
./target/release/o-registry --help
./target/release/o-node --help
./target/release/octl --help
```

The type-level and state-transition suites for Information, Bridge, Provenance, Registry, pairing, TLS, Project Mesh, and World run inside the all-target, all-feature Rust matrix above.

H Implementation and qualification boundaries

These boundaries specify the present source contracts and the execution evidence needed to extend them.

Surface	Exact boundary
Source and installed tools	Source describes the pinned commit. Installed binaries have their own build identity; source support and an installation's option support must be checked separately.
Native front door	Compiled `o` owns operational routing and shared discovery. The compatibility script launches that owner. Catalog discovery alone proves neither installed flag support nor backend readiness.
Planning and grounding	Logical/deployment identities and descriptive requirements are inspectable without dispatch. Realization selection and project grounding grant no World, placement, provider, or task authority.
Execution records	Local run records bind retained content and original decisions. Their trust comes from the producing process/store; signatures or remote attestation require their own explicit protocol.
Managed MCP work	Jobs and one-use intent handles belong to a server session. Job creation precedes completion, output pages are bounded, and server restart does not restore execution. Selected-node client cancellation cannot undo remote effects; ambiguous completion is not automatically retried.
Concurrency	Trusted pure publication, deterministic settlement, and explicit autonomous effects have separate rules. Regression observations do not prove unrestricted contextual equivalence of arbitrary effectful schedules.

Surface	Exact boundary
Execution Fabric	Authenticated remote attempts support the admitted pure renderer profile. Graph publication remains coordinator-owned. Broader remote OIR, automatic placement, and external-effect retry require new contracts.
Physical graphs	Automatic OIR physical execution uses admitted local operation graphs and checked local socket transfers. It supplies no general automatic remote placement.
Native objects and migration	Live Python objects remain in their exact owner. Supported idle Python/SQLite state can migrate through acknowledged checkpoint protocols. Live handles, in-flight work, and arbitrary external resources are outside that transfer.
Wasm and runtime bundles	Direct WASI supports inline plans; hosted effects need an explicit provider. The separate Linux-image route packages supplied runtimes in an emulated guest. A supplied image or runtime tree requires its own closure and execution qualification.
Information and Registry	Information projections carry bounded descriptive metadata. Provenance admission recomputes exact images and obligations. Registry V1 verifies local signed event chains; it supplies no independent network publication service.
World formats	Canonical identities and receipt bytes carry descriptive rights. Signer enrollment, revocation, and current Governor authority come from the consuming policy.
Native qualification	Portable QEMU, hardware KVM, Linux guest, physical boot, Secure Boot, measured boot, PCI/DMA/IOMMU, and cross-machine execution require their own evidence. Passing one profile does not complete World-alpha.
Current test scope	Section 14 records retained host and native repair runs at their actual source coordinates. Document compilation and review do not rerun those execution matrices or qualify other substrates.

[docs/HUMAN_LLM_WORKFLOWS.md] [docs/OPERATION_PLANNING_V1.md] [docs/EXECUTION_OBSERVATION_CONTRACT.md] [docs/OIR_EXECUTION_FABRIC_V1.md] [docs/OIR_PHYSICAL_EXECUTION_V1.md] [docs/HOSTED_ACTOR_MIGRATION.md] [docs/OLANGC_WASM.md] [docs/NATIVE_DISTRIBUTED_LINUX_STATUS.md]